UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

FACULTAD DE INGENIERÍA EN SISTEMAS DE LA INFORMACIÓN

Y CIENCIAS DE LA COMPUTACIÓN

PROYECTO FINAL: TRANSMETRO

LUIS CARLOS BARRERA RAMIREZ

090-22-9525

GUATEMALA, JULIO DE 2026

Índice

Table de Contenido

1. Carta De Presentación 1

2. Antecedentes del problema 2

3. Definición de la Situación Actual 4

4. Deficiencias de los Controles Actuales 6

5. Descripción de la Planificación 8

5.1 Diagrama de Gantt – Proyecto Sistema de Control y Gestion de Transmetro 12

6. Análisis de Requerimientos 13

7. ESTUDIOS DE FACTIBILIDAD 30

7.1 Factibilidad técnica 30

7.2 Factibilidad operativa 35

7.3 Factibilidad económica 38

7.4 Factibilidad legal 40

7.5 Factibilidad ambiental 42

7.6 Resultado general de factibilidad 44

8. PROPUESTA MONETARIA DE LA SOLUCIÓN 44

8.1 Propuesta No. 1 45

8.2 Desarrollo del software 46

8.3 Alojamiento en Microsoft Azure 47

8.4 Total de la propuesta No. 1 49

8.5 Propuesta No. 2 49

8.6 Comparación de las propuestas 50

8.7 Elementos no incluidos 51

8.8 Vigencia de la propuesta 52

8.9 Forma de pago 53

8.10 Garantía 53

9. Modelo Entidad Relación 54

10. Script de la Base de Datos 54

11. Diagramas UML 112

11.1 Caso de Uso 112

11.2 Diagrama de Componentes 114

11.3 Diagrama de Secuencia 114

11.4 Diagrama de Actividades 115

11.5 Diagrama de Clases 115

11.6 Diagrama de Objetos 116
12. Presentación de Stack Tecnológico 116
13. Métricas de Software 119
14. demo funcional – aplicación web 124
15. Plan De Pruebas 127
15.1 Casos de prueba de acceso y configuración 130
15.2 Casos de prueba de operación y recorridos 132
15.3 Casos de prueba de tarjetas y portal ciudadano 135
15.4 Casos de prueba transversales 137
16. Criterios de Aceptación 139
16.1 Criterios de infraestructura y administración 139
16.2 Criterios de operación y control de pasajeros 140
16.3 Criterios de tarjetas y atención al ciudadano 141
16.4 Criterios transversales de calidad y seguridad 142
17. Carta de Aceptación de Proyecto 145
18. Enlace A Repositorio En GitHub 147
19. Enlace Al Sistema En La Web 147
20. Enlace Al Video Del Sistema 147

1. CARTA DE PRESENTACIÓN

Guatemala, 28 de agosto de 2026

Señores
Municipalidad de Guatemala
Dirección de Movilidad Urbana / Administración del Sistema Transmetro
Ciudad de Guatemala

Asunto: Presentación de propuesta de proyecto para el desarrollo de un sistema de control y gestión de información del Transmetro

Respetables señores:

Mi nombre es Luis Carlos Barrera Ramírez, y me desempeño como consultor en soluciones tecnológicas y desarrollo de software. En el marco de mis servicios profesionales, se presenta la propuesta orientada al análisis, diseño y desarrollo de una solución informática que permita fortalecer los procesos de control, organización y administración de la información relacionada con la operación del sistema Transmetro.

Para el presente proyecto se ha tomado como referencia el sistema Transmetro, debido a la importancia que este servicio representa dentro de la movilidad urbana de la Ciudad de Guatemala. La propuesta se enfoca específicamente en la gestión de información relacionada con las líneas de servicio, estaciones, accesos, buses asignados, recorridos, parqueos, pilotos, personal de seguridad, operadores y otros elementos necesarios para mantener un adecuado control operativo.

Para el desarrollo de la solución se realizará un análisis de requerimientos, estudio de la situación actual, identificación de deficiencias, evaluación de factibilidad y planificación de las diferentes actividades necesarias para el desarrollo del sistema. Posteriormente, se elaborarán los modelos y diagramas correspondientes, se diseñará la base de datos y se desarrollará una aplicación web funcional que permita demostrar la propuesta planteada.

Agradezco de antemano la atención brindada a la presente propuesta y manifiesto mi compromiso de desarrollar el proyecto con responsabilidad, profesionalismo y apego a los objetivos académicos establecidos.

Sin otro particular, me suscribo de ustedes.

Atentamente,

Luis Carlos Barrera Ramírez
Consultor de Ingeniería de Software

2. ANTECEDENTES DEL PROBLEMA

El crecimiento urbano y el aumento de las necesidades de movilidad dentro de la Ciudad de Guatemala han generado la necesidad de implementar diferentes alternativas de transporte que permitan movilizar a la población de una manera más ordenada, segura y eficiente. Dentro de este contexto, el Transmetro representa uno de los sistemas de transporte público que forma parte de la movilidad urbana de la ciudad, contando con una estructura basada en líneas, rutas y estaciones distribuidas en diferentes sectores.

La información oficial publicada por la Municipalidad de Guatemala permite observar que Transmetro cuenta con recursos de consulta relacionados con mapas, estaciones, rutas, simbología e información por línea. Esto demuestra que el servicio requiere una organización estructurada de los elementos que forman parte de su funcionamiento, especialmente debido a la relación existente entre las líneas de transporte y las diferentes estaciones que conforman cada recorrido.

Sin embargo, el funcionamiento de un sistema de transporte de esta naturaleza implica la administración de una cantidad considerable de información. No solamente es necesario conocer qué líneas existen o cuáles son las estaciones disponibles, sino también mantener un control sobre el orden en que estas estaciones son visitadas durante cada recorrido.

De acuerdo con el planteamiento proporcionado para el proyecto, una línea está compuesta por una serie de estaciones organizadas en un orden específico, por lo que resulta importante almacenar tanto la información de la ruta como las visitas o recorridos que realiza cada unidad. Además, una estación puede pertenecer a una o varias líneas, lo que genera una relación que debe ser correctamente administrada dentro de un sistema de información.

También existe la necesidad de controlar los accesos de cada estación. Una estación puede contar con varios accesos, mientras que cada acceso pertenece únicamente a una estación. Esta información es relevante porque el proyecto requiere conocer todos los accesos disponibles para cada línea y mantener control sobre los recursos y el personal asignado a dichos puntos.

Otro antecedente importante se encuentra relacionado con la administración de la flota de buses. Cada línea debe tener asignada una serie de buses, aunque un bus solamente puede estar asignado a una línea a la vez. También se establecen reglas respecto a la cantidad de buses que una línea puede tener en relación con el número de estaciones que la conforman. Además,

algunos puntos pueden contar con parqueos y cada bus debe tener un parqueo asignado, pudiendo cambiar de ubicación sin quedar sin una asignación.

El recurso humano constituye otro elemento fundamental dentro de los antecedentes del problema. Se establece la necesidad de almacenar información sobre los pilotos, incluyendo datos relacionados con su residencia, comunicación e historial educativo. Además, las estaciones deben contar con personal de seguridad, considerando como mínimo un guardia por cada acceso, y también se contempla la existencia de un operador encargado del equipo de trabajo disponible en cada estación.

A nivel operativo, también se identifica situaciones relacionadas con la demanda de pasajeros. Se plantea que los buses tienen una capacidad máxima y que, cuando la cantidad de personas presentes en una estación supera determinados niveles establecidos, debe generarse una alerta para apoyar la llegada de otra unidad. De igual manera, se establece una regla relacionada con el nivel de ocupación de un bus y el tiempo de espera adicional en las estaciones cuando no se alcance un porcentaje mínimo de su capacidad.

Otro aspecto importante identificado en el planteamiento inicial es que todavía debe determinarse la mejor forma de conectar los equipos de trabajo ubicados en los diferentes puntos. Además, se menciona la necesidad de considerar los métodos de pago y las tarjetas de acceso como parte de los elementos que intervienen en el sistema.

Por lo tanto, el antecedente principal del problema se encuentra en la necesidad de contar con una propuesta que permita estructurar y centralizar la información necesaria para controlar los diferentes elementos que intervienen en la operación de un sistema como Transmetro. La diversidad de entidades involucradas, tales como municipalidades, líneas, estaciones, accesos, buses, parqueos, pilotos, guardias, operadores y pasajeros, hace necesario establecer una solución informática que permita relacionar correctamente estos elementos.

3. Definición de la Situación Actual

Actualmente, Transmetro forma parte de la movilidad urbana de la Ciudad de Guatemala y cuenta con información pública relacionada con sus rutas, estaciones y líneas. La plataforma oficial de la Municipalidad de Guatemala presenta recursos de consulta como mapas y apartados de información por línea, lo que permite a los usuarios conocer la estructura general del servicio. Además, la Municipalidad proporciona información de contacto institucional relacionada con sus servicios.

Sin embargo, el problema no consiste únicamente en conocer las rutas existentes, sino en determinar cómo se puede administrar de manera integral la información relacionada con todos los elementos que intervienen en la operación.

La situación planteada requiere controlar información de las líneas y mantener el orden específico de las estaciones que forman parte de cada recorrido. Esto es importante debido a que una estación puede formar parte de más de una línea, por lo que una simple lista de estaciones no sería suficiente. Es necesario establecer relaciones que permitan conocer a qué líneas pertenece cada estación y cuál es su posición dentro del recorrido correspondiente.

De igual manera, se requiere conocer los accesos existentes en cada estación. Cada acceso debe estar relacionado con una única estación, pero una estación puede disponer de varios puntos de acceso. Esta situación genera la necesidad de contar con un control detallado que permita identificar no solamente las estaciones existentes, sino también su estructura y los accesos disponibles.

En cuanto a los buses, la situación actual requiere conocer qué unidades se encuentran asignadas a cada línea. También deben controlarse las restricciones relacionadas con la cantidad mínima y máxima de buses permitidos en función del número de estaciones. Esto requiere que el sistema sea capaz de validar la información ingresada y evitar asignaciones que incumplan las reglas establecidas.

Respecto al personal, Se requiere almacenar información suficiente sobre los pilotos para poder mantener un registro organizado de sus datos. Esto incluye información personal, residencia, medios de comunicación e historial educativo. Además, debe controlarse el personal de seguridad asignado a los accesos y los operadores responsables de utilizar las computadoras de trabajo instaladas en las estaciones.

Otro elemento importante es el control de la demanda de pasajeros. El caso plantea que deben existir mecanismos para detectar situaciones en las cuales una estación presenta una cantidad elevada de usuarios y generar una alerta que permita responder operativamente.

Además, se identifica una situación pendiente relacionada con la infraestructura tecnológica. El proyecto indica que se espera contar con una computadora de trabajo en cada estación, pero todavía no se ha determinado cuál sería la mejor forma de conectar estos equipos entre los diferentes puntos. Esta situación es importante porque una solución de gestión centralizada requiere que la información pueda ser consultada y actualizada de manera controlada desde los distintos lugares donde se realicen las operaciones.

En consecuencia, puede resumirse como una necesidad de integración y centralización de la información operativa. Existen múltiples elementos que deben ser controlados y relacionados entre sí, pero el caso de estudio requiere proponer una solución que permita administrar dicha información de una forma estructurada y que facilite su consulta.

La solución propuesta consistirá, por lo tanto, en una aplicación web respaldada por una base de datos, mediante la cual se pueda registrar y consultar información sobre las líneas, estaciones, accesos, buses, parqueos y personal. Además, se buscará incorporar controles y validaciones que permitan asegurar la integridad de los datos y apoyar la generación de reportes.

4. Deficiencias de los Controles Actuales

A partir del análisis del caso de estudio, se pueden identificar diferentes áreas en las que resulta necesario fortalecer o definir controles mediante una solución informática.

Una de las principales deficiencias se relaciona con la necesidad de centralizar la información. El sistema involucra una gran cantidad de elementos, incluyendo líneas, estaciones, accesos, buses, parqueos, pilotos, guardias y operadores. Si esta información no se encuentra organizada dentro de una estructura centralizada, puede resultar más difícil mantener registros actualizados, realizar consultas y generar reportes confiables.

También se identifica una deficiencia relacionada con el control del orden de las estaciones dentro de cada línea. No basta con registrar qué estaciones pertenecen a una línea; es necesario conocer la secuencia en la que estas son recorridas. Además, una misma estación puede pertenecer a varias líneas, por lo que se requiere un modelo que permita administrar correctamente estas relaciones y evitar inconsistencias en los datos.

Otra deficiencia se encuentra en la necesidad de establecer controles sobre la asignación de buses. El proyecto define reglas específicas respecto a la relación entre el número de estaciones y la cantidad de buses asignados a una línea. Sin un mecanismo automatizado de validación, podría existir el riesgo de realizar asignaciones que no cumplan con las condiciones establecidas.

La gestión de parqueos también requiere controles adecuados. Debido a que cada bus debe tener un parqueo asignado y puede cambiar de ubicación, es necesario contar con un registro actualizado que permita identificar la asignación vigente de cada unidad. La ausencia de un control estructurado podría dificultar el seguimiento de la ubicación operativa de los vehículos.

En relación con los recursos humanos, se identifica la necesidad de mantener información organizada sobre pilotos, personal de seguridad y operadores. La información de los pilotos incluye aspectos personales, residencia, comunicación e historial educativo, mientras que el personal de seguridad debe estar relacionado con los accesos de las estaciones. La falta de una estructura centralizada podría provocar información incompleta, duplicada o difícil de consultar.

Otra deficiencia importante es la necesidad de implementar mecanismos de control sobre la capacidad de pasajeros. El proyecto establece condiciones bajo las cuales debe

generarse una alerta cuando una estación alcanza una concentración elevada de usuarios. Si este proceso depende únicamente de observación manual o comunicación informal, la respuesta podría ser más lenta y dificultar la toma oportuna de decisiones.

De igual manera, se requiere un mecanismo que permita controlar la ocupación de los buses y aplicar la regla de espera establecida cuando una unidad no alcanza el porcentaje mínimo definido. Esto evidencia la necesidad de que el sistema pueda manejar reglas de negocio y no limitarse únicamente al almacenamiento de información.

De la misma forma, existe la necesidad de integrar o considerar la información relacionada con los métodos de pago y las tarjetas de acceso. Esto representa una oportunidad para analizar cómo podría relacionarse con el sistema propuesto, especialmente desde el punto de vista del control y registro de información.

En términos generales, las principales deficiencias identificadas para el proyecto son las siguientes:

- Necesidad de centralizar la información relacionada con la operación.
- Falta de un modelo académico integrado para relacionar líneas, estaciones y recorridos.
- Necesidad de controlar el orden de las estaciones dentro de cada ruta.
- Necesidad de validar las asignaciones de buses según las reglas establecidas.
- Falta de un mecanismo definido para controlar los cambios de parqueo.
- Necesidad de mantener información estructurada sobre pilotos, guardias y operadores.
- Necesidad de automatizar alertas relacionadas con la concentración de pasajeros.
- Necesidad de aplicar controles relacionados con la ocupación de los buses..
- Necesidad de generar reportes que permitan consultar información sobre estaciones, líneas y buses.

La propuesta de solución buscará atender estas deficiencias mediante el desarrollo de una aplicación web centralizada, con una base de datos estructurada y reglas de validación. De esta manera, la información podrá ser registrada, consultada y relacionada de forma más organizada, facilitando además la generación de reportes y el seguimiento de los diferentes elementos involucrados en el caso de estudio.

5. DESCRIPCIÓN DE LA PLANIFICACIÓN

La planificación del proyecto se ha estructurado en diferentes fases que permiten organizar de manera progresiva las actividades necesarias para el análisis, diseño, desarrollo, evaluación y entrega de la solución propuesta para el sistema de control y gestión de información del Transmetro. Cada fase agrupa actividades relacionadas entre sí y cuenta con fechas específicas de entrega, permitiendo establecer un seguimiento ordenado del avance del proyecto.

El proyecto se desarrollará utilizando la metodología ágil Scrum, tomando como principio de trabajo la realización de entregas incrementales y revisiones continuas de los resultados obtenidos. Debido a que el cronograma establecido contempla diferentes entregables distribuidos entre el 28 de agosto y el 23 de octubre de 2026, las actividades serán organizadas de acuerdo con los avances y productos esperados en cada etapa. Al finalizar cada período de trabajo se realizará una revisión de los resultados para identificar posibles errores, realizar ajustes y asegurar que los entregables cumplan con los objetivos establecidos.

Fase 1. Diagnóstico y Planteamiento del Proyecto

La primera fase corresponde al diagnóstico y planteamiento del proyecto y comprende los entregables correspondientes a la carta de presentación personal, los antecedentes del problema, la definición de la situación actual, las deficiencias de los controles actuales y la planificación. Esta fase tiene como propósito establecer el contexto de la problemática que se pretende abordar, identificar las necesidades existentes y definir formalmente el alcance inicial de la propuesta.

Los primeros entregables tienen una fecha establecida para el 28 de agosto de 2026, por lo que esta fase permitirá establecer las bases sobre las cuales se desarrollarán las siguientes etapas del proyecto.

Fase 2. Análisis de Requerimientos

La segunda fase corresponde al análisis de requerimientos y tiene como fecha de entrega el 4 de septiembre de 2026. Durante esta etapa se identificarán y documentarán los requerimientos funcionales y no funcionales que deberá cumplir la solución propuesta.

En esta fase se definirán las principales funcionalidades relacionadas con la administración de líneas, estaciones, accesos, buses, parqueos, pilotos, personal de seguridad, operadores, pagos y generación de reportes. También se establecerán las condiciones de

calidad, seguridad, rendimiento, disponibilidad, integridad y mantenibilidad que deberá cumplir el sistema.

El resultado esperado será contar con una especificación clara de las necesidades que deberá satisfacer la aplicación, sirviendo como base para las etapas posteriores de diseño, desarrollo y pruebas.

Fase 3. Factibilidad y Propuesta de Solución

La tercera fase comprende los estudios de factibilidad y la propuesta monetaria de la solución, ambos establecidos para el 11 de septiembre de 2026.

Durante esta etapa se analizará la viabilidad de implementar la solución desde diferentes perspectivas. Se considerará la factibilidad técnica, operativa, económica, legal y ambiental, con el propósito de determinar si la propuesta puede ser desarrollada e implementada utilizando los recursos tecnológicos y organizativos considerados para el proyecto.

Posteriormente se elaborará la propuesta monetaria, tomando en consideración los recursos necesarios para desarrollar la solución, incluyendo herramientas, infraestructura, desarrollo, implementación y demás elementos que sean necesarios. Esta fase permitirá establecer una visión más completa de la solución antes de iniciar su diseño técnico.

Fase 4. Diseño de la Solución

La cuarta fase corresponde al diseño de la solución y comprende los entregables programados para el 18 de septiembre de 2026: el modelo entidad-relación, el script para la creación de las estructuras de la base de datos y los diagramas UML completos.

En esta etapa se diseñará la estructura de información que utilizará el sistema, definiendo las entidades, atributos y relaciones necesarias para administrar los datos de líneas, estaciones, accesos, buses, parqueos, municipalidades y personal.

También se elaborarán los diferentes diagramas UML requeridos para representar el comportamiento y la estructura de la solución. Estos incluirán los diagramas de casos de uso, componentes, secuencia, actividades, clases y objetos. El objetivo será contar con una representación técnica que sirva como guía para el desarrollo de la aplicación web.

Fase 5. Definición Tecnológica y Calidad

La quinta fase corresponde a la definición tecnológica y calidad, con fecha de entrega establecida para el 2 de octubre de 2026. Esta fase contempla la presentación del stack tecnológico y las métricas de software.

Durante esta etapa se definirán las tecnologías que serán utilizadas para construir la solución, incluyendo la base de datos, metodología, lenguaje de programación, framework y entorno de desarrollo. La selección se realizará considerando las necesidades del sistema, la facilidad de mantenimiento, seguridad, escalabilidad y disponibilidad de las herramientas.

También se establecerán las métricas que permitirán evaluar diferentes aspectos de la calidad del software. Estas métricas servirán posteriormente para determinar si la solución cumple con los niveles de calidad esperados y facilitarán la evaluación objetiva del producto desarrollado.

Fase 6. Desarrollo y Demostración Funcional

La sexta fase corresponde al desarrollo y demostración funcional de la aplicación web, cuya fecha está establecida para el 10 de octubre de 2026.

Durante esta etapa se implementará la aplicación web tomando como base los requerimientos y diseños definidos en las fases anteriores. Se desarrollarán progresivamente las funcionalidades principales del sistema, procurando que cada módulo cumpla con los requerimientos establecidos.

La demostración funcional permitirá comprobar el funcionamiento general de la aplicación y verificar que las principales operaciones puedan ejecutarse correctamente. También permitirá identificar posibles ajustes antes de iniciar la etapa formal de pruebas.

Fase 7. Pruebas, Aceptación y Validación

La séptima fase corresponde a pruebas, aceptación y validación, con fecha de entrega establecida para el 17 de octubre de 2026. Esta etapa comprende el plan de pruebas, los criterios de aceptación y la carta de aceptación del proyecto.

Durante esta fase se ejecutarán diferentes pruebas sobre la aplicación con el objetivo de comprobar que las funcionalidades desarrolladas cumplan con los requerimientos establecidos. Se validarán especialmente las reglas de negocio relacionadas con las líneas, estaciones, buses, capacidad, alertas, asignaciones y demás procesos definidos.

Los errores o comportamientos inesperados identificados durante las pruebas serán analizados y corregidos antes de considerar finalizada la solución. Asimismo, los criterios de aceptación permitirán establecer las condiciones necesarias para determinar si cada componente desarrollado cumple con lo esperado.

Finalmente, la carta de aceptación representará la confirmación del cumplimiento de los objetivos y condiciones establecidos para el proyecto.

Fase 8. Cierre y Entrega Final

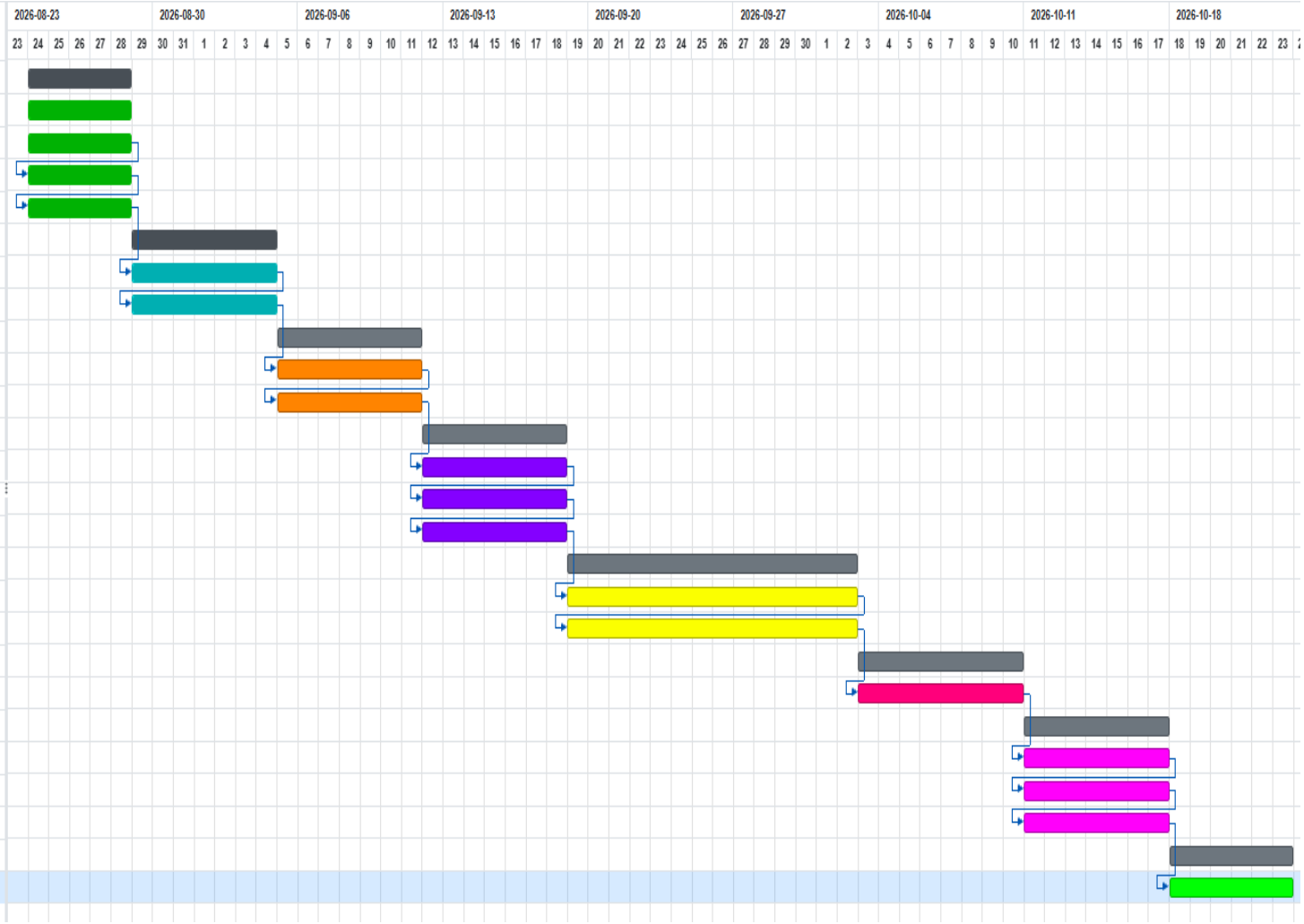
La última fase corresponde al cierre y entrega final del proyecto, programada para el 23 de octubre de 2026.

En esta etapa se realizará una revisión general de los componentes desarrollados y de la documentación generada durante todo el proyecto. Se verificará que los entregables se encuentren completos y organizados, incluyendo la documentación, aplicación web, base de datos, diagramas, pruebas, métricas y demás elementos solicitados.

También se realizará la organización final del repositorio del proyecto y se comprobará que los recursos necesarios para la revisión y presentación se encuentren disponibles. Con esta actividad se dará por concluido el desarrollo de la propuesta y se realizará la entrega formal del proyecto.

5.1 Diagrama de Gantt – Proyecto Sistema de Control y Gestion de Transmetro

ID	Task Name	Start	End	Duration
1	Fase 1. Diagnóstico y Planteamiento del Proy...	2026-08-24	2026-08-28	5 days
2	Carta de presentación personal	2026-08-24	2026-08-28	5 days
4	Definición de la situación actual	2026-08-24	2026-08-28	5 days
5	Deficiencias de los controles actuales	2026-08-24	2026-08-28	5 days
6	Planificación	2026-08-24	2026-08-28	5 days
7	Fase 2. Análisis de Requerimientos	2026-08-29	2026-09-04	7 days
8	Requerimiento Funcionales	2026-08-29	2026-09-04	7 days
9	Requerimientos No Funcionales	2026-08-29	2026-09-04	7 days
13	Fase 3. Factibilidad y Propuesta de Solución	2026-09-05	2026-09-11	7 days
14	Propuesta Monetaria de su Solución	2026-09-05	2026-09-11	7 days
15	Estudios de Factibilidad	2026-09-05	2026-09-11	7 days
19	Fase 4. Diseño de la Solución	2026-09-12	2026-09-18	7 days
20	Modelo Entidad Relación	2026-09-12	2026-09-18	7 days
21	Script de la BD	2026-09-12	2026-09-18	7 days
49	Diagramas UML	2026-09-12	2026-09-18	7 days
25	Fase 5. Definición Tecnológica y Calidad	2026-09-19	2026-10-02	14 days
26	Presentación de Stack Tecnológico	2026-09-19	2026-10-02	14 days
27	Métricas de Software	2026-09-19	2026-10-02	14 days
31	Fase 6. Desarrollo y Demostración Funcional	2026-10-03	2026-10-10	8 days
32	Demo funcional - Aplicación web	2026-10-03	2026-10-10	8 days
37	Fase 7. Pruebas, Aceptación y Validación	2026-10-11	2026-10-17	7 days
38	Plan de Pruebas	2026-10-11	2026-10-17	7 days
39	Criterios de Aceptación	2026-10-11	2026-10-17	7 days
40	Carta de Aceptación de proyecto	2026-10-11	2026-10-17	7 days
43	Fase 8. Cierre y Entrega Final	2026-10-18	2026-10-23	6 days
44	Entrega Final	2026-10-18	2026-10-23	6 days



6. ANÁLISIS DE REQUERIMIENTOS

Requerimientos Funcionales

REQ001 — Registro de líneas

Id. Requerimiento	*RF-01*	Clasificación o prioridad	*MUY ALTA*
Nombre Requerimiento	*Registro de líneas de Transmetro*		
Referencia	*Gestión de líneas y recorridos*		

No. Func.	Funcionalidad
001	El sistema deberá permitir registrar una nueva línea de Transmetro.
002	El sistema deberá permitir ingresar el nombre o identificador de la línea.
003	El sistema deberá permitir asociar una o varias estaciones a una línea.
004	El sistema deberá permitir establecer el orden en que cada estación es visitada durante el recorrido de la línea.
005	El sistema deberá permitir consultar las estaciones que conforman una línea en el orden correspondiente.

REQ002 — Asociación de estaciones a líneas

Id. Requerimiento	*RF-02*	Clasificación o prioridad	*MUY ALTA*
Nombre Requerimiento	*Asociación de estaciones a líneas*		
Referencia	*Gestión de líneas y estaciones*		

No. Func.	Funcionalidad
001	El sistema deberá permitir asociar una estación a una línea de Transmetro.
002	El sistema deberá permitir que una misma estación pertenezca a una o varias líneas.
003	El sistema deberá permitir consultar las líneas a las que pertenece una estación.
004	El sistema deberá evitar asociaciones duplicadas entre una misma estación y una línea.

REQ003 — Registro de estaciones

Id. Requerimiento	*RF-03*	Clasificación o prioridad	*MUY ALTA*
Nombre Requerimiento	*Registro de estaciones*		
Referencia	*Gestión de estaciones*		

No. Func.	Funcionalidad
001	El sistema deberá permitir registrar nuevas estaciones de Transmetro.
002	El sistema deberá permitir registrar el nombre o identificador de cada estación.
003	El sistema deberá permitir asociar cada estación a una municipalidad.
004	El sistema deberá permitir consultar la información de las estaciones registradas.
005	El sistema deberá permitir actualizar la información de una estación registrada.

REQ004 — Registro de accesos

Id. Requerimiento	*RF-04*	Clasificación o prioridad	*ALTA*
Nombre Requerimiento	*Registro de accesos de estaciones*		
Referencia	*Gestión de estaciones y seguridad*		

No. Func.	Funcionalidad
001	El sistema deberá permitir registrar los accesos correspondientes a cada estación.
002	El sistema deberá permitir identificar cada acceso mediante un código o identificador único.
003	El sistema deberá garantizar que cada acceso pertenezca a una única estación.
004	El sistema deberá permitir consultar todos los accesos registrados de una estación.
005	El sistema deberá permitir actualizar la información de los accesos registrados.

REQ005 — Asignación de buses

Id. Requerimiento	*RF-05*	Clasificación o prioridad	*MUY ALTA*
Nombre Requerimiento	*Asignación de buses a líneas*		
Referencia	*Gestión de flota*		

No. Func.	Funcionalidad
001	El sistema deberá permitir registrar los buses que forman parte de la flota.
002	El sistema deberá permitir asignar un bus a una línea determinada.
003	El sistema deberá garantizar que un bus no pueda estar asignado simultáneamente a más de una línea.
004	El sistema deberá permitir que un bus permanezca temporalmente sin una línea asignada.
005	El sistema deberá permitir consultar los buses asignados a cada línea.

REQ006 — Validación de cantidad de buses

Id. Requerimiento	*RF-06*	Clasificación o prioridad	*MUY ALTA*
Nombre Requerimiento	*Validación de buses por línea*		
Referencia	*Control de flota*		

No. Func.	Funcionalidad
001	El sistema deberá determinar la cantidad de estaciones que posee cada línea.
002	El sistema deberá validar que cada línea tenga como mínimo tantos buses como estaciones tenga.
003	El sistema deberá validar que cada línea tenga como máximo el doble de buses respecto a su cantidad de estaciones.
004	El sistema deberá impedir una asignación que incumpla las cantidades mínima o máxima establecidas.
005	El sistema deberá informar al operador cuando una asignación no cumpla con las reglas establecidas.

REQ007 — Gestión de parqueos

Id. Requerimiento	*RF-07*	Clasificación o prioridad	*ALTA*
Nombre Requerimiento	*Asignación y control de parqueos*		
Referencia	*Gestión de flota y parqueos*		

No. Func.	Funcionalidad
001	El sistema deberá permitir registrar los parqueos disponibles en las estaciones que cuenten con este recurso.
002	El sistema deberá permitir asignar un parqueo a cada bus.
003	El sistema deberá garantizar que un bus siempre mantenga un parqueo asignado.
004	El sistema deberá permitir reasignar un bus a otro parqueo.
005	El sistema deberá permitir consultar el parqueo actualmente asignado a cada bus.
006	El sistema deberá mantener el registro de las reasignaciones de parqueo realizadas.

REQ008 — Consulta de accesos por línea

Id. Requerimiento	*RF-08*	Clasificación o prioridad	*ALTA*
Nombre Requerimiento	*Consulta de accesos por línea*		
Referencia	*Gestión de líneas y estaciones*		

No. Func.	Funcionalidad
001	El sistema deberá permitir seleccionar una línea determinada.
002	El sistema deberá consultar las estaciones asociadas a la línea.
003	El sistema deberá identificar los accesos pertenecientes a cada estación de la línea.
004	El sistema deberá mostrar todos los accesos correspondientes a la línea seleccionada.

REQ009 — Registro de pilotos

Id. Requerimiento	*RF-09*	Clasificación o prioridad	*ALTA*
Nombre Requerimiento	*Registro de información de pilotos*		
Referencia	*Gestión de personal*		

No. Func.	Funcionalidad
001	El sistema deberá permitir registrar la información personal de cada piloto.
002	El sistema deberá permitir registrar el historial educativo del piloto.
003	El sistema deberá permitir registrar los datos de residencia del piloto.
004	El sistema deberá permitir registrar los datos de comunicación del piloto.
005	El sistema deberá permitir consultar y actualizar la información registrada de cada piloto.

REQ010 — Control de guardias

Id. Requerimiento	*RF-10*	Clasificación o prioridad	*ALTA*
Nombre Requerimiento	*Asignación de guardias de seguridad*		
Referencia	*Gestión de seguridad*		

No. Func.	Funcionalidad
001	El sistema deberá permitir registrar al personal de seguridad.
002	El sistema deberá permitir asignar guardias de seguridad a los accesos de las estaciones.
003	El sistema deberá garantizar que cada acceso tenga como mínimo un guardia de seguridad asignado.
004	El sistema deberá permitir consultar los guardias asignados a cada acceso.
005	El sistema deberá permitir actualizar las asignaciones de seguridad.

REQ011 — Gestión de municipalidades

Id. Requerimiento	***RF-11***	**Clasificación o prioridad**	***ALTA***
Nombre Requerimiento	***Asociación de líneas y estaciones a municipalidades***		
Referencia	***Gestión administrativa***		

No. Func.	**Funcionalidad**
001	El sistema deberá permitir registrar las municipalidades involucradas en el sistema.
002	El sistema deberá permitir asociar cada línea a una municipalidad.
003	El sistema deberá permitir asociar cada estación a una municipalidad.
004	El sistema deberá permitir consultar las líneas pertenecientes a una municipalidad.
005	El sistema deberá permitir consultar las estaciones pertenecientes a una municipalidad.

REQ012 — Control de distancias

Id. Requerimiento	***RF-12***	**Clasificación o prioridad**	***ALTA***
Nombre Requerimiento	***Asociación de líneas y estaciones a municipalidades***		
Referencia	***Gestión administrativa***		

No. Func.	**Funcionalidad**
001	El sistema deberá permitir registrar la distancia existente entre estaciones consecutivas.
002	El sistema deberá permitir consultar la distancia entre dos estaciones consecutivas de una línea.
003	El sistema deberá calcular automáticamente la distancia total de una línea a partir de las distancias entre sus estaciones.
004	El sistema deberá permitir consultar la distancia total correspondiente a cada línea.

REQ013 — Alertas por ocupación

Id. Requerimiento	*RF-13*	Clasificación o prioridad	*MUY ALTA*
Nombre Requerimiento	*Alertas de ocupación de estaciones*		
Referencia	*Control de capacidad y demanda*		

No. Func.	Funcionalidad
001	El sistema deberá permitir registrar la capacidad máxima de pasajeros de cada bus.
002	El sistema deberá registrar la cantidad estimada de pasajeros en una estación.
003	El sistema deberá comparar la cantidad de pasajeros registrada con la capacidad establecida.
004	El sistema deberá generar automáticamente una alerta cuando la ocupación alcance o supere el 150% de la capacidad establecida.
005	El sistema deberá indicar que se requiere el envío de una unidad adicional cuando se genere la alerta.
006	El sistema deberá permitir consultar las alertas generadas por ocupación.

REQ014 — Control de espera

Id. Requerimiento	*RF-14*	Clasificación o prioridad	*ALTA*
Nombre Requerimiento	*Control de tiempo de espera por baja ocupación*		
Referencia	*Control operativo de buses*		

No. Func.	Funcionalidad
001	El sistema deberá registrar la ocupación actual de cada bus.
002	El sistema deberá comparar la ocupación del bus con el 25% de su capacidad máxima.
003	El sistema deberá identificar cuando la ocupación sea menor al 25% de la capacidad del bus.
004	El sistema deberá indicar al operador que el bus debe esperar 5 minutos adicionales en la estación.
005	El sistema deberá registrar la aplicación de la regla de espera correspondiente.

REQ015 — Gestión de operadores

Id. Requerimiento	*RF-15*	Clasificación o prioridad	*ALTA*
Nombre Requerimiento	*Asignación de operadores a estaciones*		
Referencia	*Gestión de estaciones y operadores*		

No. Func.	Funcionalidad
001	El sistema deberá permitir registrar operadores del sistema.
002	El sistema deberá permitir asignar un operador a una estación.
003	El sistema deberá permitir identificar el operador responsable de cada estación.
004	El sistema deberá permitir al operador gestionar las funciones correspondientes a la estación asignada.
005	El sistema deberá permitir actualizar la asignación del operador.

REQ016 — Gestión de tarjetas y pagos

Id. Requerimiento	*RF-16*	Clasificación o prioridad	*MUY ALTA*
Nombre Requerimiento	*Gestión de pagos mediante tarjeta de acceso*		
Referencia	*Control de acceso y pagos*		

No. Func.	Funcionalidad
001	El sistema deberá permitir registrar tarjetas de acceso.
002	El sistema deberá asociar una tarjeta de acceso con su información correspondiente.
003	El sistema deberá permitir registrar pagos realizados mediante tarjeta de acceso.
004	El sistema deberá validar el saldo disponible de la tarjeta antes de registrar un pago.
005	El sistema deberá permitir realizar recargas de saldo.
006	El sistema deberá actualizar el saldo de la tarjeta después de una recarga o pago.
007	El sistema deberá registrar el historial de transacciones realizadas mediante la tarjeta.

REQ017 — Reportes

Id. Requerimiento	*RF-17*	Clasificación o prioridad	*ALTA*
Nombre Requerimiento	*Generación de reportes operativos*		
Referencia	*Gestión y consulta de información*		

No. Func.	Funcionalidad
001	El sistema deberá generar un reporte de las estaciones registradas.
002	El sistema deberá generar un reporte de las líneas registradas.
003	El sistema deberá generar un reporte de los buses asignados a cada línea.
004	El sistema deberá permitir consultar los reportes desde la aplicación web.
005	El sistema deberá mostrar información actualizada en los reportes generados.

REQ018 — Usuarios y roles

Id. Requerimiento	*RF-18*	Clasificación o prioridad	*MUY ALTA*
Nombre Requerimiento	*Gestión de usuarios y roles*		
Referencia	*Control de acceso al sistema*		

No. Func.	Funcionalidad
001	El sistema deberá permitir registrar usuarios internos de la aplicación.
002	El sistema deberá permitir asignar un rol a cada usuario.
003	El sistema deberá permitir iniciar sesión mediante credenciales válidas.
004	El sistema deberá restringir las funciones disponibles de acuerdo con el rol asignado.
005	El sistema deberá permitir administrar los usuarios registrados.
006	El sistema deberá impedir el acceso a usuarios que no posean credenciales válidas.

REQ019 — Historial y auditoría

Id. Requerimiento	*RF-19*	Clasificación o prioridad	*ALTA*
Nombre Requerimiento	*Registro de operaciones del sistema*		
Referencia	*Control y auditoría*		

No. Func.	Funcionalidad
001	El sistema deberá registrar las operaciones importantes realizadas por los usuarios.
002	El sistema deberá registrar la fecha y hora de las operaciones realizadas.
003	El sistema deberá identificar al usuario que realizó cada operación.
004	El sistema deberá permitir consultar el historial de operaciones registradas.
005	El sistema deberá conservar el historial de modificaciones realizadas sobre la información crítica.

REQ020 — Validación de información

Id. Requerimiento	*RF-20*	Clasificación o prioridad	*MUY ALTA*
Nombre Requerimiento	*Validación e integridad de información*		
Referencia	*Control de datos*		

No. Func.	Funcionalidad
001	El sistema deberá validar que los campos obligatorios sean completados antes de registrar información.
002	El sistema deberá evitar el registro de información duplicada cuando corresponda.
003	El sistema deberá validar las relaciones existentes entre líneas, estaciones, buses, accesos y demás entidades.
004	El sistema deberá impedir registros que incumplan las reglas de negocio establecidas.
005	El sistema deberá mostrar mensajes informativos cuando una operación no pueda realizarse.

Requerimientos No Funcionales

RNF-01 — Seguridad de acceso

Id. Requerimiento	***RNF-01***	**Clasificación o prioridad**	***MUY ALTA***
Nombre Requerimiento	***Seguridad de acceso al sistema***		
Referencia	***Seguridad de la aplicación***		

No. Func.	**Funcionalidad**
001	El sistema deberá requerir autenticación mediante usuario y contraseña para acceder a las funciones administrativas.
002	El sistema deberá proteger las credenciales de los usuarios mediante mecanismos seguros de almacenamiento.
003	El sistema deberá impedir el acceso a las funciones del sistema a usuarios que no hayan sido autenticados.
004	El sistema deberá controlar el acceso a las funcionalidades de acuerdo con los permisos asignados al usuario.
005	El sistema deberá finalizar la sesión después de un período de inactividad establecido.

RNF-02 — Protección de información

Id. Requerimiento	***RNF-02***	**Clasificación o prioridad**	***MUY ALTA***
Nombre Requerimiento	***Protección de datos***		
Referencia	***Seguridad e integridad de la información***		

No. Func.	**Funcionalidad**
001	El sistema deberá proteger la información almacenada contra modificaciones no autorizadas.
002	El sistema deberá utilizar mecanismos seguros para transmitir información entre el navegador y el servidor.
003	El sistema deberá evitar que usuarios no autorizados puedan consultar información restringida.
004	El sistema deberá validar la información recibida antes de almacenarla en la base de datos.
005	El sistema deberá evitar la exposición de información sensible en mensajes de error.

RNF-03 — Rendimiento

Id. Requerimiento	*RNF-03*	Clasificación o prioridad	*MUY ALTA*
Nombre Requerimiento	***Rendimiento de la aplicación***		
Referencia	***Desempeño del sistema***		

No. Func.	Funcionalidad
001	El sistema deberá responder a las operaciones de consulta habituales en un tiempo máximo de 5 segundos bajo condiciones normales de operación.
002	El sistema deberá procesar las consultas de estaciones, líneas y buses sin afectar significativamente el funcionamiento de la aplicación.
003	El sistema deberá optimizar las consultas realizadas a la base de datos para evitar tiempos de respuesta innecesarios.
004	El sistema deberá mantener un funcionamiento estable durante operaciones simultáneas de consulta y registro.

RNF-04 — Disponibilidad

Id. Requerimiento	*RNF-04*	Clasificación o prioridad	*MUY ALTA*
Nombre Requerimiento	***Disponibilidad del sistema***		
Referencia	***Disponibilidad de la aplicación web***		

No. Func.	Funcionalidad
001	El sistema deberá encontrarse disponible para los usuarios autorizados durante el horario establecido de operación.
002	La aplicación web deberá estar alojada en un servicio que permita su acceso mediante Internet.
003	El sistema deberá informar al usuario cuando exista una interrupción temporal del servicio.
004	El sistema deberá permitir la recuperación del servicio después de una interrupción.
005	El sistema deberá minimizar los períodos de indisponibilidad durante las actividades de mantenimiento.

RNF-05 — Usabilidad

Id. Requerimiento	*RNF-05*	Clasificación o prioridad	*ALTA*
Nombre Requerimiento	*Facilidad de uso*		
Referencia	*Interfaz de usuario*		

No. Func.	Funcionalidad
001	El sistema deberá presentar una interfaz clara y organizada para facilitar la navegación de los usuarios.
002	El sistema deberá utilizar nombres y etiquetas comprensibles para identificar las funciones disponibles.
003	El sistema deberá mostrar mensajes claros cuando una operación sea realizada correctamente.
004	El sistema deberá mostrar mensajes descriptivos cuando ocurra un error durante una operación.
005	El sistema deberá mantener una estructura de navegación consistente en los diferentes módulos.

RNF-06 — Compatibilidad

Id. Requerimiento	*RNF-06*	Clasificación o prioridad	*ALTA*
Nombre Requerimiento	*Compatibilidad con navegadores web*		
Referencia	*Plataforma tecnológica*		

No. Func.	Funcionalidad
001	La aplicación deberá funcionar correctamente en los principales navegadores web actuales.
002	La aplicación deberá mantener sus funciones principales independientemente del navegador utilizado.
003	La interfaz deberá adaptarse correctamente a diferentes resoluciones de pantalla.
004	El sistema deberá permitir el acceso mediante computadoras utilizadas como estaciones de trabajo.

RNF-07 — Integridad de la información

Id. Requerimiento	*RNF-07*	Clasificación o prioridad	*MUY ALTA*
Nombre Requerimiento	*Integridad de datos*		
Referencia	*Base de datos*		

No. Func.	Funcionalidad
001	El sistema deberá garantizar la integridad de la información almacenada en la base de datos.
002	El sistema deberá utilizar restricciones para mantener las relaciones entre las diferentes entidades.
003	El sistema deberá evitar registros duplicados cuando exista una identificación única definida.
004	El sistema deberá impedir la eliminación de información que genere relaciones inválidas dentro de la base de datos.
005	El sistema deberá validar las reglas de negocio antes de almacenar información.

RNF-08 — Respaldo y recuperación

Id. Requerimiento	*RNF-08*	Clasificación o prioridad	*MUY ALTA*
Nombre Requerimiento	*Respaldo de información*		
Referencia	*Continuidad y recuperación*		

No. Func.	Funcionalidad
001	El sistema deberá contar con mecanismos para realizar respaldos periódicos de la base de datos.
002	Los respaldos deberán conservarse en una ubicación diferente a la base de datos principal.
003	El sistema deberá permitir recuperar la información a partir de un respaldo válido.
004	Los respaldos deberán protegerse contra modificaciones o eliminaciones no autorizadas.
005	Se deberá verificar periódicamente la disponibilidad e integridad de los respaldos realizados.

RNF-09 — Mantenibilidad

Id. Requerimiento	*RNF-09*	Clasificación o prioridad	*ALTA*
Nombre Requerimiento	*Mantenibilidad del software*		
Referencia	*Desarrollo y mantenimiento*		

No. Func.	Funcionalidad
001	El sistema deberá desarrollarse utilizando una estructura organizada que facilite su mantenimiento.
002	El código fuente deberá estar organizado por módulos o componentes de acuerdo con las funciones del sistema.
003	Las funciones principales deberán contar con documentación técnica que facilite su comprensión.
004	Las modificaciones realizadas al sistema deberán registrarse en el repositorio del proyecto.
005	Las actualizaciones deberán poder realizarse sin afectar innecesariamente las funcionalidades existentes.

RNF-10 — Escalabilidad

Id. Requerimiento	*RNF-10*	Clasificación o prioridad	*ALTA*
Nombre Requerimiento	*Capacidad de crecimiento*		
Referencia	*Arquitectura del sistema*		

No. Func.	Funcionalidad
001	El sistema deberá permitir agregar nuevas líneas sin modificar la estructura principal de la aplicación.
002	El sistema deberá permitir registrar nuevas estaciones, buses, accesos y usuarios conforme aumenten las necesidades del sistema.
003	La base de datos deberá estar diseñada para soportar el crecimiento progresivo de la información.
004	La arquitectura deberá permitir incorporar nuevos módulos en futuras versiones.

RNF-11 — Auditoría y trazabilidad

Id. Requerimiento	*RNF-11*	Clasificación o prioridad	*ALTA*
Nombre Requerimiento	*Trazabilidad de operaciones*		
Referencia	*Control y auditoría*		

No. Func.	Funcionalidad
001	El sistema deberá registrar las operaciones críticas realizadas por los usuarios.
002	El sistema deberá registrar la fecha y hora de las operaciones registradas.
003	El sistema deberá identificar al usuario responsable de cada operación.
004	El sistema deberá conservar un historial de las modificaciones realizadas sobre información crítica.
005	El sistema deberá permitir consultar el historial de operaciones para fines de auditoría.

RNF-12 — Generación de reportes

Id. Requerimiento	*RNF-12*	Clasificación o prioridad	*ALTA*
Nombre Requerimiento	*Calidad y presentación de reportes*		
Referencia	*Reportes del sistema*		

No. Func.	Funcionalidad
001	Los reportes deberán presentar información clara, ordenada y legible.
002	Los reportes deberán mostrar información actualizada de acuerdo con los registros existentes en la base de datos.
003	Los reportes deberán identificar claramente el tipo de información presentada.
004	Los reportes deberán poder consultarse desde la aplicación web.
005	Las consultas utilizadas para generar los reportes deberán quedar documentadas en el proyecto.

RNF-13 — Pruebas y calidad del software

Id. Requerimiento	*RNF-13*	Clasificación o prioridad	*MUY ALTA*
Nombre Requerimiento	*Calidad y verificabilidad del sistema*		
Referencia	*Aseguramiento de la calidad*		

No. Func.	Funcionalidad
001	El sistema deberá contar con pruebas para verificar el funcionamiento de sus principales funcionalidades.
002	Las pruebas deberán documentar los resultados obtenidos y los errores identificados.
003	Los errores detectados deberán ser corregidos y sometidos nuevamente a pruebas.
004	El sistema deberá contar con criterios de aceptación para determinar si cada funcionalidad cumple con los requisitos establecidos.
005	Las pruebas deberán incluir la validación de las reglas de negocio y controles de seguridad implementados.

RNF-14 — Documentación técnica

Id. Requerimiento	*RNF-14*	Clasificación o prioridad	*ALTA*
Nombre Requerimiento	*Documentación del sistema*		
Referencia	*Documentación y mantenimiento*		

No. Func.	Funcionalidad
001	El proyecto deberá contar con documentación técnica de la solución desarrollada.
002	La documentación deberá incluir la estructura de la base de datos utilizada.
003	La documentación deberá incluir los diagramas UML correspondientes al sistema.
004	La documentación deberá incluir los procedimientos necesarios para instalar o ejecutar la aplicación.
005	La documentación deberá mantenerse disponible junto con los demás archivos del proyecto en el repositorio correspondiente.

7. ESTUDIOS DE FACTIBILIDAD

El sistema propuesto para Transmetro busca centralizar en una sola plataforma web la información operativa relacionada con líneas, estaciones, accesos, buses, parqueos, pilotos, operadores, personal de seguridad, municipalidades, tarjetas de acceso, pagos, alertas y reportes.

Actualmente la Municipalidad de Guatemala dispone de un portal oficial de Movilidad Urbana donde se publica información de las distintas líneas y estaciones de Transmetro, lo cual confirma que la operación se encuentra distribuida en múltiples puntos de la ciudad y que existe una estructura considerable que administrar. El portal municipal mantiene apartados específicos por línea, mapas y ubicación de estaciones.

Además, el portal de Turismo de la Municipalidad reporta como referencia 79 estaciones, junto con estaciones de transbordo y centrales de transferencia. Esta dimensión hace necesario que una solución de control no dependa de archivos aislados en cada estación, sino de una plataforma centralizada a la que puedan conectarse los usuarios autorizados desde diferentes ubicaciones.

El planteamiento original del proyecto también requiere que cada estación cuente con una computadora de trabajo, que exista un operador responsable, que se determine una forma adecuada de interconectar los equipos y que se consideren métodos de pago mediante tarjeta.

A partir de estas condiciones se analizaron los siguientes aspectos.

7.1 Factibilidad técnica

La propuesta puede desarrollarse con tecnologías disponibles actualmente y que cuentan con suficiente soporte para implementar una aplicación web institucional.

Para el desarrollo se utilizará Visual Studio 2022 Community como entorno de programación y C# con ASP.NET Core MVC para construir la aplicación. Esta combinación permitirá separar de manera organizada la lógica del sistema, las interfaces y el acceso a los datos.

El acceso a la base de datos se realizará utilizando Entity Framework Core, permitiendo trabajar con las entidades del proyecto desde el código y mantener una relación directa entre la aplicación y las tablas correspondientes.

La información se almacenará utilizando tecnología de SQL Server, manteniendo una base de datos relacional adecuada para las relaciones que existen entre líneas, estaciones, buses, accesos, parqueos, pilotos, usuarios y demás elementos definidos en los requerimientos.

La interfaz será construida mediante HTML, CSS y Bootstrap, procurando que las pantallas sean claras y se adapten correctamente a diferentes tamaños de monitor. JavaScript será utilizado solamente en aquellas funciones donde se necesite interacción dinámica, validaciones adicionales o actualización de elementos de la interfaz sin recargar completamente la página.

El alojamiento de la solución se plantea mediante Microsoft Azure. La aplicación ASP.NET Core puede publicarse en Azure App Service, mientras que la información puede mantenerse en Azure SQL Database. Microsoft describe Azure SQL Database como un servicio administrado y escalable orientado precisamente a aplicaciones que requieren bases de datos relacionales en la nube.

La estructura de funcionamiento propuesta será la siguiente:

Computadora de la estación → navegador web → conexión a Internet → Microsoft Azure → aplicación ASP.NET Core MVC → Entity Framework Core → base de datos SQL

Este modelo resulta conveniente para el proyecto porque cada estación no necesitaría mantener una copia independiente del sistema ni de la base de datos.

El operador solamente necesitaría iniciar sesión desde el navegador y trabajar sobre la plataforma central.

Equipamiento de las estaciones

Debido a que el caso solicita una computadora de trabajo por estación, no se requiere equipo especializado de alto rendimiento.

Una configuración apropiada para el sistema sería:

Característica	Recomendación
Procesador	Intel Core i5 o equivalente
Memoria RAM	8 GB mínimo
Almacenamiento	SSD de 512 GB
Monitor	23.8 o 24 pulgadas Full HD
Conexión de red	Ethernet Gigabit
Sistema operativo	Windows 11 Pro
Navegadores	Microsoft Edge, Google Chrome o equivalente actualizado
Protección eléctrica	UPS de aproximadamente 1000 VA

El sistema no necesitará tarjetas gráficas dedicadas ni procesadores de gama alta, porque la mayor parte del procesamiento de la aplicación se ejecutará sobre la infraestructura central.

Tomando en cuenta los productos de mercado en Guatemala, Intelaf mantiene monitores AOC de 24 pulgadas Full HD alrededor de Q817 en promoción, mientras que un UPS de 1000 VA puede encontrarse alrededor de Q535 a Q780, dependiendo de la marca y características.

Para la cotización institucional se utilizará un monto promedio que permita adquirir equipos de oficina con garantía y características adecuadas, sin depender exclusivamente de ofertas temporales.

Conectividad entre estaciones

La utilización de una aplicación web elimina la necesidad de crear una red física exclusiva que conecte directamente una estación con otra.

Cada estación podrá comunicarse con el sistema central mediante Internet.

El tráfico se dirigirá hacia Microsoft Azure utilizando conexiones HTTPS, evitando transmitir credenciales y datos importantes en texto abierto.

En una implementación real deberán revisarse individualmente las estaciones para determinar cuáles ya cuentan con conectividad municipal y cuáles requerirían un nuevo enlace.

No sería conveniente contratar de manera automática un servicio de Internet nuevo para las 79 estaciones sin verificar previamente la infraestructura existente.

En los lugares donde la conexión sea crítica también podría considerarse un enlace secundario móvil para contingencia.

Capacidad de crecimiento

La utilización de Azure permite comenzar con recursos de menor capacidad y aumentarlos si crece el número de operadores, consultas y transacciones.

Microsoft Azure ofrece distintos niveles de App Service. Como referencia, un plan B1 dispone de 1 núcleo y 1.75 GB de memoria, mientras que el B2 dispone de 2 núcleos y 3.5 GB.

Para una operación más cercana a un entorno institucional se consideraría inicialmente un nivel B2 o equivalente y posteriormente se evaluaría su comportamiento.

La aplicación podrá crecer sin modificar las computadoras instaladas en cada estación, porque el aumento de capacidad se realizaría principalmente en los servicios de Azure.

Base de datos

La información del sistema se organizará mediante una base de datos relacional.

Esto permite controlar relaciones como:

- estaciones pertenecientes a líneas
- orden de estaciones dentro de una ruta
- buses asignados a cada línea
- buses y parqueos
- accesos pertenecientes a estaciones
- guardias asignados a accesos
- pilotos y sus registros
- operadores y estaciones

- usuarios y roles
- tarjetas y transacciones
- historial de modificaciones.

El sistema original exige además reglas específicas, como impedir que un bus se encuentre asignado simultáneamente a más de una línea y mantener las cantidades mínimas y máximas de buses según el número de estaciones.

Estas reglas pueden implementarse mediante validaciones en C#, Entity Framework Core y restricciones dentro de la base de datos.

Seguridad de la plataforma

La plataforma deberá manejar información que no puede quedar disponible para cualquier persona, especialmente datos relacionados con empleados, pilotos y operaciones internas.

Se implementarán como mínimo:

- autenticación mediante usuario y contraseña
- almacenamiento seguro de contraseñas
- perfiles y roles
- restricciones de acceso según responsabilidad
- cierre de sesión por inactividad
- HTTPS
- validaciones de datos
- protección contra solicitudes inválidas
- control de operaciones sensibles
- bitácora de modificaciones
- respaldo de información
- recuperación de datos.

Respaldos

La información no deberá depender de una sola copia.

Se plantea mantener respaldos periódicos de la base de datos y establecer procedimientos de recuperación ante errores.

Los respaldos deberán protegerse de modificaciones no autorizadas y verificarse periódicamente.

Esto también se encuentra alineado con el requerimiento RNF-08 ya documentado, donde se establece que la base de datos deberá contar con respaldos periódicos, conservar copias separadas y permitir recuperación.

Resultado técnico

Después de analizar la arquitectura, herramientas, infraestructura y conectividad necesarias, la propuesta se considera técnicamente factible.

No requiere una tecnología experimental ni infraestructura difícil de encontrar. Las herramientas seleccionadas cuentan con compatibilidad directa entre sí y permiten desarrollar, publicar y ampliar la plataforma conforme aumenten las necesidades.

7.2 Factibilidad operativa

La propuesta también puede incorporarse a la operación diaria de las estaciones sin requerir que los operadores tengan conocimientos de programación o administración de servidores.

Las funciones se presentarán mediante pantallas y formularios accesibles desde un navegador.

Un operador podrá ingresar al sistema, autenticarse y utilizar únicamente las funciones que le correspondan.

El proyecto ya contempla expresamente la existencia de un operador encargado de la computadora disponible en cada estación. Por esta razón, la plataforma será diseñada principalmente para que las tareas habituales puedan ejecutarse mediante opciones claras y no mediante comandos técnicos.

Usuarios previstos

Se plantea trabajar con distintos perfiles.

- **Administrador general**
  Tendrá control sobre usuarios, roles, configuraciones y consultas generales.
- **Supervisor operativo**
  Podrá consultar líneas, estaciones, buses, asignaciones, alertas y reportes.
- **Operador de estación**
  Administrará únicamente las funciones relacionadas con la estación a la que haya sido asignado.
- **Personal administrativo**
  Podrá consultar y actualizar registros que correspondan a flota, pilotos, seguridad o administración.
- **Usuario de consulta o auditoría**
  Tendrá acceso principalmente a reportes, historiales y registros autorizados.

Esta separación permitirá evitar que un operador modifique datos administrativos que no sean de su competencia.

Capacitación

Antes de habilitar la plataforma será necesario realizar una capacitación básica a los usuarios.

La capacitación deberá abarcar:

- inicio y cierre de sesión
- navegación dentro de la plataforma
- registro de información
- modificación de información
- consulta de estaciones
- consulta de buses
- manejo de alertas
- generación de reportes
- manejo de errores

- procedimientos en caso de interrupción
- seguridad de credenciales.

También se entregará un manual de usuario que permita consultar las funciones principales sin depender permanentemente del desarrollador.

Implementación progresiva

Para reducir errores durante el inicio de operaciones, no se recomienda activar todas las estaciones al mismo tiempo.

Una estrategia más adecuada sería comenzar con un grupo reducido de estaciones. Por ejemplo, podría realizarse un piloto inicial con cinco estaciones.

Durante ese período se observarían:

- tiempos de respuesta
- facilidad de uso
- estabilidad
- conexión a Azure
- errores de captura
- comportamiento de los permisos
- consultas a la base de datos
- generación de reportes
- recepción de alertas.

Después de corregir los problemas detectados, el sistema podría habilitarse progresivamente en más estaciones.

Manejo de interrupciones

La dependencia de Internet implica que pueden presentarse interrupciones externas.

En ese caso deberá existir un procedimiento de contingencia.

El operador podrá registrar temporalmente la información necesaria mediante el procedimiento definido por la institución y actualizarla cuando el servicio vuelva a estar disponible.

Las estaciones consideradas críticas podrían utilizar conectividad secundaria como respaldo.

Resultado operativo

La propuesta se considera operativamente factible, ya que el personal podrá trabajar mediante una interfaz web y las tareas se pueden dividir según roles.

La capacitación y una implementación progresiva reducirán la resistencia al cambio y permitirán identificar dificultades antes de extender la plataforma a todas las estaciones.

7.3 Factibilidad económica

El mayor costo del proyecto no se encuentra únicamente en el desarrollo del sistema, sino en el escenario donde también sea necesario proporcionar equipo a todas las estaciones.

Por esta razón se plantean dos alternativas económicas.

- La primera contempla entregar tanto el sistema como el equipo necesario.
- La segunda contempla únicamente el sistema, considerando que la institución ya cuenta con computadoras y conectividad.

Esta separación evita incrementar innecesariamente el presupuesto cuando existan recursos que puedan reutilizarse.

Beneficios esperados

La inversión en la plataforma permitiría centralizar información que actualmente puede encontrarse distribuida en diferentes procesos.

Entre los principales beneficios se encuentran:

- mayor rapidez para consultar información
- menor duplicación de registros
- reducción de errores
- actualización centralizada

- trazabilidad
- generación automática de reportes
- mejor control de buses
- mejor control de parqueos
- acceso organizado a información de pilotos
- administración de usuarios
- registro de transacciones
- disponibilidad de información para supervisión
- automatización de reglas operativas
- generación de alertas.

Un ejemplo concreto es la asignación de buses.

El proyecto exige que un bus no pueda estar asignado a más de una línea y que la cantidad de buses de una línea mantenga relación con el número de estaciones.

Una plataforma centralizada puede validar automáticamente estas condiciones y reducir la posibilidad de registrar información incorrecta.

Costos recurrentes

Además del costo inicial, deben contemplarse gastos recurrentes relacionados principalmente con:

- Microsoft Azure
- mantenimiento de software
- respaldo
- soporte
- conectividad
- eventual reemplazo de equipos.

Azure trabaja bajo un modelo de consumo y permite escalar recursos. Microsoft señala que, fuera del nivel gratuito, App Service genera cargos de acuerdo con los recursos de procesamiento utilizados.

Por esta razón, el presupuesto de producción debe considerar una capacidad adecuada y no depender de recursos temporales.

Resultado económico

La propuesta se considera económicamente factible, especialmente si se reutiliza parte del equipo que ya se encuentre disponible.

La alternativa que incluye hardware representa una inversión inicial considerable, mientras que la propuesta solamente de software reduce de manera importante ese monto.

Por esta razón, antes de comprar equipo se recomienda realizar un inventario físico de las estaciones.

7.4 Factibilidad legal

El sistema manejará información institucional y datos personales, por lo que no deberá limitarse únicamente a funcionar técnicamente.

Será necesario restringir quién puede consultar y modificar cada dato.

La Ley de Acceso a la Información Pública, Decreto 57-2008, reconoce dentro de sus definiciones los datos personales y los datos personales sensibles.

En el proyecto se contempla almacenar información de pilotos como datos de residencia, comunicación e historial educativo.

Por esta razón, esta información no deberá mostrarse a todos los usuarios del sistema.

Se utilizarán roles para controlar el acceso.

Un operador de estación, por ejemplo, no necesitará consultar todos los datos personales de un piloto, mientras que un usuario administrativo autorizado podría requerir determinada información. También será necesario registrar quién modifica información importante.

La bitácora permitirá conocer:

- usuario responsable
- fecha
- hora
- operación realizada.

Contratación pública

La propuesta está dirigida a una dependencia municipal.

El Decreto 57-92, Ley de Contrataciones del Estado, regula las compras y contrataciones realizadas por organismos del Estado, entidades autónomas, municipalidades y empresas públicas.

Por esta razón, una adquisición real de hardware, servicios o desarrollo deberá seguir los procedimientos administrativos correspondientes.

La propuesta monetaria presentada dentro de este proyecto representa una estimación comercial de los recursos necesarios y permite establecer un valor de referencia para la solución.

Licenciamiento

Las tecnologías seleccionadas deberán utilizarse respetando los términos de sus respectivas licencias.

Se trabajará principalmente con herramientas del ecosistema Microsoft y librerías utilizadas legalmente dentro del proyecto.

Los paquetes de terceros deberán revisarse antes de incorporarlos para verificar sus condiciones de uso.

Información de pago

El proyecto solicita administrar métodos de pago y tarjetas de acceso.

La información oficial de Movilidad Urbana confirma el uso de medios electrónicos dentro del sistema municipal de transporte; otras rutas operadas por Transmetro han publicado el uso de Tarjeta Ciudadana y tarjetas bancarias como formas de pago.

La aplicación propuesta no deberá almacenar información bancaria sensible innecesaria.

Si posteriormente se integrara un proveedor externo de pagos, la aplicación deberá enviar las operaciones hacia el mecanismo autorizado en lugar de guardar números completos de tarjetas bancarias.

Resultado legal

La solución se considera legalmente factible, siempre que se implementen controles de acceso, manejo responsable de datos personales, trazabilidad y los procedimientos de contratación que correspondan.

7.5 Factibilidad ambiental

El principal producto del proyecto es software, por lo que no se requiere construcción de infraestructura física especial.

El mayor impacto ambiental estaría relacionado con el consumo eléctrico de las computadoras y la sustitución de equipo tecnológico.

Por esta razón, no se recomienda adquirir equipos de mayor capacidad de la necesaria.

Una computadora de oficina con procesador Core i5, SSD y monitor LED es suficiente para utilizar la plataforma.

Aprovechamiento de equipos existentes

Antes de realizar compras nuevas se deberá verificar qué estaciones ya cuentan con computadoras capaces de ejecutar un navegador moderno.

Si un equipo cumple los requisitos mínimos, puede reutilizarse.

Esta medida reduce costos y disminuye la generación de residuos electrónicos.

Digitalización

Uno de los beneficios adicionales del sistema será reducir el uso de documentos impresos.

Información como:

- asignaciones
- historiales

- reportes
- registros de buses
- información de estaciones
- cambios de parqueo
- operaciones de usuarios

podrá consultarse digitalmente.

Esto puede disminuir la impresión de formularios y reportes utilizados únicamente para consulta interna.

Residuos

Guatemala cuenta con el Reglamento para la Gestión Integral de Residuos y Desechos Sólidos Comunes, contenido en el Acuerdo Gubernativo 164-2021. El reglamento establece normas sanitarias y ambientales orientadas a prevenir contaminación y proteger la salud.

Los equipos que lleguen al final de su vida útil no deberán eliminarse de manera indiscriminada.

Esto incluye:

- computadoras
- monitores
- baterías
- UPS
- equipos de comunicación.

Se deberá favorecer reutilización, reparación y disposición responsable.

Resultado ambiental

La propuesta se considera ambientalmente factible, ya que no genera impactos significativos durante su operación y además favorece la digitalización de procesos.

El impacto relacionado con equipos puede reducirse utilizando hardware existente, realizando mantenimiento preventivo y evitando sustituciones innecesarias.

7.6 Resultado general de factibilidad

Estudio	Resultado
Técnica	Factible
Operativa	Factible
Económica	Factible
Legal	Factible con controles
Ambiental	Factible

En conjunto, el desarrollo del Sistema Web de Control y Gestión para Transmetro resulta viable.

La arquitectura seleccionada permite centralizar la información, proporcionar acceso a los operadores desde diferentes estaciones, aplicar controles de seguridad y ampliar los recursos cuando aumente la cantidad de usuarios.

La principal recomendación antes de una implementación general consiste en efectuar un inventario de equipo y conectividad, iniciar con un grupo piloto y posteriormente extender el sistema de manera gradual.

8. PROPUESTA MONETARIA DE LA SOLUCIÓN

Para la propuesta económica se plantean dos modalidades.

- La primera incluye el desarrollo completo del sistema y el suministro de equipos.
- La segunda corresponde únicamente al desarrollo, implementación y publicación de la plataforma, utilizando la infraestructura disponible en las estaciones.

Los valores de hardware se establecen tomando como referencia precios disponibles en proveedores tecnológicos de Guatemala durante septiembre de 2026. Por ejemplo, actualmente pueden encontrarse monitores Full HD de 24 pulgadas alrededor de Q817 y UPS de 1000 VA entre aproximadamente Q535 y Q780 dependiendo del fabricante.

Para efectos presupuestarios se utilizarán precios normales de referencia y no únicamente promociones temporales.

8.1 Propuesta No. 1

Desarrollo del sistema y suministro de equipo

Esta propuesta contempla entregar una solución completa para las estaciones consideradas dentro del alcance.

Como base de cálculo se utilizarán las 79 estaciones publicadas por la Municipalidad de Guatemala.

Equipamiento propuesto por estación

Se propone presupuestar cada puesto de trabajo con los siguientes elementos:

Concepto / Descipción	**Precio unitario estimado**
Computadora de oficina Core i5, 8 GB RAM, SSD 512 GB, Windows 11 Pro	Q6,500
Monitor Full HD de 24"	Q900
UPS 1000 VA	Q800
Equipo de red y cableado básico	Q500
Total estimado por estación	**Q8,700**

Los Q900 utilizados para monitor se encuentran ligeramente por encima de una promoción actual de Q817, mientras que Q800 para UPS se encuentra dentro del rango de referencias de mercado local.

Se utiliza un margen razonable para evitar construir el presupuesto alrededor de una promoción específica que podría desaparecer al momento de efectuar la compra.

Total de hardware

Q8,700 × 79 estaciones = Q687,300

Instalación y configuración

Se considera un valor promedio de:

Q500 por estación

Incluye:

- instalación del puesto
- configuración del sistema operativo
- configuración de navegador
- conexión a red
- acceso al sistema
- pruebas
- identificación del equipo
- configuración del UPS.

Q500 × 79 = Q39,500

Total de equipo e instalación

Q687,300 + Q39,500 = Q726,800

8.2 Desarrollo del software

La cotización del desarrollo se distribuye según los principales componentes del proyecto.

Actividad	Precio
Levantamiento, validación y análisis detallado de requerimientos	Q10,000
Diseño de arquitectura y base de datos	Q10,000
Configuración de proyecto ASP.NET Core MVC	Q4,000
Desarrollo de módulos de líneas y estaciones	Q10,000
Desarrollo de accesos y municipalidades	Q6,000
Desarrollo de buses, parqueos y recorridos	Q14,000

Desarrollo de pilotos, operadores y guardias	Q10,000
Desarrollo de usuarios, roles y seguridad	Q10,000
Desarrollo de tarjetas, pagos y transacciones	Q10,000
Desarrollo de alertas y reglas de capacidad	Q8,000
Desarrollo de reportes	Q8,000
Historial y auditoría	Q6,000
Pruebas y correcciones	Q8,000
Configuración y publicación en Azure	Q4,000
Documentación y manuales	Q5,000
Capacitación inicial	Q4,000
Soporte inicial de implementación	Q3,000
Subtotal de desarrollo	**Q130,000**

Este precio considera una solución hecha específicamente según los requerimientos del proyecto y no una plataforma genérica.

El alcance es amplio porque actualmente se han definido veinte requerimientos funcionales, incluyendo gestión de líneas, estaciones, flota, personal, pagos, reportes, usuarios, historial y validaciones.

8.3 Alojamiento en Microsoft Azure

Para una implementación institucional se plantea utilizar:

- Azure App Service
- Azure SQL Database
- almacenamiento para respaldos
- registro y monitoreo
- tráfico de datos

- servicios complementarios.

Como referencia actual, Microsoft publica un precio de aproximadamente USD 109.50 al mes para un plan B2 de App Service con 2 núcleos y 3.5 GB de memoria.

Utilizando como referencia un tipo de cambio cercano a Q7.63 por dólar, el App Service representa aproximadamente:

USD 109.50 × Q7.63 = Q835.49 mensuales

Anualmente:

Q835.49 × 12 = Q10,025.88

Para Azure SQL puede utilizarse inicialmente un nivel estándar moderado. Un S0 utiliza 10 DTU y hasta 250 GB según la información de Microsoft.

Como referencia histórica y de soporte de Microsoft, el nivel S0 se ha situado alrededor de USD 15 mensuales.

Esto equivale aproximadamente a:

USD 15 × Q7.63 = Q114.45 mensuales

Q114.45 × 12 = Q1,373.40 anuales

Agregando una reserva para almacenamiento, respaldos, monitoreo y consumo adicional, se plantea:

Servicio Azure	**Estimación anual**
Azure App Service B2	Q10,026
Azure SQL Database	Q1,374
Respaldo, almacenamiento y monitoreo	Q2,600
Total anual estimado Azure	**Q14,000**

El valor deberá revisarse al momento de contratar el servicio porque Microsoft indica que los precios varían según región, configuración y consumo.

8.4 Total de la propuesta No. 1

Concepto	Valor
Hardware para 79 estaciones	Q687,300
Instalación y configuración	Q39,500
Desarrollo del sistema	Q130,000
Microsoft Azure primer año	Q14,000
TOTAL PROPUESTA No. 1	**Q870,800**

Total propuesto

Q886,400.00

Esta propuesta incluye tanto la infraestructura de estaciones como el desarrollo y publicación inicial de la plataforma.

No se incluye una nueva conexión de Internet para cada estación porque primero debe determinarse cuáles ubicaciones ya poseen conectividad.

8.5 Propuesta No. 2

Desarrollo del sistema utilizando equipo existente

La segunda alternativa elimina la adquisición de computadoras, monitores, UPS y equipo de red.

La institución proporcionaría los puestos de trabajo y conectividad necesarios.

El proveedor se responsabilizaría por:

- análisis
- diseño
- programación
- base de datos
- seguridad
- pruebas
- documentación

- capacitación
- configuración
- publicación en Azure
- soporte inicial.

La cotización sería:

Concepto	Precio
Desarrollo del sistema	Q130,000
Microsoft Azure primer año	Q14,000
TOTAL PROPUESTA No. 2	**Q144,000**

Total propuesto

Q144,000.00

8.6 Comparación de las propuestas

Elemento	Propuesta 1	Propuesta 2
Aplicación ASP.NET Core MVC	Sí	Sí
C#	Sí	Sí
Entity Framework Core	Sí	Sí
SQL	Sí	Sí
Microsoft Azure	Sí	Sí
Usuarios y roles	Sí	Sí
Módulo de líneas	Sí	Sí
Módulo de estaciones	Sí	Sí
Gestión de buses	Sí	Sí
Parqueos	Sí	Sí
Pilotos	Sí	Sí

Operadores	Sí	Sí
Guardias	Sí	Sí
Tarjetas y pagos	Sí	Sí
Alertas	Sí	Sí
Reportes	Sí	Sí
Auditoría	Sí	Sí
Pruebas	Sí	Sí
Capacitación	Sí	Sí
Equipos para 79 estaciones	Sí	No
Monitores	Sí	No
UPS	Sí	No
Red básica	Sí	No
Instalación física básica	Sí	No
Costo inicial estimado	**Q886,400**	**Q144,000**

8.7 Elementos no incluidos

Para evitar que el alcance económico quede abierto, se excluyen los siguientes conceptos:

- adquisición de buses
- combustible
- mantenimiento mecánico
- construcción de estaciones
- remodelaciones físicas
- cámaras de videovigilancia
- torniquetes

- sensores automáticos de pasajeros
- máquinas de recarga
- validadores de Tarjeta Ciudadana
- infraestructura de fibra óptica metropolitana
- servicios de Internet nuevos por estación
- guardias
- pilotos
- operadores
- mobiliario
- aire acondicionado
- obra civil
- integración con plataformas externas para las cuales no se proporcione acceso técnico.

Si posteriormente se solicita integrar la aplicación con sistemas ya existentes, deberá realizarse primero un análisis técnico de la interfaz disponible.

8.8 Vigencia de la propuesta

La propuesta tendrá una vigencia de 30 días calendario.

Los precios de hardware pueden modificarse de acuerdo con disponibilidad, marca, promociones, importación o variaciones del mercado.

Los servicios de Microsoft Azure también pueden variar de acuerdo con región, capacidad y consumo.

8.9 Forma de pago

Para el desarrollo del sistema se propone distribuir el pago de acuerdo con el avance.

Etapa	Porcentaje
Inicio del proyecto y aprobación de requerimientos	20 %
Diseño de base de datos y arquitectura	20 %
Entrega del primer conjunto de módulos funcionales	25 %
Aplicación completa para pruebas	20 %
Implementación, documentación y aceptación	15 %
Total	**100 %**

En caso de seleccionar la propuesta con hardware, la adquisición de equipos deberá manejarse de manera separada debido a que el proveedor deberá disponer de los fondos antes de realizar las compras.

8.10 Garantía

El sistema contará con una garantía inicial de 90 días posteriores a su aceptación.

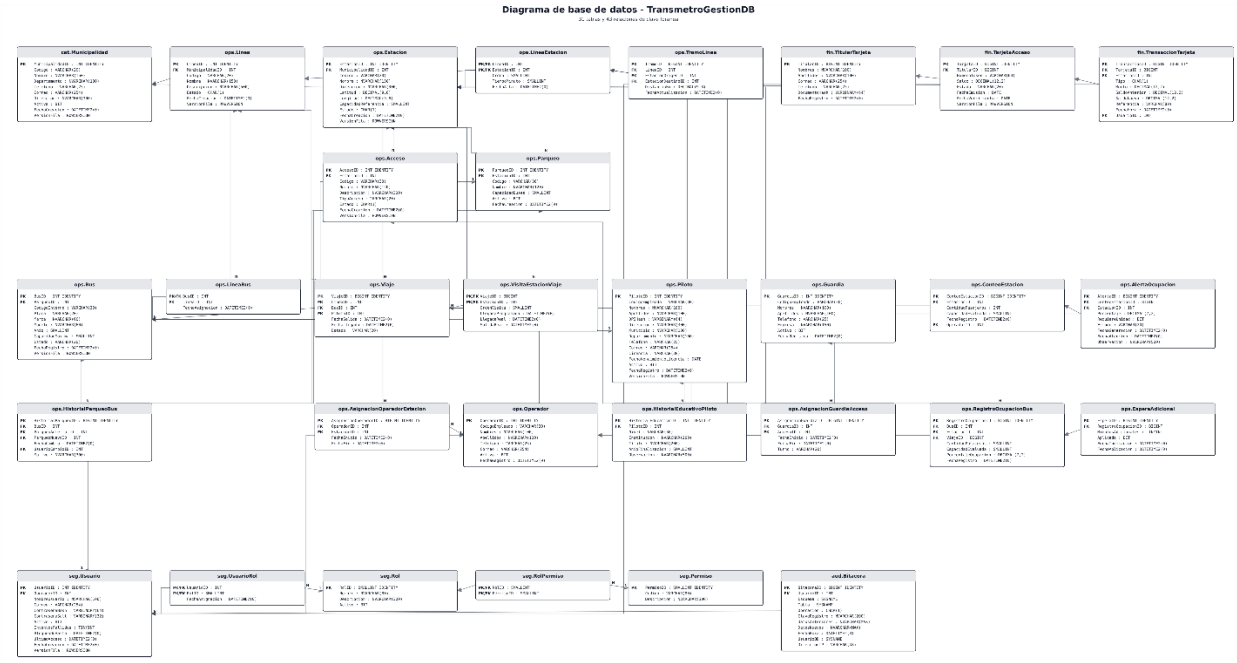
Durante este período se corregirán sin costo adicional los errores directamente relacionados con las funcionalidades incluidas en el alcance original.

No se considerarán parte de la garantía:

1. nuevas funcionalidades
2. cambios solicitados después de la aceptación
3. fallos de Internet
4. daños eléctricos
5. daños físicos en equipos
6. modificación del código por terceros
7. problemas provocados por plataformas externas
8. ampliaciones de infraestructura.

Posteriormente podrá contratarse mantenimiento y soporte mediante un acuerdo adicional.

9. MODELO ENTIDAD RELACIÓN



10. SCRIPT DE LA BASE DE DATOS

```
CREATE DATABASE TransmetroDB;

END;

GO

USE TransmetroDB;

GO

SET ANSI_NULLS ON;

SET QUOTED_IDENTIFIER ON;

GO



/*============================================================================

  1. ESQUEMAS

============================================================================*/
```

```
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = N'cat') EXEC(N'CREATE SCHEMA cat');

IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = N'ops') EXEC(N'CREATE SCHEMA ops');

IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = N'seg') EXEC(N'CREATE SCHEMA seg');

IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = N'fin') EXEC(N'CREATE SCHEMA fin');

IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = N'aud') EXEC(N'CREATE SCHEMA aud');

IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = N'rep') EXEC(N'CREATE SCHEMA rep');

GO

/*=============================================================

  2. CATALOGOS, MUNICIPALIDADES Y ESTRUCTURA DE LINEAS/ESTACIONES

=============================================================*/

CREATE TABLE cat.Municipalidad

(

   MunicipalidadID INT IDENTITY(1,1) NOT NULL,

   Codigo          VARCHAR(20)      NOT NULL,

   Nombre          NVARCHAR(150)    NOT NULL,

   Departamento                NVARCHAR(100)                NOT NULL CONSTRAINT DF_Municipalidad_Dep DEFAULT N'Guatemala',

   Telefono        VARCHAR(25)      NULL,

   Correo          VARCHAR(254)     NULL,

   Direccion       NVARCHAR(300)    NULL,
```

```
    Activa          BIT             NOT NULL CONSTRAINT DF_Municipalidad_Activa DEFAULT
(1),
    FechaCreacion           DATETIME2(0)            NOT NULL CONSTRAINT
DF_Municipalidad_Fecha DEFAULT SYSUTCDATETIME(),
    VersionFila     ROWVERSION      NOT NULL,
    CONSTRAINT PK_Municipalidad PRIMARY KEY (MunicipalidadID),
    CONSTRAINT UQ_Municipalidad_Codigo UNIQUE (Codigo),
    CONSTRAINT UQ_Municipalidad_Nombre UNIQUE (Nombre),
    CONSTRAINT CK_Municipalidad_Correo CHECK (Correo IS NULL OR Correo LIKE
'%_@_%._%')
);
GO
CREATE TABLE ops.Linea
(
    LineaID         INT IDENTITY(1,1) NOT NULL,
    MunicipalidadID  INT            NOT NULL,
    Codigo          VARCHAR(20)     NOT NULL,
    Nombre          NVARCHAR(150)   NOT NULL,
    Descripcion     NVARCHAR(500)   NULL,
    Estado          CHAR(1)         NOT NULL CONSTRAINT DF_Linea_Estado DEFAULT
('P'),
    FechaCreacion           DATETIME2(0)            NOT NULL CONSTRAINT DF_Linea_Fecha
DEFAULT SYSUTCDATETIME(),
    VersionFila     ROWVERSION      NOT NULL,
    CONSTRAINT PK_Linea PRIMARY KEY (LineaID),
    CONSTRAINT FK_Linea_Municipalidad FOREIGN KEY (MunicipalidadID)
```

```
    REFERENCES cat.Municipalidad(MunicipalidadID),
  CONSTRAINT UQ_Linea_Codigo UNIQUE (Codigo),
  CONSTRAINT CK_Linea_Estado CHECK (Estado IN ('P','A','I'))
);
GO
CREATE TABLE ops.Estacion
(
  EstacionID        INT IDENTITY(1,1) NOT NULL,
  MunicipalidadID   INT               NOT NULL,
  Codigo            VARCHAR(20)       NOT NULL,
  Nombre            NVARCHAR(150)     NOT NULL,
  Direccion         NVARCHAR(300)     NULL,
  Latitud           DECIMAL(9,6)      NULL,
  Longitud          DECIMAL(9,6)      NULL,
  CapacidadReferencia SMALLINT NOT NULL CONSTRAINT DF_Estacion_Capacidad DEFAULT (80),
  Estado            CHAR(1)           NOT NULL CONSTRAINT DF_Estacion_Estado DEFAULT ('P'),
  FechaCreacion     DATETIME2(0)      NOT NULL CONSTRAINT DF_Estacion_Fecha DEFAULT SYSUTCDATETIME(),
  VersionFila       ROWVERSION        NOT NULL,
  CONSTRAINT PK_Estacion PRIMARY KEY (EstacionID),
  CONSTRAINT FK_Estacion_Municipalidad FOREIGN KEY (MunicipalidadID)
    REFERENCES cat.Municipalidad(MunicipalidadID),
  CONSTRAINT UQ_Estacion_Codigo UNIQUE (Codigo),
```

```
    CONSTRAINT CK_Estacion_Capacidad CHECK (CapacidadReferencia > 0),
    CONSTRAINT CK_Estacion_Estado CHECK (Estado IN ('P','A','I')),
    CONSTRAINT CK_Estacion_Latitud CHECK (Latitud IS NULL OR Latitud BETWEEN -90 AND 90),
    CONSTRAINT CK_Estacion_Longitud CHECK (Longitud IS NULL OR Longitud BETWEEN -180 AND 180)
);
GO
CREATE TABLE ops.LineaEstacion
(
    LineaID      INT          NOT NULL,
    EstacionID   INT          NOT NULL,
    Orden        SMALLINT     NOT NULL,
    TiempoMinuto SMALLINT     NULL,
    FechaAlta    DATETIME2(0) NOT NULL CONSTRAINT DF_LineaEstacion_Fecha DEFAULT SYSUTCDATETIME(),
    CONSTRAINT PK_LineaEstacion PRIMARY KEY (LineaID, EstacionID),
    CONSTRAINT FK_LineaEstacion_Linea FOREIGN KEY (LineaID)
        REFERENCES ops.Linea(LineaID),
    CONSTRAINT FK_LineaEstacion_Estacion FOREIGN KEY (EstacionID)
        REFERENCES ops.Estacion(EstacionID),
    CONSTRAINT UQ_LineaEstacion_Orden UNIQUE (LineaID, Orden),
    CONSTRAINT CK_LineaEstacion_Orden CHECK (Orden > 0),
    CONSTRAINT CK_LineaEstacion_Tiempo CHECK (TiempoMinuto IS NULL OR TiempoMinuto >= 0)
```

```
);
GO
CREATE TABLE ops.TramoLinea
(
   TramoID          BIGINT IDENTITY(1,1) NOT NULL,
   LineaID          INT              NOT NULL,
   EstacionOrigenID   INT              NOT NULL,
   EstacionDestinoID  INT              NOT NULL,
   DistanciaKm       DECIMAL(9,3)       NOT NULL,
   FechaActualizacion DATETIME2(0)        NOT NULL CONSTRAINT DF_Tramo_Fecha
DEFAULT SYSUTCDATETIME(),
   CONSTRAINT PK_TramoLinea PRIMARY KEY (TramoID),
   CONSTRAINT FK_Tramo_Origen FOREIGN KEY (LineaID, EstacionOrigenID)
      REFERENCES ops.LineaEstacion(LineaID, EstacionID),
   CONSTRAINT FK_Tramo_Destino FOREIGN KEY (LineaID, EstacionDestinoID)
      REFERENCES ops.LineaEstacion(LineaID, EstacionID),
   CONSTRAINT UQ_Tramo_Origen UNIQUE (LineaID, EstacionOrigenID),
   CONSTRAINT UQ_Tramo_Destino UNIQUE (LineaID, EstacionDestinoID),
   CONSTRAINT CK_Tramo_Estaciones CHECK (EstacionOrigenID <>
EstacionDestinoID),
   CONSTRAINT CK_Tramo_Distancia CHECK (DistanciaKm > 0)
);
GO
CREATE TABLE ops.Acceso
(
```

```
    AccesoID       INT IDENTITY(1,1) NOT NULL,
    EstacionID     INT               NOT NULL,
    Codigo         VARCHAR(30)       NOT NULL,
    Nombre         NVARCHAR(120)     NOT NULL,
    Descripcion    NVARCHAR(300)     NULL,
    TipoAcceso     VARCHAR(20)       NOT NULL CONSTRAINT DF_Acceso_Tipo
DEFAULT ('PEATONAL'),
    Estado         CHAR(1)           NOT NULL CONSTRAINT DF_Acceso_Estado DEFAULT
('P'),
    FechaCreacion  DATETIME2(0)      NOT NULL CONSTRAINT DF_Acceso_Fecha
DEFAULT SYSUTCDATETIME(),
    VersionFila    ROWVERSION        NOT NULL,
    CONSTRAINT PK_Acceso PRIMARY KEY (AccesoID),
    CONSTRAINT FK_Acceso_Estacion FOREIGN KEY (EstacionID)
        REFERENCES ops.Estacion(EstacionID),
    CONSTRAINT UQ_Acceso_Codigo UNIQUE (Codigo),
    CONSTRAINT CK_Acceso_Tipo CHECK (TipoAcceso IN
('PEATONAL','VEHICULAR','MIXTO','EMERGENCIA')),
    CONSTRAINT CK_Acceso_Estado CHECK (Estado IN ('P','A','I'))
);
GO
/*=============================================================
  3. PARQUEOS, FLOTA, ASIGNACIONES, RECORRIDOS Y OCUPACION
=============================================================*/
CREATE TABLE ops.Parqueo
```

```
(
    ParqueoID       INT IDENTITY(1,1) NOT NULL,
    EstacionID      INT               NOT NULL,
    Codigo          VARCHAR(30)       NOT NULL,
    Nombre          NVARCHAR(120)     NOT NULL,
    CapacidadBuses  SMALLINT          NOT NULL,
    Activo          BIT               NOT NULL CONSTRAINT DF_Parqueo_Activo DEFAULT (1),
    FechaCreacion   DATETIME2(0)      NOT NULL CONSTRAINT DF_Parqueo_Fecha
DEFAULT SYSUTCDATETIME(),
    CONSTRAINT PK_Parqueo PRIMARY KEY (ParqueoID),
    CONSTRAINT FK_Parqueo_Estacion FOREIGN KEY (EstacionID)
        REFERENCES ops.Estacion(EstacionID),
    CONSTRAINT UQ_Parqueo_Codigo UNIQUE (Codigo),
    CONSTRAINT CK_Parqueo_Capacidad CHECK (CapacidadBuses > 0)
);
GO
CREATE TABLE ops.Bus
(
    BusID           INT IDENTITY(1,1) NOT NULL,
    ParqueoID       INT               NOT NULL,
    CodigoInterno   VARCHAR(30)       NOT NULL,
    Placa           VARCHAR(20)       NOT NULL,
    Marca           NVARCHAR(60)      NULL,
    Modelo          NVARCHAR(60)      NULL,
    Anio            SMALLINT          NULL,
```

```
    CapacidadMaxima  SMALLINT          NOT NULL,
    Estado           VARCHAR(20)       NOT NULL CONSTRAINT DF_Bus_Estado DEFAULT ('DISPONIBLE'),
    FechaRegistro    DATETIME2(0)      NOT NULL CONSTRAINT DF_Bus_Fecha DEFAULT SYSUTCDATETIME(),
    VersionFila      ROWVERSION        NOT NULL,
    CONSTRAINT PK_Bus PRIMARY KEY (BusID),
    CONSTRAINT FK_Bus_Parqueo FOREIGN KEY (ParqueoID)
        REFERENCES ops.Parqueo(ParqueoID),
    CONSTRAINT UQ_Bus_Codigo UNIQUE (CodigoInterno),
    CONSTRAINT UQ_Bus_Placa UNIQUE (Placa),
    CONSTRAINT CK_Bus_Anio CHECK (Anio IS NULL OR Anio BETWEEN 2000 AND 2100),
    CONSTRAINT CK_Bus_Capacidad CHECK (CapacidadMaxima > 0),
    CONSTRAINT CK_Bus_Estado CHECK (Estado IN ('DISPONIBLE','EN_RUTA','MANTENIMIENTO','INACTIVO'))
);
GO
CREATE TABLE ops.LineaBus
(
    BusID            INT               NOT NULL,
    LineaID          INT               NOT NULL,
    FechaAsignacion  DATETIME2(0)      NOT NULL CONSTRAINT DF_LineaBus_Fecha DEFAULT SYSUTCDATETIME(),
    CONSTRAINT PK_LineaBus PRIMARY KEY (BusID),
```

```
    CONSTRAINT FK_LineaBus_Bus FOREIGN KEY (BusID) REFERENCES ops.Bus(BusID),
    CONSTRAINT FK_LineaBus_Linea FOREIGN KEY (LineaID) REFERENCES ops.Linea(LineaID)
);
GO
CREATE TABLE ops.HistorialParqueoBus
(
    HistorialParqueoID BIGINT IDENTITY(1,1) NOT NULL,
    BusID              INT              NOT NULL,
    ParqueoAnteriorID  INT              NULL,
    ParqueoNuevoID     INT              NOT NULL,
    FechaCambio        DATETIME2(0)     NOT NULL CONSTRAINT DF_HistParqueo_Fecha DEFAULT SYSUTCDATETIME(),
    UsuarioCambioID    INT              NULL,
    Motivo             NVARCHAR(300)    NULL,
    CONSTRAINT PK_HistorialParqueoBus PRIMARY KEY (HistorialParqueoID),
    CONSTRAINT FK_HistParqueo_Bus FOREIGN KEY (BusID) REFERENCES ops.Bus(BusID),
    CONSTRAINT FK_HistParqueo_Anterior FOREIGN KEY (ParqueoAnteriorID) REFERENCES ops.Parqueo(ParqueoID),
    CONSTRAINT FK_HistParqueo_Nuevo FOREIGN KEY (ParqueoNuevoID) REFERENCES ops.Parqueo(ParqueoID),
    CONSTRAINT CK_HistParqueo_Cambio CHECK (ParqueoAnteriorID IS NULL OR ParqueoAnteriorID <> ParqueoNuevoID)
);
```

```
GO
CREATE TABLE ops.Viaje
(
   ViajeID        BIGINT IDENTITY(1,1) NOT NULL,
   LineaID        INT                  NOT NULL,
   BusID          INT                  NOT NULL,
   PilotoID       INT                  NULL,
   FechaSalida    DATETIME2(0)         NOT NULL,
   FechaLlegada   DATETIME2(0)         NULL,
   Estado         VARCHAR(20)          NOT NULL CONSTRAINT DF_Viaje_Estado DEFAULT ('PROGRAMADO'),
   CONSTRAINT PK_Viaje PRIMARY KEY (ViajeID),
   CONSTRAINT FK_Viaje_Linea FOREIGN KEY (LineaID) REFERENCES ops.Linea(LineaID),
   CONSTRAINT FK_Viaje_Bus FOREIGN KEY (BusID) REFERENCES ops.Bus(BusID),
   CONSTRAINT CK_Viaje_Fechas CHECK (FechaLlegada IS NULL OR FechaLlegada >= FechaSalida),
   CONSTRAINT CK_Viaje_Estado CHECK (Estado IN ('PROGRAMADO','EN_CURSO','FINALIZADO','CANCELADO'))
);
GO
CREATE TABLE ops.VisitaEstacionViaje
(
   ViajeID        BIGINT       NOT NULL,
   EstacionID     INT          NOT NULL,
```

```
    OrdenVisita     SMALLINT     NOT NULL,
    LlegadaProgramada DATETIME2(0) NULL,
    LlegadaReal     DATETIME2(0) NULL,
    SalidaReal      DATETIME2(0) NULL,
    CONSTRAINT PK_VisitaEstacionViaje PRIMARY KEY (ViajeID, EstacionID),
    CONSTRAINT FK_Visita_Viaje FOREIGN KEY (ViajeID) REFERENCES ops.Viaje(ViajeID),
    CONSTRAINT FK_Visita_Estacion FOREIGN KEY (EstacionID) REFERENCES ops.Estacion(EstacionID),
    CONSTRAINT UQ_Visita_Orden UNIQUE (ViajeID, OrdenVisita),
    CONSTRAINT CK_Visita_Orden CHECK (OrdenVisita > 0),
    CONSTRAINT CK_Visita_Fechas CHECK (SalidaReal IS NULL OR LlegadaReal IS NULL OR SalidaReal >= LlegadaReal)
);
GO
CREATE TABLE ops.ConteoEstacion
(
    ConteoEstacionID BIGINT IDENTITY(1,1) NOT NULL,
    EstacionID      INT             NOT NULL,
    CantidadPasajeros INT           NOT NULL,
    CapacidadEvaluada SMALLINT      NOT NULL,
    FechaRegistro   DATETIME2(0)    NOT NULL CONSTRAINT DF_Conteo_Fecha DEFAULT SYSUTCDATETIME(),
    OperadorID      INT             NULL,
    CONSTRAINT PK_ConteoEstacion PRIMARY KEY (ConteoEstacionID),
```

```
    CONSTRAINT FK_Conteo_Estacion FOREIGN KEY (EstacionID) REFERENCES ops.Estacion(EstacionID),
    CONSTRAINT CK_Conteo_Cantidad CHECK (CantidadPasajeros >= 0),
    CONSTRAINT CK_Conteo_Capacidad CHECK (CapacidadEvaluada > 0)
);
GO
CREATE TABLE ops.AlertaOcupacion
(
    AlertaID        BIGINT IDENTITY(1,1) NOT NULL,
    ConteoEstacionID BIGINT             NOT NULL,
    EstacionID      INT                 NOT NULL,
    Porcentaje      DECIMAL(7,2)        NOT NULL,
    RequiereUnidad  BIT                 NOT NULL CONSTRAINT DF_Alerta_Unidad DEFAULT (1),
    Estado          VARCHAR(20)         NOT NULL CONSTRAINT DF_Alerta_Estado DEFAULT ('PENDIENTE'),
    FechaGeneracion DATETIME2(0)        NOT NULL CONSTRAINT DF_Alerta_Fecha DEFAULT SYSUTCDATETIME(),
    FechaAtencion   DATETIME2(0)        NULL,
    Observacion     NVARCHAR(500)       NULL,
    CONSTRAINT PK_AlertaOcupacion PRIMARY KEY (AlertaID),
    CONSTRAINT FK_Alerta_Conteo FOREIGN KEY (ConteoEstacionID) REFERENCES ops.ConteoEstacion(ConteoEstacionID),
    CONSTRAINT FK_Alerta_Estacion FOREIGN KEY (EstacionID) REFERENCES ops.Estacion(EstacionID),
    CONSTRAINT UQ_Alerta_Conteo UNIQUE (ConteoEstacionID),
```

```
    CONSTRAINT CK_Alerta_Porcentaje CHECK (Porcentaje >= 150),
    CONSTRAINT CK_Alerta_Estado CHECK (Estado IN ('PENDIENTE','ATENDIDA','CANCELADA'))
);
GO
CREATE TABLE ops.RegistroOcupacionBus
(
    RegistroOcupacionID BIGINT IDENTITY(1,1) NOT NULL,
    BusID INT NOT NULL,
    EstacionID INT NOT NULL,
    ViajeID BIGINT NULL,
    CantidadPasajeros SMALLINT NOT NULL,
    CapacidadEvaluada SMALLINT NOT NULL,
    PorcentajeOcupacion DECIMAL(7,2) NOT NULL,
    FechaRegistro DATETIME2(0) NOT NULL CONSTRAINT DF_RegOcupacion_Fecha DEFAULT SYSUTCDATETIME(),
    CONSTRAINT PK_RegistroOcupacionBus PRIMARY KEY (RegistroOcupacionID),
    CONSTRAINT FK_RegOcupacion_Bus FOREIGN KEY (BusID) REFERENCES ops.Bus(BusID),
    CONSTRAINT FK_RegOcupacion_Estacion FOREIGN KEY (EstacionID) REFERENCES ops.Estacion(EstacionID),
    CONSTRAINT FK_RegOcupacion_Viaje FOREIGN KEY (ViajeID) REFERENCES ops.Viaje(ViajeID),
    CONSTRAINT CK_RegOcupacion_Cantidad CHECK (CantidadPasajeros >= 0),
    CONSTRAINT CK_RegOcupacion_Capacidad CHECK (CapacidadEvaluada > 0),
    CONSTRAINT CK_RegOcupacion_Porcentaje CHECK (PorcentajeOcupacion >= 0)
```

```
);
GO
CREATE TABLE ops.EsperaAdicional
(
   EsperaID           BIGINT IDENTITY(1,1) NOT NULL,
   RegistroOcupacionID BIGINT             NOT NULL,
   MinutosAdicionales  TINYINT            NOT NULL CONSTRAINT DF_Espera_Minutos
DEFAULT (5),
   Aplicada           BIT                 NOT NULL CONSTRAINT DF_Espera_Aplicada DEFAULT
(0),
   FechaIndicacion    DATETIME2(0)        NOT NULL CONSTRAINT DF_Espera_Fecha
DEFAULT SYSUTCDATETIME(),
   FechaAplicacion    DATETIME2(0)        NULL,
   CONSTRAINT PK_EsperaAdicional PRIMARY KEY (EsperaID),
   CONSTRAINT FK_Espera_Registro FOREIGN KEY (RegistroOcupacionID)
      REFERENCES ops.RegistroOcupacionBus(RegistroOcupacionID),
   CONSTRAINT UQ_Espera_Registro UNIQUE (RegistroOcupacionID),
   CONSTRAINT CK_Espera_Minutos CHECK (MinutosAdicionales = 5)
);
GO
/*================================================================
  4. PERSONAL: PILOTOS, EDUCACION, GUARDIAS Y OPERADORES
================================================================*/
CREATE TABLE ops.Piloto
(
```

```
    PilotoID        INT IDENTITY(1,1) NOT NULL,
    CodigoEmpleado  VARCHAR(30)       NOT NULL,
    Nombres         NVARCHAR(100)     NOT NULL,
    Apellidos       NVARCHAR(100)     NOT NULL,
    DPIHash         VARBINARY(64)     NULL,
    Direccion       NVARCHAR(300)     NOT NULL,
    Municipio       NVARCHAR(100)     NOT NULL,
    Departamento    NVARCHAR(100)     NOT NULL,
    Telefono        VARCHAR(25)       NOT NULL,
    Correo          VARCHAR(254)      NULL,
    Licencia        VARCHAR(30)       NOT NULL,
    FechaVencimientoLicencia DATE     NOT NULL,
    Activo          BIT               NOT NULL CONSTRAINT DF_Piloto_Activo DEFAULT (1),
    FechaRegistro   DATETIME2(0)      NOT NULL CONSTRAINT DF_Piloto_Fecha
DEFAULT SYSUTCDATETIME(),
    VersionFila     ROWVERSION        NOT NULL,
    CONSTRAINT PK_Piloto PRIMARY KEY (PilotoID),
    CONSTRAINT UQ_Piloto_Codigo UNIQUE (CodigoEmpleado),
    CONSTRAINT UQ_Piloto_Licencia UNIQUE (Licencia),
    CONSTRAINT CK_Piloto_Correo CHECK (Correo IS NULL OR Correo LIKE
'%_@_%._%')
);
GO
CREATE TABLE ops.HistorialEducativoPiloto
(
```

```
    HistorialEducativoID INT IDENTITY(1,1) NOT NULL,
    PilotoID            INT             NOT NULL,
    Nivel               VARCHAR(30)     NOT NULL,
    Institucion         NVARCHAR(200)   NOT NULL,
    Titulo              NVARCHAR(200)   NULL,
    AnioFinalizacion    SMALLINT        NULL,
    Observacion         NVARCHAR(500)   NULL,
    CONSTRAINT PK_HistorialEducativoPiloto PRIMARY KEY (HistorialEducativoID),
    CONSTRAINT FK_HistorialEducativo_Piloto FOREIGN KEY (PilotoID) REFERENCES ops.Piloto(PilotoID),
    CONSTRAINT CK_HistorialEducativo_Nivel CHECK (Nivel IN ('PRIMARIA','BASICOS','DIVERSIFICADO','TECNICO','UNIVERSITARIO','OTRO')),
    CONSTRAINT CK_HistorialEducativo_Anio CHECK (AnioFinalizacion IS NULL OR AnioFinalizacion BETWEEN 1950 AND 2100)
);
GO
ALTER TABLE ops.Viaje
ADD CONSTRAINT FK_Viaje_Piloto FOREIGN KEY (PilotoID) REFERENCES ops.Piloto(PilotoID);
GO

CREATE TABLE ops.Guardia
(
    GuardiaID       INT IDENTITY(1,1) NOT NULL,
    CodigoEmpleado  VARCHAR(30)       NOT NULL,
```

```
    Nombres         NVARCHAR(100)     NOT NULL,
    Apellidos       NVARCHAR(100)     NOT NULL,
    Telefono        VARCHAR(25)       NULL,
    Empresa         NVARCHAR(150)     NULL,
    Activo          BIT               NOT NULL CONSTRAINT DF_Guardia_Activo DEFAULT (1),
    FechaRegistro   DATETIME2(0)      NOT NULL CONSTRAINT DF_Guardia_Fecha DEFAULT SYSUTCDATETIME(),
    CONSTRAINT PK_Guardia PRIMARY KEY (GuardiaID),
    CONSTRAINT UQ_Guardia_Codigo UNIQUE (CodigoEmpleado)
);
GO
CREATE TABLE ops.AsignacionGuardiaAcceso
(
    AsignacionGuardiaID BIGINT IDENTITY(1,1) NOT NULL,
    GuardiaID       INT               NOT NULL,
    AccesoID        INT               NOT NULL,
    FechaInicio     DATETIME2(0)      NOT NULL CONSTRAINT DF_AsigGuardia_Inicio DEFAULT SYSUTCDATETIME(),
    FechaFin        DATETIME2(0)      NULL,
    Turno           VARCHAR(20)       NOT NULL CONSTRAINT DF_AsigGuardia_Turno DEFAULT ('DIURNO'),
    CONSTRAINT PK_AsignacionGuardiaAcceso PRIMARY KEY (AsignacionGuardiaID),
    CONSTRAINT FK_AsigGuardia_Guardia FOREIGN KEY (GuardiaID) REFERENCES ops.Guardia(GuardiaID),
    CONSTRAINT FK_AsigGuardia_Acceso FOREIGN KEY (AccesoID) REFERENCES ops.Acceso(AccesoID),
```

```
    CONSTRAINT CK_AsigGuardia_Fechas CHECK (FechaFin IS NULL OR FechaFin > FechaInicio),
    CONSTRAINT CK_AsigGuardia_Turno CHECK (Turno IN ('DIURNO','NOCTURNO','MIXTO'))
);
GO
CREATE UNIQUE INDEX UX_AsigGuardia_Activa
ON ops.AsignacionGuardiaAcceso(GuardiaID)
WHERE FechaFin IS NULL;
GO
CREATE TABLE ops.Operador
(
    OperadorID      INT IDENTITY(1,1) NOT NULL,
    CodigoEmpleado  VARCHAR(30)       NOT NULL,
    Nombres         NVARCHAR(100)     NOT NULL,
    Apellidos       NVARCHAR(100)     NOT NULL,
    Telefono        VARCHAR(25)       NULL,
    Correo          VARCHAR(254)      NULL,
    Activo          BIT               NOT NULL CONSTRAINT DF_Operador_Activo DEFAULT (1),
    FechaRegistro   DATETIME2(0)      NOT NULL CONSTRAINT DF_Operador_Fecha DEFAULT SYSUTCDATETIME(),
    CONSTRAINT PK_Operador PRIMARY KEY (OperadorID),
    CONSTRAINT UQ_Operador_Codigo UNIQUE (CodigoEmpleado),
    CONSTRAINT CK_Operador_Correo CHECK (Correo IS NULL OR Correo LIKE '%_@_%._%')
```

```
);
GO
CREATE TABLE ops.AsignacionOperadorEstacion
(
    AsignacionOperadorID BIGINT IDENTITY(1,1) NOT NULL,
    OperadorID          INT              NOT NULL,
    EstacionID          INT              NOT NULL,
    FechaInicio                 DATETIME2(0)                NOT NULL CONSTRAINT
DF_AsigOperador_Inicio DEFAULT SYSUTCDATETIME(),
    FechaFin            DATETIME2(0)        NULL,
    CONSTRAINT          PK_AsignacionOperadorEstacion          PRIMARY          KEY
(AsignacionOperadorID),
    CONSTRAINT          FK_AsigOperador_Operador          FOREIGN          KEY          (OperadorID)
REFERENCES ops.Operador(OperadorID),
    CONSTRAINT FK_AsigOperador_Estacion FOREIGN KEY (EstacionID) REFERENCES
ops.Estacion(EstacionID),
    CONSTRAINT CK_AsigOperador_Fechas CHECK (FechaFin IS NULL OR FechaFin >
FechaInicio)
);
GO
CREATE UNIQUE INDEX UX_AsigOperador_OperadorActivo
ON ops.AsignacionOperadorEstacion(OperadorID)
WHERE FechaFin IS NULL;
GO
CREATE UNIQUE INDEX UX_AsigOperador_EstacionActiva
ON ops.AsignacionOperadorEstacion(EstacionID)
```

```
WHERE FechaFin IS NULL;
GO
/* FK diferidas de responsables que ya cuentan con tabla de personal. */
ALTER TABLE ops.ConteoEstacion
ADD CONSTRAINT FK_Conteo_Operador FOREIGN KEY (OperadorID) REFERENCES ops.Operador(OperadorID);
GO
/*=====================================================================
  5. USUARIOS, ROLES, PERMISOS Y AUDITORIA
=====================================================================*/
CREATE TABLE seg.Usuario
(
   UsuarioID       INT IDENTITY(1,1) NOT NULL,
   OperadorID      INT               NULL,
   NombreUsuario   NVARCHAR(100)     NOT NULL,
   Correo          VARCHAR(254)      NOT NULL,
   ContrasenaHash  VARBINARY(64)     NOT NULL,
   ContrasenaSalt  VARBINARY(32)     NOT NULL,
   Activo          BIT               NOT NULL CONSTRAINT DF_Usuario_Activo DEFAULT (1),
   IntentosFallidos TINYINT          NOT NULL CONSTRAINT DF_Usuario_Intentos DEFAULT (0),
   BloqueadoHasta  DATETIME2(0)      NULL,
   UltimoAcceso    DATETIME2(0)      NULL,
```

```
    FechaCreacion     DATETIME2(0)        NOT NULL CONSTRAINT DF_Usuario_Fecha
DEFAULT SYSUTCDATETIME(),
    VersionFila       ROWVERSION          NOT NULL,
    CONSTRAINT PK_Usuario PRIMARY KEY (UsuarioID),
    CONSTRAINT FK_Usuario_Operador FOREIGN KEY (OperadorID) REFERENCES
ops.Operador(OperadorID),
    CONSTRAINT UQ_Usuario_Nombre UNIQUE (NombreUsuario),
    CONSTRAINT UQ_Usuario_Correo UNIQUE (Correo),
    CONSTRAINT UQ_Usuario_Operador UNIQUE (OperadorID),
    CONSTRAINT CK_Usuario_Correo CHECK (Correo LIKE '%_@_%._%'),
    CONSTRAINT CK_Usuario_Intentos CHECK (IntentosFallidos BETWEEN 0 AND 10)
);
GO
CREATE TABLE seg.Rol
(
    RolID       SMALLINT IDENTITY(1,1) NOT NULL,
    Nombre      NVARCHAR(80)           NOT NULL,
    Descripcion NVARCHAR(300)          NULL,
    Activo      BIT                    NOT NULL CONSTRAINT DF_Rol_Activo DEFAULT (1),
    CONSTRAINT PK_Rol PRIMARY KEY (RolID),
    CONSTRAINT UQ_Rol_Nombre UNIQUE (Nombre)
);
GO
CREATE TABLE seg.Permiso
(
```

```
    PermisoID   SMALLINT IDENTITY(1,1) NOT NULL,
    Codigo      VARCHAR(80)          NOT NULL,
    Descripcion NVARCHAR(300)        NOT NULL,
    CONSTRAINT PK_Permiso PRIMARY KEY (PermisoID),
    CONSTRAINT UQ_Permiso_Codigo UNIQUE (Codigo)
);
GO
CREATE TABLE seg.UsuarioRol
(
    UsuarioID     INT         NOT NULL,
    RolID         SMALLINT    NOT NULL,
    FechaAsignacion DATETIME2(0) NOT NULL CONSTRAINT DF_UsuarioRol_Fecha
DEFAULT SYSUTCDATETIME(),
    CONSTRAINT PK_UsuarioRol PRIMARY KEY (UsuarioID, RolID),
    CONSTRAINT FK_UsuarioRol_Usuario FOREIGN KEY (UsuarioID) REFERENCES
seg.Usuario(UsuarioID),
    CONSTRAINT FK_UsuarioRol_Rol FOREIGN KEY (RolID) REFERENCES
seg.Rol(RolID)
);
GO
CREATE TABLE seg.RolPermiso
(
    RolID     SMALLINT NOT NULL,
    PermisoID SMALLINT NOT NULL,
    CONSTRAINT PK_RolPermiso PRIMARY KEY (RolID, PermisoID),
```

```
    CONSTRAINT FK_RolPermiso_Rol FOREIGN KEY (RolID) REFERENCES seg.Rol(RolID),
    CONSTRAINT FK_RolPermiso_Permiso FOREIGN KEY (PermisoID) REFERENCES seg.Permiso(PermisoID)
);
GO
CREATE TABLE aud.Bitacora
(
    BitacoraID     BIGINT IDENTITY(1,1) NOT NULL,
    UsuarioID      INT              NULL,
    Esquema        SYSNAME          NOT NULL,
    Tabla          SYSNAME          NOT NULL,
    Operacion      CHAR(1)          NOT NULL,
    ClaveRegistro  NVARCHAR(200)    NULL,
    DatosAnteriores NVARCHAR(MAX)   NULL,
    DatosNuevos    NVARCHAR(MAX)    NULL,
    FechaHora      DATETIME2(3)     NOT NULL CONSTRAINT DF_Bitacora_Fecha DEFAULT SYSUTCDATETIME(),
    UsuarioBD      SYSNAME          NOT NULL CONSTRAINT DF_Bitacora_UsuarioBD DEFAULT ORIGINAL_LOGIN(),
    DireccionIP    VARCHAR(48)      NULL,
    CONSTRAINT PK_Bitacora PRIMARY KEY (BitacoraID),
    CONSTRAINT FK_Bitacora_Usuario FOREIGN KEY (UsuarioID) REFERENCES seg.Usuario(UsuarioID),
    CONSTRAINT CK_Bitacora_Operacion CHECK (Operacion IN ('I','U','D')),
```

```
    CONSTRAINT CK_Bitacora_JSON_Anterior CHECK (DatosAnteriores IS NULL OR ISJSON(DatosAnteriores) = 1),
    CONSTRAINT CK_Bitacora_JSON_Nuevo CHECK (DatosNuevos IS NULL OR ISJSON(DatosNuevos) = 1)
);
GO
ALTER TABLE ops.HistorialParqueoBus
ADD CONSTRAINT FK_HistParqueo_Usuario FOREIGN KEY (UsuarioCambioID) REFERENCES seg.Usuario(UsuarioID);
GO

/*=====================================================================
  6. TARJETAS, RECARGAS Y PAGOS
=====================================================================*/
CREATE TABLE fin.TitularTarjeta
(
    TitularID      BIGINT IDENTITY(1,1) NOT NULL,
    Nombres        NVARCHAR(100)        NOT NULL,
    Apellidos      NVARCHAR(100)        NOT NULL,
    Correo         VARCHAR(254)         NULL,
    Telefono       VARCHAR(25)          NULL,
    DocumentoHash  VARBINARY(64)        NULL,
    FechaRegistro  DATETIME2(0)         NOT NULL CONSTRAINT DF_Titular_Fecha DEFAULT SYSUTCDATETIME(),
    CONSTRAINT PK_TitularTarjeta PRIMARY KEY (TitularID),
```

```
    CONSTRAINT CK_Titular_Correo CHECK (Correo IS NULL OR Correo LIKE '%_@_%._%')
);
GO
CREATE TABLE fin.TarjetaAcceso
(
    TarjetaID      BIGINT IDENTITY(1,1) NOT NULL,
    TitularID      BIGINT           NULL,
    NumeroToken    VARCHAR(64)      NOT NULL,
    Saldo          DECIMAL(12,2)    NOT NULL CONSTRAINT DF_Tarjeta_Saldo DEFAULT (0),
    Estado         VARCHAR(20)      NOT NULL CONSTRAINT DF_Tarjeta_Estado DEFAULT ('ACTIVA'),
    FechaEmision   DATE             NOT NULL CONSTRAINT DF_Tarjeta_Emision DEFAULT CAST(SYSUTCDATETIME() AS DATE),
    FechaVencimiento DATE           NULL,
    VersionFila    ROWVERSION       NOT NULL,
    CONSTRAINT PK_TarjetaAcceso PRIMARY KEY (TarjetaID),
    CONSTRAINT FK_Tarjeta_Titular FOREIGN KEY (TitularID) REFERENCES fin.TitularTarjeta(TitularID),
    CONSTRAINT UQ_Tarjeta_Token UNIQUE (NumeroToken),
    CONSTRAINT CK_Tarjeta_Saldo CHECK (Saldo >= 0),
    CONSTRAINT CK_Tarjeta_Estado CHECK (Estado IN ('ACTIVA','BLOQUEADA','VENCIDA','INACTIVA')),
    CONSTRAINT CK_Tarjeta_Vigencia CHECK (FechaVencimiento IS NULL OR FechaVencimiento >= FechaEmision)
```

```
);
GO
CREATE TABLE fin.TransaccionTarjeta
(
    TransaccionID  BIGINT IDENTITY(1,1) NOT NULL,
    TarjetaID      BIGINT              NOT NULL,
    EstacionID     INT                 NULL,
    Tipo           CHAR(1)             NOT NULL,
    Monto          DECIMAL(12,2)       NOT NULL,
    SaldoAnterior  DECIMAL(12,2)       NOT NULL,
    SaldoNuevo     DECIMAL(12,2)       NOT NULL,
    Referencia     VARCHAR(80)         NOT NULL,
    FechaHora      DATETIME2(0)        NOT NULL CONSTRAINT DF_Transaccion_Fecha DEFAULT SYSUTCDATETIME(),
    UsuarioID      INT                 NULL,
    CONSTRAINT PK_TransaccionTarjeta PRIMARY KEY (TransaccionID),
    CONSTRAINT FK_Transaccion_Tarjeta FOREIGN KEY (TarjetaID) REFERENCES fin.TarjetaAcceso(TarjetaID),
    CONSTRAINT FK_Transaccion_Estacion FOREIGN KEY (EstacionID) REFERENCES ops.Estacion(EstacionID),
    CONSTRAINT FK_Transaccion_Usuario FOREIGN KEY (UsuarioID) REFERENCES seg.Usuario(UsuarioID),
    CONSTRAINT UQ_Transaccion_Referencia UNIQUE (Referencia),
    CONSTRAINT CK_Transaccion_Tipo CHECK (Tipo IN ('R','P')),
    CONSTRAINT CK_Transaccion_Monto CHECK (Monto > 0),
```

```
    CONSTRAINT CK_Transaccion_Saldos CHECK (SaldoAnterior >= 0 AND SaldoNuevo
>= 0),
    CONSTRAINT CK_Transaccion_Calculo CHECK
    (
        (Tipo = 'R' AND SaldoNuevo = SaldoAnterior + Monto) OR
        (Tipo = 'P' AND SaldoNuevo = SaldoAnterior - Monto)
    )
);
GO
/*=====================================================================
  7. INDICES DE CONSULTA Y RENDIMIENTO
=====================================================================*/
CREATE INDEX IX_Linea_Municipalidad ON ops.Linea(MunicipalidadID, Estado);
CREATE INDEX IX_Estacion_Municipalidad ON ops.Estacion(MunicipalidadID, Estado);
CREATE INDEX IX_LineaEstacion_Estacion ON ops.LineaEstacion(EstacionID, LineaID);
CREATE INDEX IX_Acceso_Estacion ON ops.Acceso(EstacionID, Estado);
CREATE INDEX IX_Parqueo_Estacion ON ops.Parqueo(EstacionID, Activo);
CREATE INDEX IX_LineaBus_Linea ON ops.LineaBus(LineaID, BusID);
CREATE INDEX IX_Alerta_EstadoFecha ON ops.AlertaOcupacion(Estado,
FechaGeneracion DESC);
CREATE INDEX IX_Conteo_EstacionFecha ON ops.ConteoEstacion(EstacionID,
FechaRegistro DESC);
CREATE INDEX IX_RegOcupacion_BusFecha ON ops.RegistroOcupacionBus(BusID,
FechaRegistro DESC);
CREATE INDEX IX_Transaccion_TarjetaFecha ON fin.TransaccionTarjeta(TarjetaID,
FechaHora DESC);
```

```
CREATE INDEX IX_Bitacora_TablaFecha ON aud.Bitacora(Esquema, Tabla, FechaHora DESC);

CREATE INDEX IX_Bitacora_UsuarioFecha ON aud.Bitacora(UsuarioID, FechaHora DESC);

GO

/*=============================================================================

  8. TRIGGERS DE REGLAS DE NEGOCIO

=============================================================================*/

CREATE OR ALTER TRIGGER ops.tr_TramoLinea_Consecutivo

ON ops.TramoLinea

AFTER INSERT, UPDATE

AS

BEGIN

   SET NOCOUNT ON;

   IF EXISTS

   (

      SELECT 1

      FROM inserted i

      JOIN ops.LineaEstacion o ON o.LineaID=i.LineaID AND o.EstacionID=i.EstacionOrigenID

      JOIN ops.LineaEstacion d ON d.LineaID=i.LineaID AND d.EstacionID=i.EstacionDestinoID

      WHERE d.Orden <> o.Orden + 1

   )

      THROW 51001, 'El tramo solo puede relacionar estaciones consecutivas de la misma linea.', 1;
```

```
END;
GO
CREATE OR ALTER TRIGGER ops.tr_LineaBus_Reglas
ON ops.LineaBus
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
  SET NOCOUNT ON;
  IF EXISTS
  (
    SELECT 1
    FROM ops.Linea l
    CROSS APPLY (SELECT COUNT(*) CantEst FROM ops.LineaEstacion le WHERE le.LineaID=l.LineaID) e
    CROSS APPLY (SELECT COUNT(*) CantBus FROM ops.LineaBus lb WHERE lb.LineaID=l.LineaID) b
    WHERE l.LineaID IN (SELECT LineaID FROM inserted UNION SELECT LineaID FROM deleted)
      AND
      (
        b.CantBus > 2 * e.CantEst
        OR (l.Estado='A' AND (e.CantEst=0 OR b.CantBus < e.CantEst))
      )
  )
```

```
    THROW 51002, 'La linea activa debe tener entre una y dos veces la cantidad de buses respecto de sus estaciones.', 1;
END;
GO
CREATE OR ALTER TRIGGER ops.tr_LineaEstacion_Reglas
ON ops.LineaEstacion
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
  SET NOCOUNT ON;
  IF EXISTS
  (
    SELECT 1
    FROM ops.Linea l
    CROSS APPLY (SELECT COUNT(*) CantEst FROM ops.LineaEstacion le WHERE le.LineaID=l.LineaID) e
    CROSS APPLY (SELECT COUNT(*) CantBus FROM ops.LineaBus lb WHERE lb.LineaID=l.LineaID) b
    WHERE l.LineaID IN (SELECT LineaID FROM inserted UNION SELECT LineaID FROM deleted)
      AND l.Estado='A'
      AND (e.CantEst=0 OR b.CantBus < e.CantEst OR b.CantBus > 2*e.CantEst)
  )
    THROW 51003, 'El cambio deja una linea activa fuera del limite permitido de buses.', 1;
  IF EXISTS
```

```
    (
        SELECT 1
        FROM ops.Estacion e
        WHERE e.EstacionID IN (SELECT EstacionID FROM deleted)
          AND e.Estado='A'
          AND NOT EXISTS (SELECT 1 FROM ops.LineaEstacion le WHERE le.EstacionID=e.EstacionID)
    )
        THROW 51004, 'Una estacion activa debe pertenecer al menos a una linea.', 1;
END;
GO
CREATE OR ALTER TRIGGER ops.tr_GuardiaAcceso_Minimo
ON ops.AsignacionGuardiaAcceso
AFTER UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS
    (
        SELECT 1
        FROM ops.Acceso a
        WHERE a.AccesoID IN (SELECT AccesoID FROM deleted)
          AND a.Estado='A'
          AND NOT EXISTS
              (SELECT 1 FROM ops.AsignacionGuardiaAcceso x
```

```
            WHERE x.AccesoID=a.AccesoID AND x.FechaFin IS NULL)
    )
        THROW 51005, 'Un acceso activo debe conservar por lo menos un guardia asignado.', 1;
END;
GO
CREATE OR ALTER TRIGGER ops.tr_OperadorEstacion_Minimo
ON ops.AsignacionOperadorEstacion
AFTER UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS
    (
        SELECT 1
        FROM ops.Estacion e
        WHERE e.EstacionID IN (SELECT EstacionID FROM deleted)
          AND e.Estado='A'
          AND NOT EXISTS
              (SELECT 1 FROM ops.AsignacionOperadorEstacion x
               WHERE x.EstacionID=e.EstacionID AND x.FechaFin IS NULL)
    )
        THROW 51006, 'Una estacion activa debe conservar un operador responsable.', 1;
END;
GO
```

```
CREATE OR ALTER TRIGGER ops.tr_Bus_HistorialParqueo
ON ops.Bus
AFTER INSERT, UPDATE
AS
BEGIN
   SET NOCOUNT ON;
   IF EXISTS
   (
      SELECT 1 FROM inserted i
      JOIN ops.Parqueo p ON p.ParqueoID=i.ParqueoID
      WHERE p.Activo=0
        OR (SELECT COUNT(*) FROM ops.Bus b WHERE
b.ParqueoID=i.ParqueoID)>p.CapacidadBuses
   )
      THROW 51014, 'El parqueo esta inactivo o no tiene espacios disponibles.', 1;
   INSERT ops.HistorialParqueoBus
      (BusID, ParqueoAnteriorID, ParqueoNuevoID, UsuarioCambioID, Motivo)
   SELECT i.BusID, d.ParqueoID, i.ParqueoID,
         TRY_CONVERT(INT, SESSION_CONTEXT(N'UsuarioAplicacionID')),
         N'Reasignacion de parqueo'
   FROM inserted i
   JOIN deleted d ON d.BusID=i.BusID
   WHERE i.ParqueoID <> d.ParqueoID;
END;
GO
```

```
CREATE OR ALTER TRIGGER ops.tr_AsignacionGuardia_ValidarActivo
ON ops.AsignacionGuardiaAcceso
AFTER INSERT, UPDATE
AS
BEGIN
   SET NOCOUNT ON;
   IF EXISTS
   (
      SELECT 1 FROM inserted i
      JOIN ops.Guardia g ON g.GuardiaID=i.GuardiaID
      JOIN ops.Acceso a ON a.AccesoID=i.AccesoID
      WHERE i.FechaFin IS NULL AND (g.Activo=0 OR a.Estado='I')
   )
      THROW 51015, 'Una asignacion vigente requiere un guardia activo y un acceso no inactivo.', 1;
END;
GO
CREATE OR ALTER TRIGGER ops.tr_AsignacionOperador_ValidarActivo
ON ops.AsignacionOperadorEstacion
AFTER INSERT, UPDATE
AS
BEGIN
   SET NOCOUNT ON;
   IF EXISTS
   (
```

```
        SELECT 1 FROM inserted i
        JOIN ops.Operador o ON o.OperadorID=i.OperadorID
        JOIN ops.Estacion e ON e.EstacionID=i.EstacionID
        WHERE i.FechaFin IS NULL AND (o.Activo=0 OR e.Estado='I')
    )
        THROW 51016, 'Una asignacion vigente requiere un operador activo y una estacion no inactiva.', 1;
END;
GO
CREATE OR ALTER TRIGGER ops.tr_Viaje_ValidarAsignacion
ON ops.Viaje
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS
    (
        SELECT 1
        FROM inserted i
        WHERE i.Estado<>'CANCELADO'
          AND NOT EXISTS
            (SELECT 1 FROM ops.LineaBus lb WHERE lb.BusID=i.BusID AND lb.LineaID=i.LineaID)
    )
        THROW 51008, 'El bus del viaje debe encontrarse asignado a la linea seleccionada.', 1;
```

```
    IF EXISTS
    (
        SELECT 1 FROM inserted i
        JOIN ops.Piloto p ON p.PilotoID=i.PilotoID
        WHERE p.Activo=0
    )
        THROW 51009, 'No se puede asignar un piloto inactivo al viaje.', 1;
END;
GO
CREATE OR ALTER TRIGGER ops.tr_VisitaEstacion_ValidarRuta
ON ops.VisitaEstacionViaje
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS
    (
        SELECT 1
        FROM inserted i
        JOIN ops.Viaje v ON v.ViajeID=i.ViajeID
        LEFT JOIN ops.LineaEstacion le
          ON le.LineaID=v.LineaID AND le.EstacionID=i.EstacionID AND
le.Orden=i.OrdenVisita
        WHERE le.LineaID IS NULL
    )
```

```
    THROW 51010, 'La visita debe respetar las estaciones y el orden definido para la linea del viaje.', 1;
END;
GO
CREATE OR ALTER TRIGGER fin.tr_Tarjeta_ProtegerSaldo
ON fin.TarjetaAcceso
AFTER UPDATE
AS
BEGIN
  SET NOCOUNT ON;
  IF EXISTS (SELECT 1 FROM inserted i JOIN deleted d ON d.TarjetaID=i.TarjetaID WHERE i.Saldo <> d.Saldo)
    AND TRY_CONVERT(INT, SESSION_CONTEXT(N'MovimientoTarjetaAutorizado')) <> 1
    THROW 51007, 'El saldo solo puede modificarse mediante fin.sp_RegistrarMovimientoTarjeta.', 1;
END;
GO
CREATE OR ALTER TRIGGER fin.tr_Transaccion_ProtegerHistorial
ON fin.TransaccionTarjeta
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
  SET NOCOUNT ON;
  IF TRY_CONVERT(INT, SESSION_CONTEXT(N'MovimientoTarjetaAutorizado')) <> 1
```

```
    THROW 51011, 'Las transacciones solo pueden registrarse mediante fin.sp_RegistrarMovimientoTarjeta y no pueden editarse ni eliminarse.', 1;
  IF EXISTS (SELECT 1 FROM deleted)
    THROW 51012, 'El historial financiero es inmutable; no se permite actualizar ni eliminar transacciones.', 1;
  IF EXISTS
  (
    SELECT 1 FROM inserted i
    JOIN fin.TarjetaAcceso t ON t.TarjetaID=i.TarjetaID
    WHERE t.Saldo<>i.SaldoNuevo
  )
    THROW 51013, 'El saldo de la tarjeta no coincide con el saldo final de la transaccion.', 1;
END;
GO
CREATE OR ALTER TRIGGER ops.tr_Linea_Auditoria ON ops.Linea AFTER INSERT, UPDATE, DELETE AS
BEGIN
 SET NOCOUNT ON;
 INSERT aud.Bitacora(UsuarioID,Esquema,Tabla,Operacion,ClaveRegistro,DatosAnteriores,DatosNuevos,DireccionIP)
 SELECT TRY_CONVERT(INT,SESSION_CONTEXT(N'UsuarioAplicacionID')),N'ops',N'Linea',
    CASE WHEN i.LineaID IS NULL THEN 'D' WHEN d.LineaID IS NULL THEN 'I' ELSE 'U' END,
    CONVERT(NVARCHAR(200),COALESCE(i.LineaID,d.LineaID)),
```

```
    CASE WHEN d.LineaID IS NULL THEN NULL ELSE (SELECT d.* FOR JSON
PATH, WITHOUT_ARRAY_WRAPPER) END,

    CASE WHEN i.LineaID IS NULL THEN NULL ELSE (SELECT i.* FOR JSON PATH,
WITHOUT_ARRAY_WRAPPER) END,

    CONVERT(VARCHAR(48),CONNECTIONPROPERTY('client_net_address'))

 FROM inserted i FULL JOIN deleted d ON d.LineaID=i.LineaID;

END;

GO

CREATE OR ALTER TRIGGER ops.tr_Estacion_Auditoria ON ops.Estacion AFTER
INSERT, UPDATE, DELETE AS

BEGIN

 SET NOCOUNT ON;

 INSERT
aud.Bitacora(UsuarioID,Esquema,Tabla,Operacion,ClaveRegistro,DatosAnteriores,DatosNue
vos,DireccionIP)

 SELECT
TRY_CONVERT(INT,SESSION_CONTEXT(N'UsuarioAplicacionID')),N'ops',N'Estacion',

    CASE WHEN i.EstacionID IS NULL THEN 'D' WHEN d.EstacionID IS NULL THEN
'I' ELSE 'U' END,

    CONVERT(NVARCHAR(200),COALESCE(i.EstacionID,d.EstacionID)),

    CASE WHEN d.EstacionID IS NULL THEN NULL ELSE (SELECT d.* FOR JSON
PATH, WITHOUT_ARRAY_WRAPPER) END,

    CASE WHEN i.EstacionID IS NULL THEN NULL ELSE (SELECT i.* FOR JSON
PATH, WITHOUT_ARRAY_WRAPPER) END,

    CONVERT(VARCHAR(48),CONNECTIONPROPERTY('client_net_address'))

 FROM inserted i FULL JOIN deleted d ON d.EstacionID=i.EstacionID;

END;
```

```
GO

CREATE OR ALTER TRIGGER ops.tr_Bus_Auditoria ON ops.Bus AFTER INSERT, UPDATE, DELETE AS

BEGIN

 SET NOCOUNT ON;

 INSERT aud.Bitacora(UsuarioID,Esquema,Tabla,Operacion,ClaveRegistro,DatosAnteriores,DatosNuevos,DireccionIP)

 SELECT TRY_CONVERT(INT,SESSION_CONTEXT(N'UsuarioAplicacionID')),N'ops',N'Bus',

      CASE WHEN i.BusID IS NULL THEN 'D' WHEN d.BusID IS NULL THEN 'I' ELSE 'U' END,

      CONVERT(NVARCHAR(200),COALESCE(i.BusID,d.BusID)),

      CASE WHEN d.BusID IS NULL THEN NULL ELSE (SELECT d.* FOR JSON PATH, WITHOUT_ARRAY_WRAPPER) END,

      CASE WHEN i.BusID IS NULL THEN NULL ELSE (SELECT i.* FOR JSON PATH, WITHOUT_ARRAY_WRAPPER) END,

      CONVERT(VARCHAR(48),CONNECTIONPROPERTY('client_net_address'))

 FROM inserted i FULL JOIN deleted d ON d.BusID=i.BusID;

END;

GO

CREATE OR ALTER TRIGGER fin.tr_Tarjeta_Auditoria ON fin.TarjetaAcceso AFTER INSERT, UPDATE, DELETE AS

BEGIN

 SET NOCOUNT ON;
```

```
 INSERT
aud.Bitacora(UsuarioID,Esquema,Tabla,Operacion,ClaveRegistro,DatosAnteriores,DatosNue
vos,DireccionIP)

 SELECT
TRY_CONVERT(INT,SESSION_CONTEXT(N'UsuarioAplicacionID')),N'fin',N'TarjetaAcc
eso',

    CASE WHEN i.TarjetaID IS NULL THEN 'D' WHEN d.TarjetaID IS NULL THEN 'I'
ELSE 'U' END,

    CONVERT(NVARCHAR(200),COALESCE(i.TarjetaID,d.TarjetaID)),

    CASE WHEN d.TarjetaID IS NULL THEN NULL ELSE (SELECT d.* FOR JSON
PATH, WITHOUT_ARRAY_WRAPPER) END,

    CASE WHEN i.TarjetaID IS NULL THEN NULL ELSE (SELECT i.* FOR JSON
PATH, WITHOUT_ARRAY_WRAPPER) END,

    CONVERT(VARCHAR(48),CONNECTIONPROPERTY('client_net_address'))

 FROM inserted i FULL JOIN deleted d ON d.TarjetaID=i.TarjetaID;

END;

GO

/*=====================================================================

 9. PROCEDIMIENTOS TRANSACCIONALES

=====================================================================*/

CREATE OR ALTER PROCEDURE ops.sp_ActivarLinea

   @LineaID INT

AS

BEGIN

   SET NOCOUNT ON;

   DECLARE @Estaciones INT=(SELECT COUNT(*) FROM ops.LineaEstacion WHERE
LineaID=@LineaID),
```

```
    @Buses INT=(SELECT COUNT(*) FROM ops.LineaBus WHERE LineaID=@LineaID);
  IF NOT EXISTS (SELECT 1 FROM ops.Linea WHERE LineaID=@LineaID)
    THROW 51100, 'La linea indicada no existe.', 1;
  IF @Estaciones=0 OR @Buses<@Estaciones OR @Buses>2*@Estaciones
    THROW 51101, 'No se puede activar: la linea debe tener estaciones y entre una y dos veces esa cantidad de buses.', 1;
  IF EXISTS
  (
    SELECT 1 FROM ops.LineaEstacion le
    LEFT JOIN ops.TramoLinea t ON t.LineaID=le.LineaID AND t.EstacionOrigenID=le.EstacionID
    WHERE le.LineaID=@LineaID
      AND le.Orden < @Estaciones
      AND t.TramoID IS NULL
  )
    THROW 51102, 'No se puede activar: faltan distancias entre estaciones consecutivas.', 1;
  UPDATE ops.Linea SET Estado='A' WHERE LineaID=@LineaID;
END;
GO
CREATE OR ALTER PROCEDURE ops.sp_ActivarEstacion
  @EstacionID INT
AS
BEGIN
  SET NOCOUNT ON;
```

```
    IF NOT EXISTS (SELECT 1 FROM ops.Estacion WHERE EstacionID=@EstacionID)
        THROW 51103, 'La estacion indicada no existe.', 1;
    IF NOT EXISTS (SELECT 1 FROM ops.LineaEstacion WHERE EstacionID=@EstacionID)
        THROW 51104, 'La estacion debe pertenecer al menos a una linea.', 1;
    IF NOT EXISTS (SELECT 1 FROM ops.AsignacionOperadorEstacion WHERE EstacionID=@EstacionID AND FechaFin IS NULL)
        THROW 51105, 'La estacion debe tener un operador responsable.', 1;
    UPDATE ops.Estacion SET Estado='A' WHERE EstacionID=@EstacionID;
END;
GO
CREATE OR ALTER PROCEDURE ops.sp_ActivarAcceso
    @AccesoID INT
AS
BEGIN
    SET NOCOUNT ON;
    IF NOT EXISTS (SELECT 1 FROM ops.Acceso WHERE AccesoID=@AccesoID)
        THROW 51106, 'El acceso indicado no existe.', 1;
    IF NOT EXISTS (SELECT 1 FROM ops.AsignacionGuardiaAcceso WHERE AccesoID=@AccesoID AND FechaFin IS NULL)
        THROW 51107, 'El acceso debe tener por lo menos un guardia asignado.', 1;
    UPDATE ops.Acceso SET Estado='A' WHERE AccesoID=@AccesoID;
END;
GO
CREATE OR ALTER PROCEDURE ops.sp_AsignarBusLinea
```

```
    @BusID INT,
    @LineaID INT
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;
    BEGIN TRAN;
    IF NOT EXISTS (SELECT 1 FROM ops.Bus WITH (UPDLOCK,HOLDLOCK) WHERE BusID=@BusID)
        THROW 51110, 'El bus indicado no existe.', 1;
    IF EXISTS (SELECT 1 FROM ops.LineaBus WHERE BusID=@BusID)
        THROW 51111, 'El bus ya se encuentra asignado a una linea.', 1;
    IF NOT EXISTS (SELECT 1 FROM ops.Linea WHERE LineaID=@LineaID AND Estado<>'I')
        THROW 51112, 'La linea indicada no existe o esta inactiva.', 1;
    DECLARE @Estaciones INT=(SELECT COUNT(*) FROM ops.LineaEstacion WHERE LineaID=@LineaID),
            @Buses INT=(SELECT COUNT(*) FROM ops.LineaBus WITH (UPDLOCK,HOLDLOCK) WHERE LineaID=@LineaID);
    IF @Estaciones=0 OR @Buses+1 > 2*@Estaciones
        THROW 51113, 'La asignacion supera el maximo de dos buses por estacion o la linea no tiene estaciones.', 1;
    INSERT ops.LineaBus(BusID,LineaID) VALUES(@BusID,@LineaID);
    COMMIT;
END;
GO
```

```
CREATE OR ALTER PROCEDURE ops.sp_ReasignarParqueoBus
   @BusID INT,
   @ParqueoNuevoID INT,
   @Motivo NVARCHAR(300)=NULL
AS
BEGIN
   SET NOCOUNT ON;
   SET XACT_ABORT ON;
   BEGIN TRAN;
   IF NOT EXISTS (SELECT 1 FROM ops.Parqueo WHERE ParqueoID=@ParqueoNuevoID
AND Activo=1)
      THROW 51120, 'El parqueo nuevo no existe o esta inactivo.', 1;
   IF NOT EXISTS (SELECT 1 FROM ops.Bus WITH (UPDLOCK,HOLDLOCK) WHERE
BusID=@BusID)
      THROW 51121, 'El bus indicado no existe.', 1;
   UPDATE ops.Bus SET ParqueoID=@ParqueoNuevoID WHERE BusID=@BusID;
   IF @Motivo IS NOT NULL
      UPDATE ops.HistorialParqueoBus
      SET Motivo=@Motivo
      WHERE HistorialParqueoID=(SELECT MAX(HistorialParqueoID) FROM
ops.HistorialParqueoBus WHERE BusID=@BusID);
   COMMIT;
END;
GO
```

```
CREATE OR ALTER PROCEDURE ops.sp_RegistrarConteoEstacion
   @EstacionID INT,
   @CantidadPasajeros INT,
   @OperadorID INT=NULL,
   @CapacidadEvaluada SMALLINT=NULL
AS
BEGIN
   SET NOCOUNT ON;
   SET XACT_ABORT ON;
   IF @CantidadPasajeros < 0 THROW 51130, 'La cantidad de pasajeros no puede ser negativa.', 1;
   IF @CapacidadEvaluada IS NULL
      SELECT @CapacidadEvaluada=CapacidadReferencia FROM ops.Estacion WHERE EstacionID=@EstacionID;
   IF @CapacidadEvaluada IS NULL OR @CapacidadEvaluada<=0
      THROW 51131, 'La estacion no existe o no tiene capacidad valida.', 1;
   BEGIN TRAN;
   INSERT ops.ConteoEstacion(EstacionID,CantidadPasajeros,CapacidadEvaluada,OperadorID)
   VALUES(@EstacionID,@CantidadPasajeros,@CapacidadEvaluada,@OperadorID);
   DECLARE @ConteoID BIGINT=SCOPE_IDENTITY(),
           @Porcentaje DECIMAL(7,2)=ROUND(@CantidadPasajeros*100.0/@CapacidadEvaluada,2);
   IF @Porcentaje>=150
      INSERT ops.AlertaOcupacion(ConteoEstacionID,EstacionID,Porcentaje,RequiereUnidad)
```

```
      VALUES(@ConteoID,@EstacionID,@Porcentaje,1);
   COMMIT;
   SELECT @ConteoID AS ConteoEstacionID, @Porcentaje AS Porcentaje,
        CONVERT(BIT,CASE WHEN @Porcentaje>=150 THEN 1 ELSE 0 END) AS AlertaGenerada;
END;
GO
CREATE OR ALTER PROCEDURE ops.sp_RegistrarOcupacionBus
   @BusID INT,
   @EstacionID INT,
   @CantidadPasajeros SMALLINT,
   @ViajeID BIGINT=NULL
AS
BEGIN
   SET NOCOUNT ON;
   SET XACT_ABORT ON;
   DECLARE @Capacidad SMALLINT=(SELECT CapacidadMaxima FROM ops.Bus WHERE BusID=@BusID);
   IF @Capacidad IS NULL THROW 51140, 'El bus indicado no existe.', 1;
   IF @CantidadPasajeros<0 OR @CantidadPasajeros>@Capacidad
      THROW 51141, 'La ocupacion debe estar entre cero y la capacidad maxima del bus.', 1;
   DECLARE @Porcentaje DECIMAL(7,2)=ROUND(@CantidadPasajeros*100.0/@Capacidad,2);
   BEGIN TRAN;
```

```
    INSERT
ops.RegistroOcupacionBus(BusID,EstacionID,ViajeID,CantidadPasajeros,CapacidadEvaluad
a,PorcentajeOcupacion)

VALUES(@BusID,@EstacionID,@ViajeID,@CantidadPasajeros,@Capacidad,@Porcentaje)
;
    DECLARE @RegistroID BIGINT=SCOPE_IDENTITY();
    IF @Porcentaje<25
       INSERT                ops.EsperaAdicional(RegistroOcupacionID,MinutosAdicionales)
VALUES(@RegistroID,5);
    COMMIT;
    SELECT @RegistroID RegistroOcupacionID,@Porcentaje PorcentajeOcupacion,
        CONVERT(BIT,CASE WHEN @Porcentaje<25 THEN 1 ELSE 0 END) DebeEsperar,
        CASE WHEN @Porcentaje<25 THEN 5 ELSE 0 END MinutosAdicionales;
END;
GO
CREATE OR ALTER PROCEDURE fin.sp_RegistrarMovimientoTarjeta
    @TarjetaID BIGINT,
    @Tipo CHAR(1),
    @Monto DECIMAL(12,2),
    @Referencia VARCHAR(80),
    @EstacionID INT=NULL,
    @UsuarioID INT=NULL
AS
BEGIN
    SET NOCOUNT ON;
```

```
SET XACT_ABORT ON;
IF @Tipo NOT IN ('R','P') THROW 51150, 'Tipo invalido: R=recarga, P=pago.', 1;
IF @Monto<=0 THROW 51151, 'El monto debe ser mayor que cero.', 1;

BEGIN TRY
    BEGIN TRAN;
    DECLARE @SaldoAnterior DECIMAL(12,2), @Estado VARCHAR(20), @Vence DATE;
    SELECT @SaldoAnterior=Saldo,@Estado=Estado,@Vence=FechaVencimiento
    FROM fin.TarjetaAcceso WITH (UPDLOCK,HOLDLOCK)
    WHERE TarjetaID=@TarjetaID;
    IF @SaldoAnterior IS NULL THROW 51152, 'La tarjeta indicada no existe.', 1;
    IF @Estado<>'ACTIVA' OR (@Vence IS NOT NULL AND @Vence<CAST(SYSUTCDATETIME() AS DATE))
        THROW 51153, 'La tarjeta no esta activa o se encuentra vencida.', 1;
    IF @Tipo='P' AND @SaldoAnterior<@Monto THROW 51154, 'Saldo insuficiente para realizar el pago.', 1;
    IF EXISTS (SELECT 1 FROM fin.TransaccionTarjeta WHERE Referencia=@Referencia)
        THROW 51155, 'La referencia de la transaccion ya fue utilizada.', 1;
    DECLARE @SaldoNuevo DECIMAL(12,2)=CASE WHEN @Tipo='R' THEN @SaldoAnterior+@Monto ELSE @SaldoAnterior-@Monto END;
    EXEC sys.sp_set_session_context @key=N'MovimientoTarjetaAutorizado',@value=1;
    UPDATE fin.TarjetaAcceso SET Saldo=@SaldoNuevo WHERE TarjetaID=@TarjetaID;
```

```
        INSERT
fin.TransaccionTarjeta(TarjetaID,EstacionID,Tipo,Monto,SaldoAnterior,SaldoNuevo,Refere
ncia,UsuarioID)
VALUES(@TarjetaID,@EstacionID,@Tipo,@Monto,@SaldoAnterior,@SaldoNuevo,@Ref
erencia,@UsuarioID);
        DECLARE @TransaccionID BIGINT=SCOPE_IDENTITY();
        EXEC                                    sys.sp_set_session_context
@key=N'MovimientoTarjetaAutorizado',@value=NULL;
        COMMIT;
        SELECT @TransaccionID TransaccionID,@SaldoNuevo SaldoActual;
    END TRY
    BEGIN CATCH
        EXEC                                    sys.sp_set_session_context
@key=N'MovimientoTarjetaAutorizado',@value=NULL;
        IF XACT_STATE()<>0 ROLLBACK;
        THROW;
    END CATCH
END;
GO
CREATE OR ALTER PROCEDURE ops.sp_ValidarIntegridadOperativa
AS
BEGIN
    SET NOCOUNT ON;
    SELECT 'LINEA_SIN_ESTACIONES' Problema,l.LineaID Referencia,l.Codigo Detalle
    FROM ops.Linea l WHERE l.Estado='A' AND NOT EXISTS(SELECT 1 FROM
ops.LineaEstacion le WHERE le.LineaID=l.LineaID)
```

```
    UNION ALL
    SELECT 'ESTACION_SIN_LINEA',e.EstacionID,e.Codigo FROM ops.Estacion e
    WHERE e.Estado='A' AND NOT EXISTS(SELECT 1 FROM ops.LineaEstacion le
WHERE le.EstacionID=e.EstacionID)
    UNION ALL
    SELECT 'ACCESO_SIN_GUARDIA',a.AccesoID,a.Codigo FROM ops.Acceso a
    WHERE a.Estado='A' AND NOT EXISTS(SELECT 1 FROM
ops.AsignacionGuardiaAcceso x WHERE x.AccesoID=a.AccesoID AND x.FechaFin IS
NULL)
    UNION ALL
    SELECT 'ESTACION_SIN_OPERADOR',e.EstacionID,e.Codigo FROM ops.Estacion e
    WHERE e.Estado='A' AND NOT EXISTS(SELECT 1 FROM
ops.AsignacionOperadorEstacion x WHERE x.EstacionID=e.EstacionID AND x.FechaFin IS
NULL)
    UNION ALL
    SELECT 'LINEA_FUERA_LIMITE_BUSES',l.LineaID,l.Codigo FROM ops.Linea l
    CROSS APPLY(SELECT COUNT(*) n FROM ops.LineaEstacion WHERE
LineaID=l.LineaID) e
    CROSS APPLY(SELECT COUNT(*) n FROM ops.LineaBus WHERE
LineaID=l.LineaID) b
    WHERE l.Estado='A' AND (b.n<e.n OR b.n>2*e.n);
END;
GO
/*=====================================================================
  10. VISTAS Y CONSULTAS PARA REPORTES REQUERIDOS
=====================================================================*/
```

```
CREATE OR ALTER VIEW rep.vw_Estaciones
AS
SELECT e.EstacionID,e.Codigo,e.Nombre,e.Direccion,e.Latitud,e.Longitud,
      e.CapacidadReferencia,e.Estado,m.Nombre Municipalidad,
      COUNT(DISTINCT le.LineaID) CantidadLineas,
      COUNT(DISTINCT a.AccesoID) CantidadAccesos
FROM ops.Estacion e
JOIN cat.Municipalidad m ON m.MunicipalidadID=e.MunicipalidadID
LEFT JOIN ops.LineaEstacion le ON le.EstacionID=e.EstacionID
LEFT JOIN ops.Acceso a ON a.EstacionID=e.EstacionID
GROUP BY e.EstacionID,e.Codigo,e.Nombre,e.Direccion,e.Latitud,e.Longitud,
        e.CapacidadReferencia,e.Estado,m.Nombre;
GO
CREATE OR ALTER VIEW rep.vw_Lineas
AS
SELECT l.LineaID,l.Codigo,l.Nombre,l.Estado,m.Nombre Municipalidad,
      e.CantidadEstaciones,b.CantidadBuses,t.DistanciaTotalKm
FROM ops.Linea l
JOIN cat.Municipalidad m ON m.MunicipalidadID=l.MunicipalidadID
OUTER APPLY
(
   SELECT COUNT(*) CantidadEstaciones
   FROM ops.LineaEstacion le WHERE le.LineaID=l.LineaID
) e
```

```
OUTER APPLY
(
    SELECT COUNT(*) CantidadBuses
    FROM ops.LineaBus lb WHERE lb.LineaID=l.LineaID
) b
OUTER APPLY
(
    SELECT COALESCE(SUM(tl.DistanciaKm),0) DistanciaTotalKm
    FROM ops.TramoLinea tl WHERE tl.LineaID=l.LineaID
) t;
GO

CREATE OR ALTER VIEW rep.vw_BusesPorLinea
AS
SELECT l.LineaID,l.Codigo CodigoLinea,l.Nombre Linea,
       b.BusID,b.CodigoInterno,b.Placa,b.Marca,b.Modelo,b.CapacidadMaxima,b.Estado,
       p.Codigo CodigoParqueo,p.Nombre Parqueo,e.Nombre EstacionParqueo
FROM ops.Linea l
JOIN ops.LineaBus lb ON lb.LineaID=l.LineaID
JOIN ops.Bus b ON b.BusID=lb.BusID
JOIN ops.Parqueo p ON p.ParqueoID=b.ParqueoID
JOIN ops.Estacion e ON e.EstacionID=p.EstacionID;
GO
CREATE OR ALTER VIEW rep.vw_EstacionesPorLinea
```

```
AS
SELECT l.LineaID,l.Codigo CodigoLinea,l.Nombre Linea,le.Orden,
    e.EstacionID,e.Codigo CodigoEstacion,e.Nombre Estacion,
    t.DistanciaKm DistanciaSiguienteKm
FROM ops.Linea l
JOIN ops.LineaEstacion le ON le.LineaID=l.LineaID
JOIN ops.Estacion e ON e.EstacionID=le.EstacionID
LEFT JOIN ops.TramoLinea t ON t.LineaID=le.LineaID AND
t.EstacionOrigenID=le.EstacionID;
GO
CREATE OR ALTER VIEW rep.vw_AccesosPorLinea
AS
SELECT l.LineaID,l.Codigo CodigoLinea,l.Nombre Linea,le.Orden,
    e.EstacionID,e.Nombre Estacion,a.AccesoID,a.Codigo CodigoAcceso,a.Nombre
Acceso,a.TipoAcceso,a.Estado
FROM ops.Linea l
JOIN ops.LineaEstacion le ON le.LineaID=l.LineaID
JOIN ops.Estacion e ON e.EstacionID=le.EstacionID
JOIN ops.Acceso a ON a.EstacionID=e.EstacionID;
GO
CREATE OR ALTER VIEW rep.vw_AlertasPendientes
AS
SELECT a.AlertaID,a.FechaGeneracion,e.Codigo CodigoEstacion,e.Nombre Estacion,
    c.CantidadPasajeros,c.CapacidadEvaluada,a.Porcentaje,a.RequiereUnidad,a.Estado
FROM ops.AlertaOcupacion a
```

```
JOIN ops.ConteoEstacion c ON c.ConteoEstacionID=a.ConteoEstacionID
JOIN ops.Estacion e ON e.EstacionID=a.EstacionID
WHERE a.Estado='PENDIENTE';
GO
/*=============================================================
  11. DATOS INICIALES DE SEGURIDAD
=============================================================*/
INSERT seg.Rol(Nombre,Descripcion)
SELECT v.Nombre,v.Descripcion
FROM (VALUES
 (N'Administrador',N'Control total de configuracion, usuarios y datos.'),
 (N'Supervisor',N'Consulta operativa, alertas y reportes.'),
 (N'Operador de estacion',N'Gestion de funciones de su estacion asignada.'),
 (N'Administrativo',N'Gestion autorizada de flota y personal.'),
 (N'Auditor',N'Consulta de reportes y bitacora sin modificacion.')
)v(Nombre,Descripcion)
WHERE NOT EXISTS(SELECT 1 FROM seg.Rol r WHERE r.Nombre=v.Nombre);
GO
INSERT seg.Permiso(Codigo,Descripcion)
SELECT v.Codigo,v.Descripcion
FROM (VALUES
 ('LINEAS_VER',N'Consultar lineas y recorridos'),('LINEAS_EDITAR',N'Administrar lineas'),
 ('ESTACIONES_VER',N'Consultar estaciones'),('ESTACIONES_EDITAR',N'Administrar estaciones y accesos'),
```

```
('FLOTA_VER',N'Consultar buses y parqueos'),('FLOTA_EDITAR',N'Administrar buses y asignaciones'),
('PERSONAL_VER',N'Consultar personal'),('PERSONAL_EDITAR',N'Administrar personal'),
('PAGOS_OPERAR',N'Realizar recargas y pagos'),('ALERTAS_GESTIONAR',N'Atender alertas operativas'),
('REPORTES_VER',N'Consultar reportes'),('USUARIOS_EDITAR',N'Administrar usuarios y roles'),
('AUDITORIA_VER',N'Consultar historial de operaciones')
)v(Codigo,Descripcion)
WHERE NOT EXISTS(SELECT 1 FROM seg.Permiso p WHERE p.Codigo=v.Codigo);
GO
INSERT seg.RolPermiso(RolID,PermisoID)
SELECT r.RolID,p.PermisoID FROM seg.Rol r CROSS JOIN seg.Permiso p
WHERE r.Nombre=N'Administrador'
  AND NOT EXISTS(SELECT 1 FROM seg.RolPermiso rp WHERE rp.RolID=r.RolID AND rp.PermisoID=p.PermisoID);
GO
INSERT seg.RolPermiso(RolID,PermisoID)
SELECT r.RolID,p.PermisoID
FROM seg.Rol r
JOIN seg.Permiso p ON
    (r.Nombre=N'Supervisor' AND p.Codigo IN ('LINEAS_VER','ESTACIONES_VER','FLOTA_VER','PERSONAL_VER','ALERTAS_GESTIONAR','REPORTES_VER'))
```

```
   OR (r.Nombre=N'Operador de estacion' AND p.Codigo IN
('LINEAS_VER','ESTACIONES_VER','FLOTA_VER','PAGOS_OPERAR','ALERTAS_GE
STIONAR','REPORTES_VER'))

   OR (r.Nombre=N'Administrativo' AND p.Codigo IN
('LINEAS_VER','ESTACIONES_VER','FLOTA_VER','FLOTA_EDITAR','PERSONAL_V
ER','PERSONAL_EDITAR','REPORTES_VER'))

   OR (r.Nombre=N'Auditor' AND p.Codigo IN
('LINEAS_VER','ESTACIONES_VER','FLOTA_VER','REPORTES_VER','AUDITORIA_
VER'))

WHERE NOT EXISTS

    (SELECT 1 FROM seg.RolPermiso rp WHERE rp.RolID=r.RolID AND
rp.PermisoID=p.PermisoID);

GO

/*=====================================================================

  12. VERIFICACION FINAL Y RESPALDO

=====================================================================*/

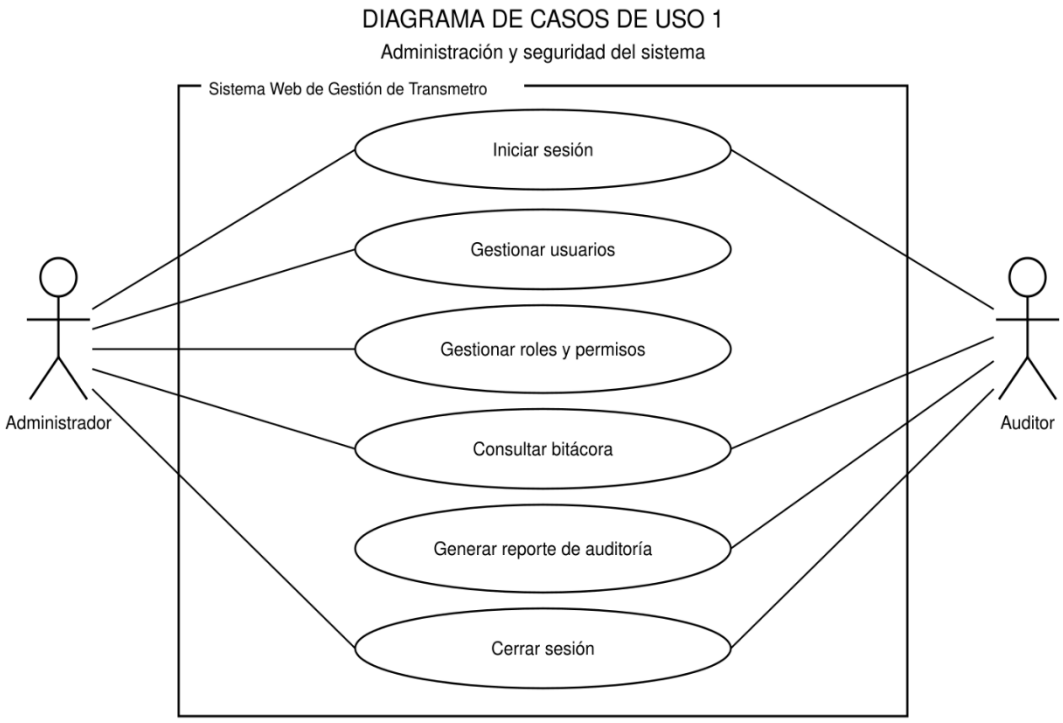
PRINT N'Base de datos TransmetroDB creada correctamente.';

PRINT N'Ejecute: EXEC ops.sp_ValidarIntegridadOperativa;';

GO
```

11. DIAGRAMAS UML

11.1 Caso de Uso



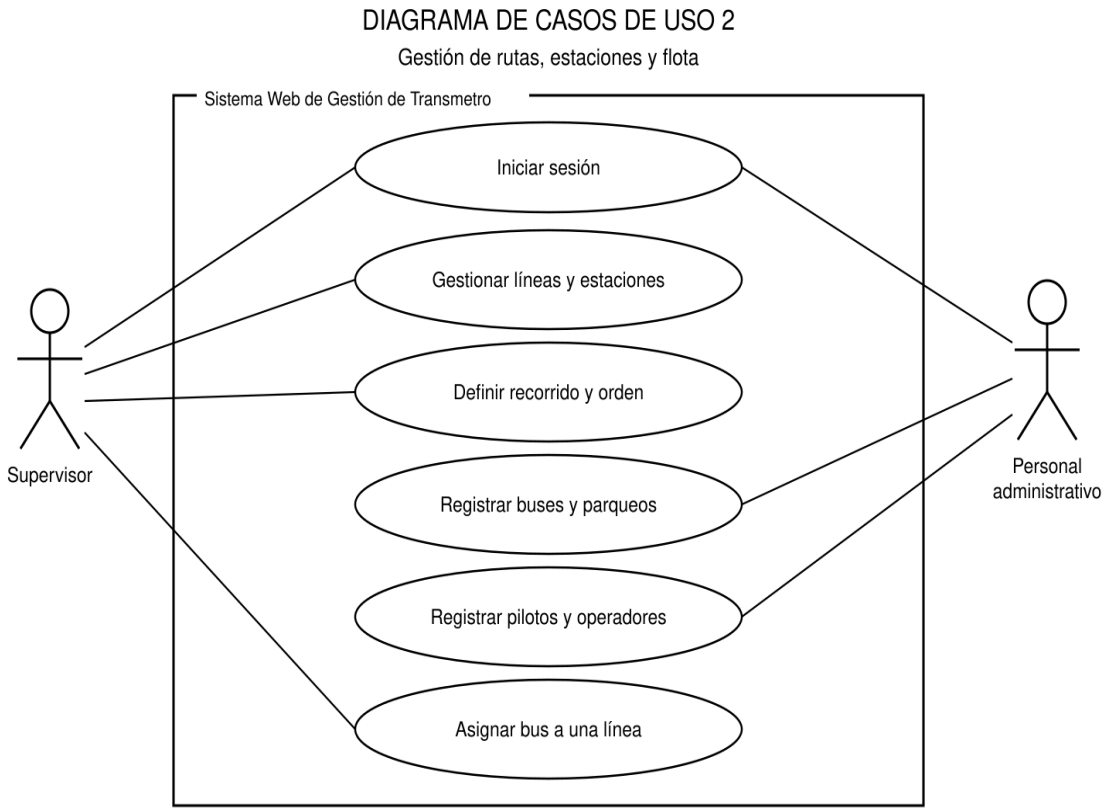


DIAGRAMA DE CASOS DE USO 3

Control de ocupación en estaciones y buses

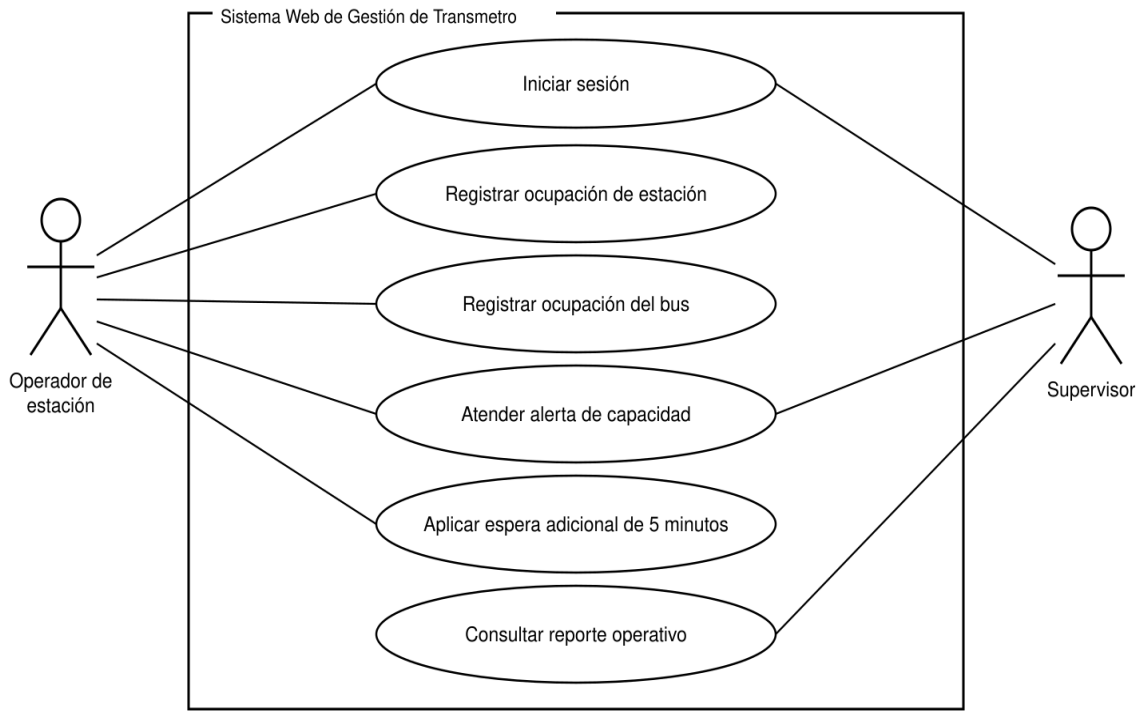
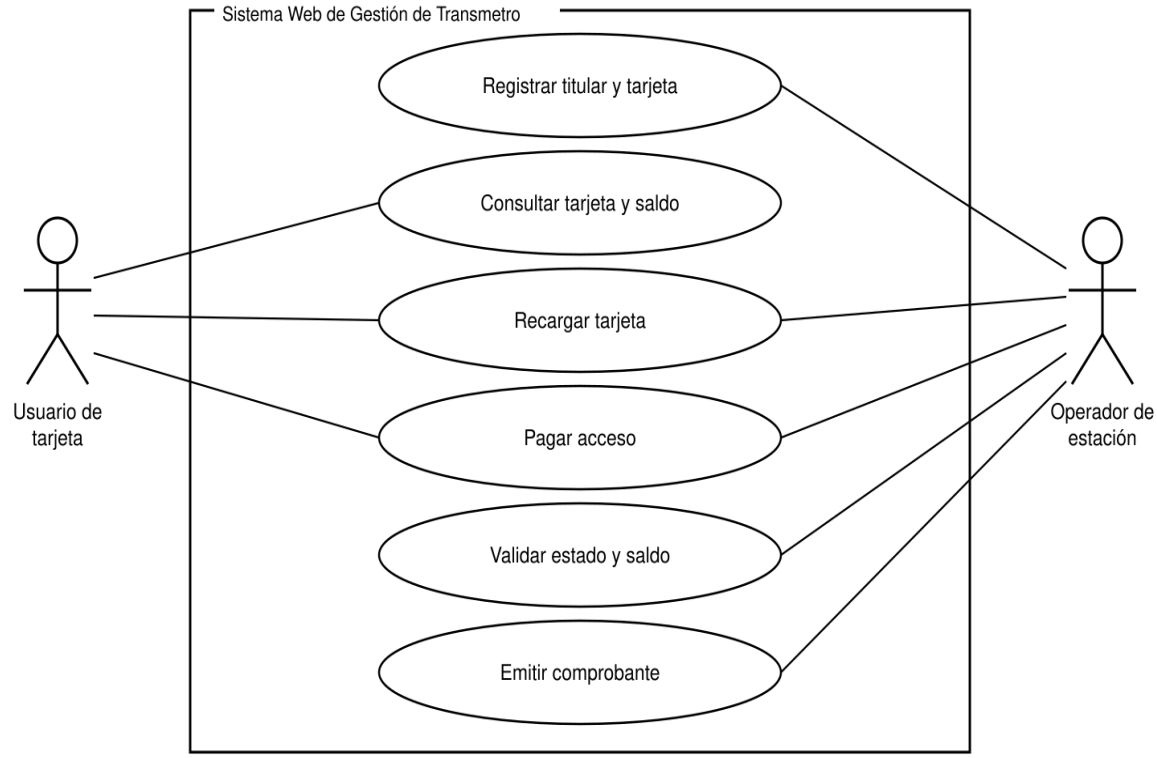
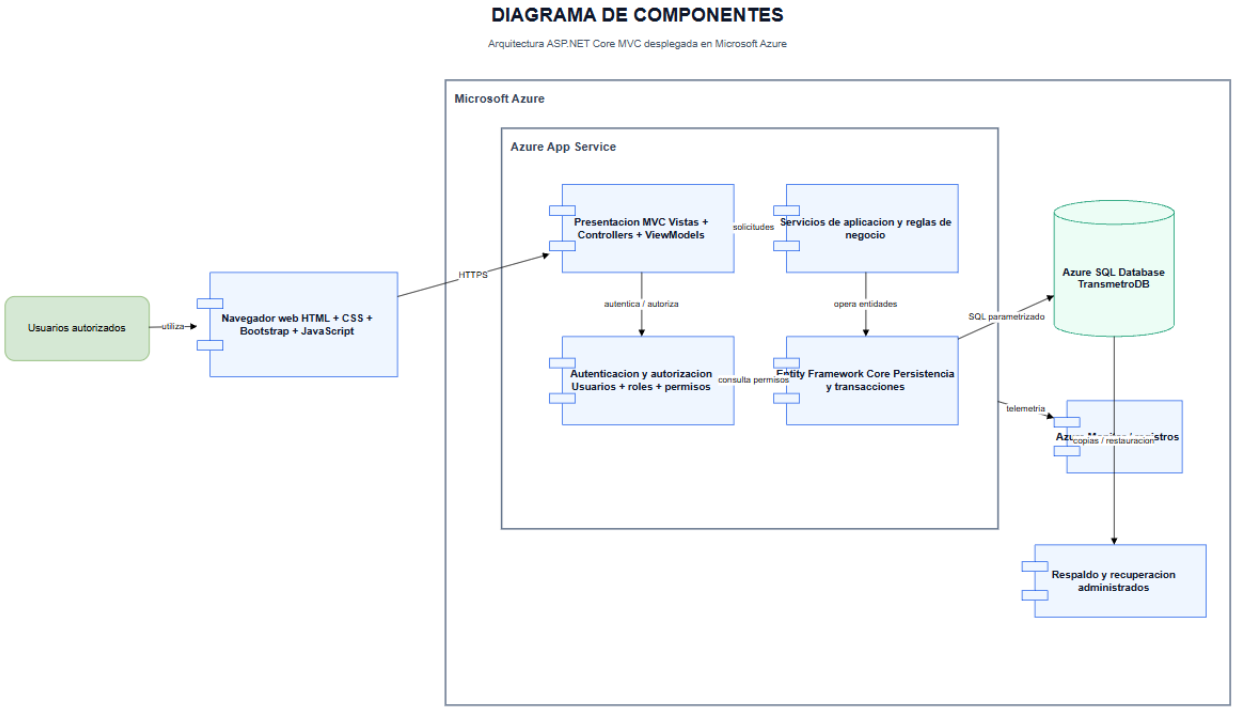


DIAGRAMA DE CASOS DE USO 4

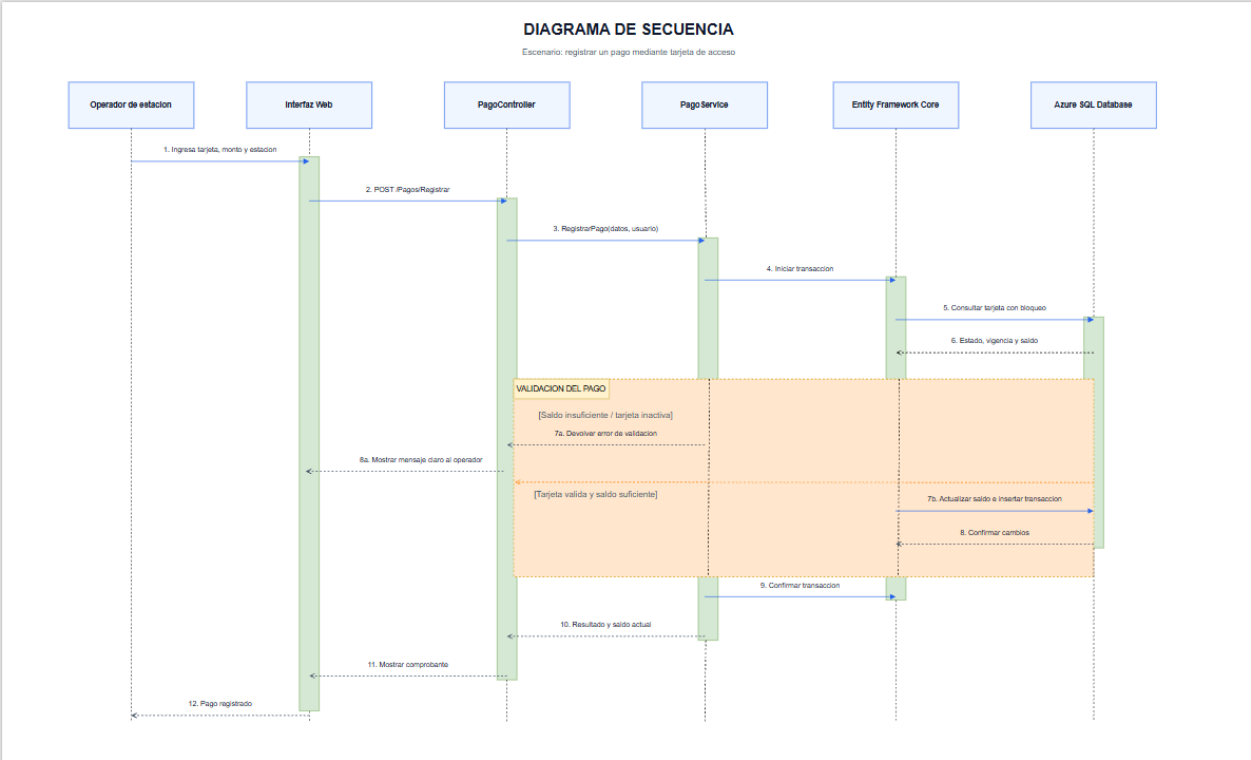
Registro de tarjetas, recargas y pagos



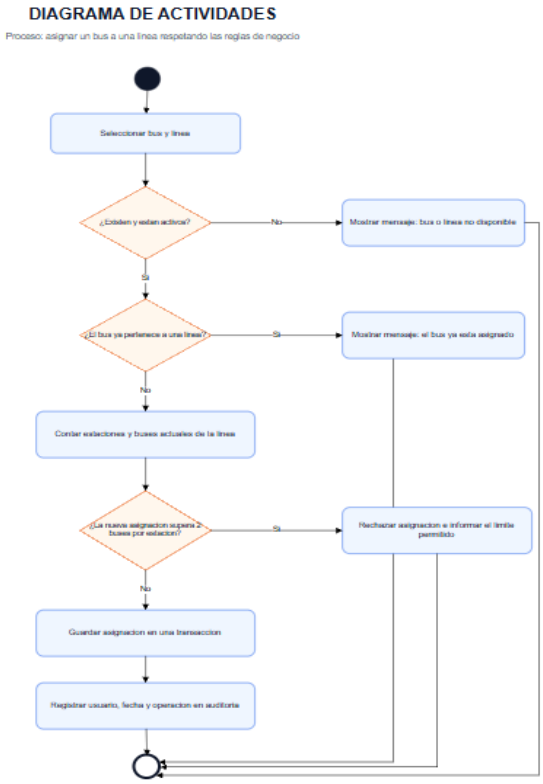
11.2 Diagrama de Componentes



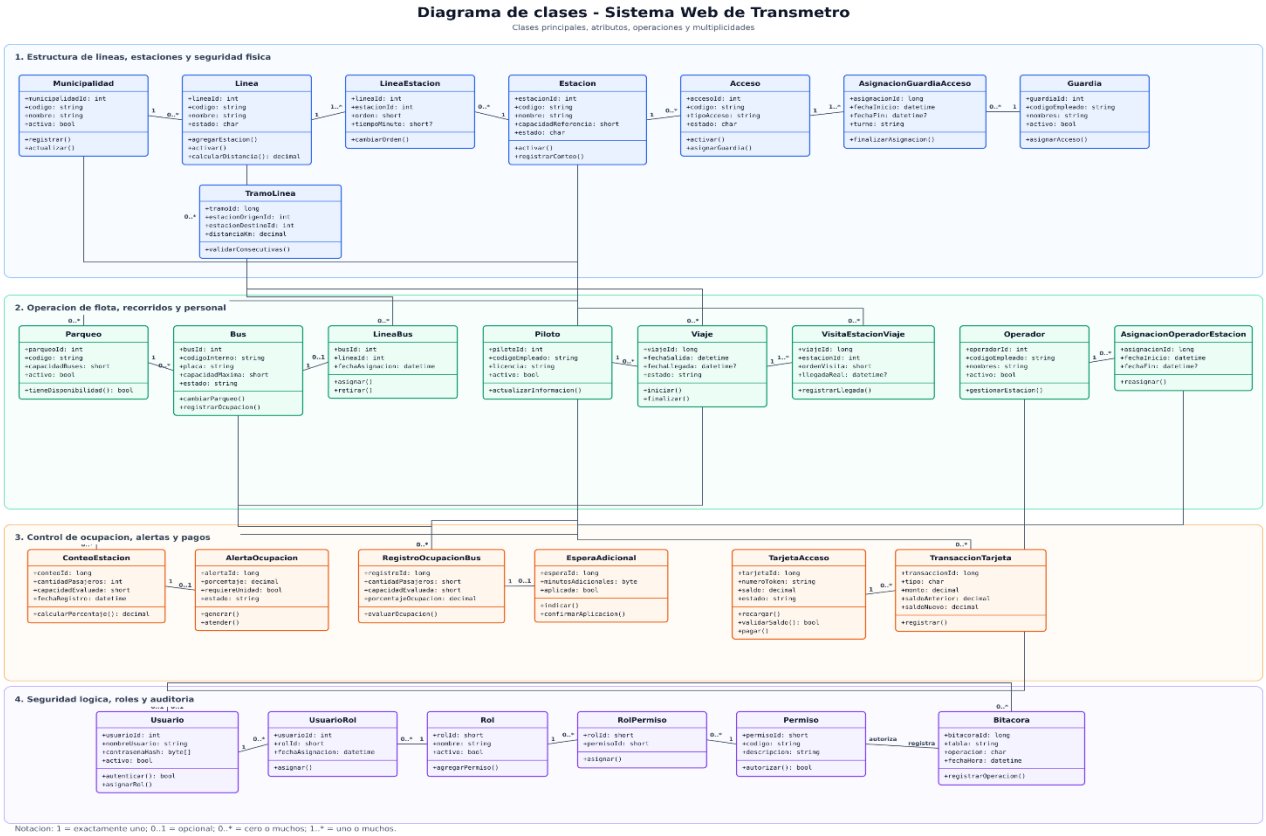
11.3 Diagrama de Secuencia



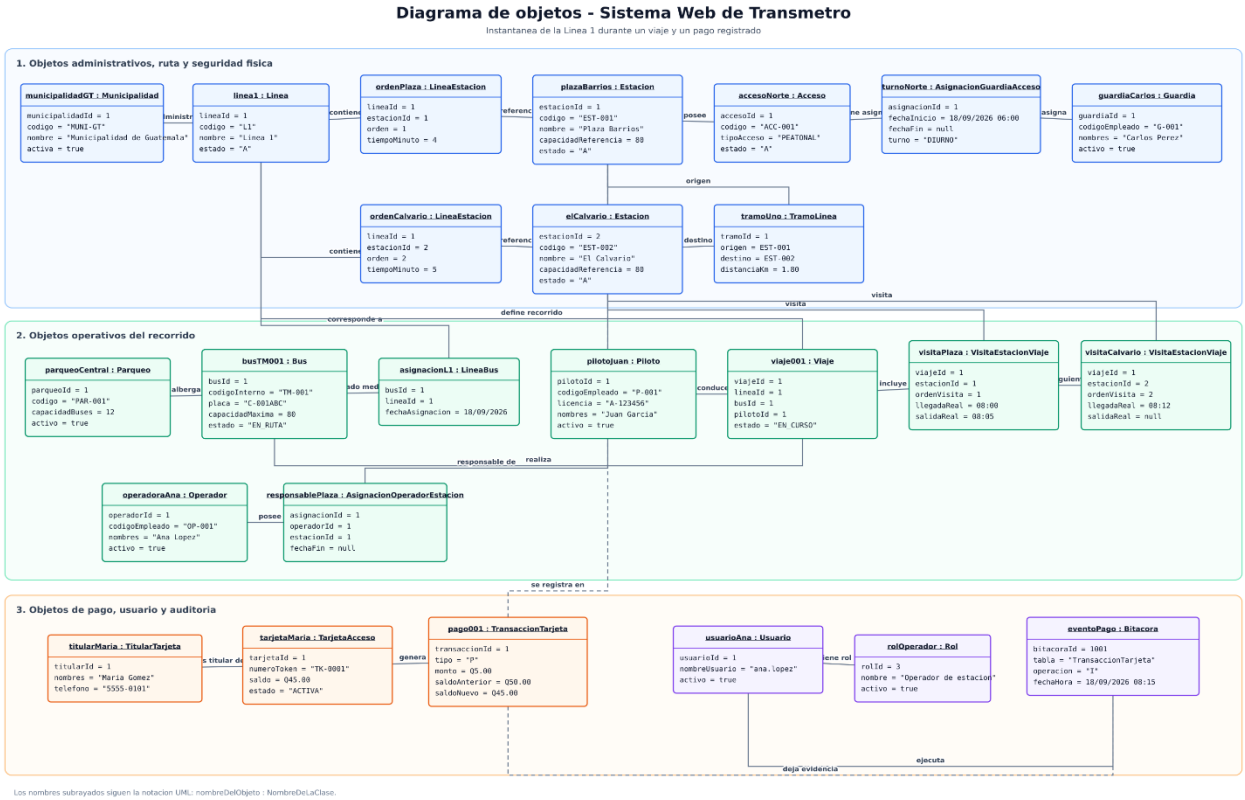
11.4 Diagrama de Actividades



11.5 Diagrama de Clases



11.6 Diagrama de Objetos



12. Presentación de Stack Tecnológico

Para desarrollar el Sistema Web de Control y Gestión de Transmetro se seleccionó un conjunto de tecnologías que permite construir una aplicación segura, organizada, escalable y fácil de mantener. El stack tecnológico incluye la base de datos, la metodología de trabajo, el lenguaje de programación, el framework y el entorno de desarrollo integrado.

Base de datos: SQL Server 2025

Como sistema gestor de base de datos se utilizará Microsoft SQL Server 2025. Esta herramienta permitirá almacenar y administrar toda la información relacionada con municipalidades, líneas, estaciones, buses, pilotos, operadores, accesos, tarjetas, pagos, usuarios, roles y permisos.

La base de datos del proyecto se denomina TransmetroGestionDB y está compuesta por 31 tablas relacionadas mediante claves primarias y claves foráneas. También utiliza seis esquemas para separar la información de acuerdo con su función:

- cat: catálogos generales.
- ops: operaciones de Transmetro.

- seg: seguridad, usuarios y permisos.
- fin: tarjetas y transacciones financieras.
- aud: auditoría y bitácora.
- rep: consultas y reportes.

SQL Server fue seleccionado porque ofrece integridad de datos, transacciones, procedimientos almacenados, vistas, restricciones, respaldos y mecanismos de seguridad. Además, posee una integración adecuada con C#, Entity Framework Core y Microsoft Azure.

Metodología: Scrum

Para organizar el desarrollo se utilizará la metodología ágil Scrum. Esta metodología divide el proyecto en periodos cortos llamados sprints, durante los cuales se desarrolla y valida una parte funcional del sistema.

El Product Backlog contendrá todos los requerimientos del proyecto, como la gestión de usuarios, estaciones, líneas, buses, personal, ocupación, alertas, tarjetas y pagos. Al inicio de cada sprint se seleccionarán las actividades que serán desarrolladas y, al finalizar, se presentará un incremento funcional.

Los principales roles de Scrum son:

- Product Owner: representa las necesidades del sistema y prioriza los requerimientos.
- Scrum Master: facilita el trabajo del equipo y ayuda a resolver impedimentos.
- Equipo de desarrollo: diseña, programa, prueba y documenta el sistema.

Scrum permitirá revisar constantemente el avance, detectar problemas a tiempo y realizar mejoras de acuerdo con los resultados obtenidos.

Lenguaje de programación: C#

El sistema será desarrollado principalmente con el lenguaje C#. Es un lenguaje moderno, orientado a objetos y fuertemente tipado, utilizado ampliamente para crear aplicaciones empresariales con tecnologías de Microsoft.

C# permitirá representar las entidades del proyecto mediante clases como Bus, Linea, Estacion, Usuario, TarjetaAcceso y TransaccionTarjeta. También facilitará la implementación de validaciones, reglas de negocio, control de errores, autenticación y acceso a la base de datos.

Este lenguaje fue seleccionado por su seguridad, rendimiento, facilidad de mantenimiento y compatibilidad con ASP.NET Core, Entity Framework Core, SQL Server y Azure.

Framework: ASP.NET Core MVC

El framework principal será ASP.NET Core MVC. Este permite desarrollar aplicaciones web utilizando el patrón Modelo-Vista-Controlador.

El sistema se dividirá de la siguiente manera:

- Modelo: representa los datos y las reglas del negocio.
- Vista: presenta la información al usuario mediante páginas web.
- Controlador: recibe las solicitudes, ejecuta las operaciones necesarias y devuelve una respuesta.

Esta separación ayuda a mantener el código organizado y facilita las pruebas y el mantenimiento. Por ejemplo, cuando un operador registra la ocupación de una estación, la vista recibe los datos, el controlador procesa la solicitud y el modelo aplica las reglas antes de guardar el registro.

También se utilizará Entity Framework Core como herramienta de persistencia. Su función será comunicar la aplicación con SQL Server mediante clases y objetos de C#, reduciendo la necesidad de escribir consultas SQL manualmente para las operaciones comunes.

IDE: Visual Studio 2022 Community

El entorno de desarrollo será Visual Studio 2022 Community. En esta herramienta se crearán los modelos, vistas, controladores, servicios y configuraciones del proyecto.

Visual Studio proporciona funciones como:

- Editor de código.
- Autocompletado e IntelliSense.
- Detección de errores.
- Depuración paso a paso.
- Administración de paquetes NuGet.
- Ejecución de pruebas.
- Integración con SQL Server, Git y Azure.

La edición Community es gratuita para fines académicos y contiene las herramientas necesarias para desarrollar completamente el proyecto.

Plataforma de despliegue: Microsoft Azure

Microsoft Azure podrá utilizarse para publicar la aplicación y permitir el acceso mediante Internet. Azure App Service alojará la aplicación web, mientras que Azure SQL Database podrá almacenar la base de datos en la nube.

También se pueden utilizar los servicios de monitoreo, registros, copias de seguridad y recuperación. Esto permite que el sistema tenga mayor disponibilidad, seguridad y capacidad de crecimiento.

Integración del stack tecnológico

El usuario accederá al sistema desde un navegador web. La solicitud será recibida por la aplicación desarrollada con ASP.NET Core MVC y C#. Los controladores utilizarán los servicios del sistema para aplicar las reglas de negocio. Posteriormente, Entity Framework Core realizará las consultas o modificaciones en SQL Server.

El desarrollo se realizará en Visual Studio 2022 y las actividades se organizarán mediante Scrum. Finalmente, la aplicación podrá desplegarse en Microsoft Azure.

13. MÉTRICAS DE SOFTWARE

En el Sistema Web de Control y Gestión de Transmetro se utilizarán métricas de producto, proceso, calidad, rendimiento, seguridad y base de datos. Estas métricas serán evaluadas durante los sprints y en las pruebas finales. Por lo que estas métricas permiten conocer el avance del proyecto, detectar problemas, comprobar el cumplimiento de los requerimientos y tomar decisiones basadas en resultados.

Objetivo general

Definir indicadores que permitan medir el avance, calidad, rendimiento, seguridad y confiabilidad del Sistema Web de Control y Gestión de Transmetro durante su desarrollo y operación.

Objetivos específicos

- Medir el cumplimiento de los requerimientos funcionales.
- Verificar la calidad del código desarrollado.

- Controlar los defectos encontrados.
- Evaluar el rendimiento del sistema.
- Comprobar la seguridad y disponibilidad de la aplicación.
- Evaluar el avance de cada sprint.
- Verificar la integridad de la base de datos.
- Facilitar la mejora continua del proyecto.

Métricas propuestas

I. Cumplimiento de requerimientos: Esta métrica permite conocer qué porcentaje de los requerimientos planificados ya fue desarrollado y aprobado.

Fórmula:

Cumplimiento = (requerimientos completados / requerimientos planificados) × 100

Por ejemplo, si se planificaron 20 requerimientos y se completaron 18:

Cumplimiento = (18 / 20) × 100 = 90 %

La meta será mantener un cumplimiento igual o superior al 90 % antes de la entrega final.

II. Cumplimiento del sprint: Permite evaluar si el equipo completa las tareas que fueron seleccionadas al inicio de cada sprint.

Fórmula:

Cumplimiento del sprint = (tareas completadas / tareas comprometidas) × 100

Si se planificaron 12 tareas y se terminaron 10:

Cumplimiento = (10 / 12) × 100 = 83.33 %

El resultado ayuda a determinar si la cantidad de trabajo planificada fue adecuada.

III. Cobertura de pruebas: Esta métrica indica qué porcentaje de las funciones desarrolladas ha sido probado. Las pruebas deben incluir funciones como inicio de sesión, gestión de usuarios, registro de buses, asignación de líneas, ocupación, alertas, recargas y pagos.

Fórmula:

Cobertura = (elementos probados / elementos desarrollados) × 100

La meta propuesta es alcanzar como mínimo un 80 % de cobertura.

IV. Tasa de éxito de pruebas: Mide la cantidad de pruebas que finalizan correctamente.

Fórmula:

Tasa de éxito = (pruebas aprobadas / pruebas ejecutadas) × 100

Por ejemplo, si se ejecutan 50 pruebas y 46 son aprobadas:

Tasa de éxito = (46 / 50) × 100 = 92 %

Esto indicaría que el sistema supera la meta propuesta del 90 %.

V. Densidad de defectos: Permite relacionar la cantidad de defectos encontrados con el tamaño del código.

Fórmula:

Densidad de defectos = defectos encontrados / miles de líneas de código

Si el proyecto contiene 10,000 líneas de código y se encuentran ocho defectos:

Densidad = 8 / 10 = 0.8 defectos por KLOC

Un valor bajo representa una mejor calidad del código.

VI. Tiempo promedio de resolución: Mide cuánto tarda el equipo en corregir los errores reportados.

Fórmula:

Tiempo promedio = total de horas utilizadas / errores corregidos

Si se utilizaron 20 horas para corregir cinco errores:

Tiempo promedio = 20 / 5 = 4 horas

Esta métrica permite identificar errores que requieren demasiado tiempo y mejorar el proceso de mantenimiento.

VII. Tiempo de respuesta: Mide cuánto tarda el sistema en responder después de que el usuario realiza una acción. Se evaluarán operaciones como iniciar sesión, consultar estaciones, registrar ocupación, buscar una tarjeta y procesar un pago. La meta será que las operaciones normales respondan en un máximo de tres segundos. Las consultas que generan reportes extensos podrán requerir un tiempo mayor, pero deberán mostrar al usuario que la solicitud está siendo procesada.

VIII. Disponibilidad: Representa el porcentaje de tiempo durante el cual el sistema permanece disponible.

Fórmula:

Disponibilidad = (tiempo disponible / tiempo total) × 100

La meta propuesta es alcanzar una disponibilidad mínima del 99 % durante el periodo de operación.

IX. Éxito de tareas del usuario: Permite evaluar la facilidad de uso del sistema. Se solicitará a los usuarios que realicen tareas comunes y se registrará cuántas completan correctamente.

Fórmula:

Éxito de tareas = (tareas completadas / intentos realizados) × 100

La meta será que al menos el 90 % de las tareas pueda completarse sin ayuda adicional.

X. Vulnerabilidades críticas: Se contabilizarán los problemas de seguridad encontrados durante las revisiones y pruebas. Antes de publicar el sistema no deberá existir ninguna vulnerabilidad crítica pendiente.

Se evaluarán principalmente:

- Accesos sin autorización.
- Contraseñas almacenadas de forma insegura.
- Inyección SQL.
- Pérdida de sesión.
- Permisos incorrectos.
- Datos sensibles expuestos.
- Validaciones insuficientes.

La meta es mantener en cero la cantidad de vulnerabilidades críticas abiertas.

XI. Integridad referencial: Esta métrica verificará que no existan registros relacionados con información inexistente. Por ejemplo, no debe existir un viaje asociado con un bus que no esté registrado o una transacción relacionada con una tarjeta inexistente.

La meta será mantener cero registros inválidos. Para cumplirla se utilizarán claves primarias, claves foráneas, restricciones y transacciones en SQL Server.

XII. Respaldos exitosos: Permite comprobar que las copias de seguridad se ejecuten correctamente.

Fórmula:

Respaldos exitosos = (respaldos completados / respaldos programados) × 100

La meta será completar correctamente el 100 % de los respaldos programados y realizar pruebas periódicas de restauración.

Herramientas de medición

Las métricas podrán obtenerse con las siguientes herramientas:

- Visual Studio 2022 para pruebas, depuración y análisis del código.
- Herramientas de prueba de ASP.NET Core para verificar los componentes.
- SQL Server Management Studio para revisar datos, consultas e integridad.
- Azure Monitor o Application Insights para rendimiento y disponibilidad.
- Tablero Scrum para medir tareas y cumplimiento de los sprints.
- Bitácora del sistema para consultar operaciones, errores y accesos.
- Encuestas o pruebas con usuarios para medir la facilidad de uso.

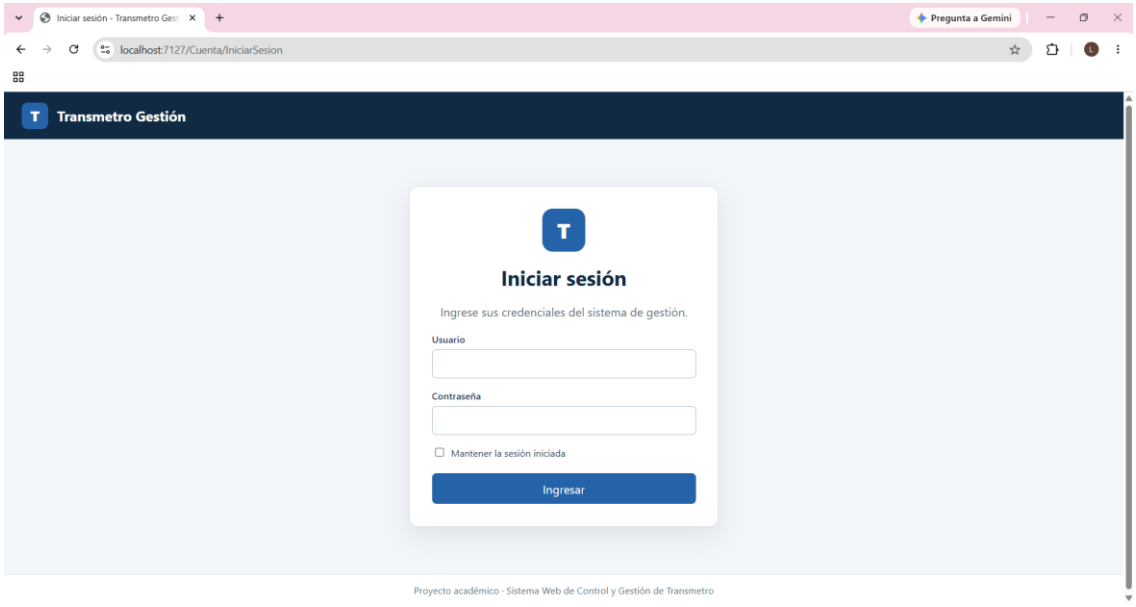
Frecuencia de evaluación

Las métricas de avance, tareas y defectos se revisarán al finalizar cada sprint. Las métricas de rendimiento, seguridad, usabilidad e integridad se evaluarán antes de cada entrega importante. Después de publicar el sistema, la disponibilidad, los errores, las transacciones y los respaldos deberán monitorearse continuamente.

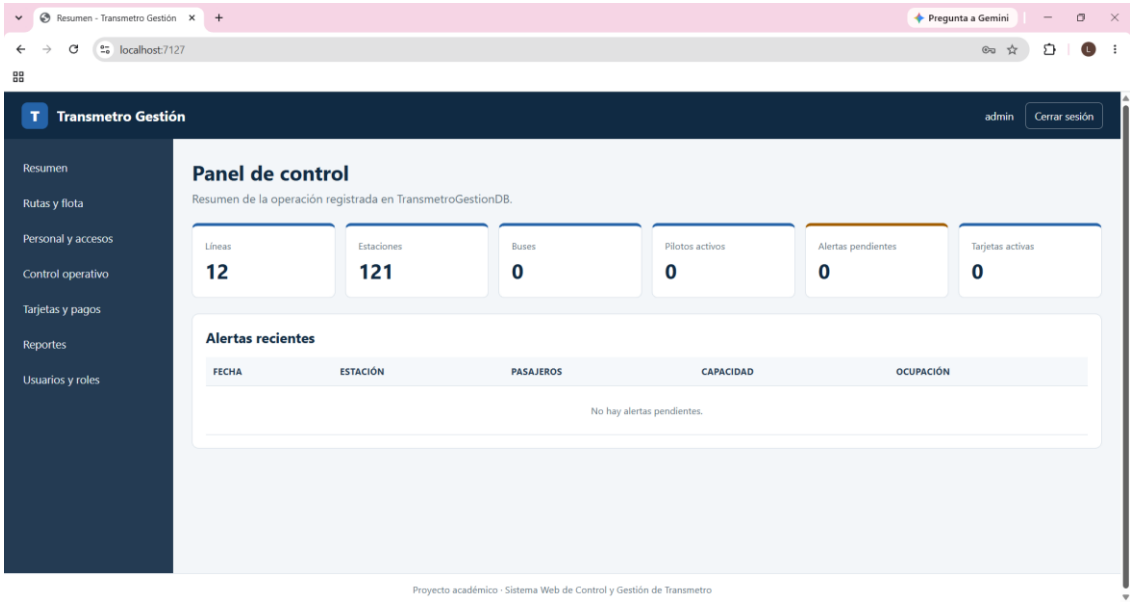
La aplicación de métricas permitirá evaluar el Sistema Web de Control y Gestión de Transmetro mediante resultados objetivos. Con ellas se podrá conocer el avance real del proyecto, comprobar la calidad del código, medir el rendimiento, controlar los errores y verificar la seguridad de la información. Además, facilitarán la mejora continua y ayudarán a garantizar que el sistema cumpla con los requerimientos establecidos.

14. DEMO FUNCIONAL – APLICACIÓN WEB

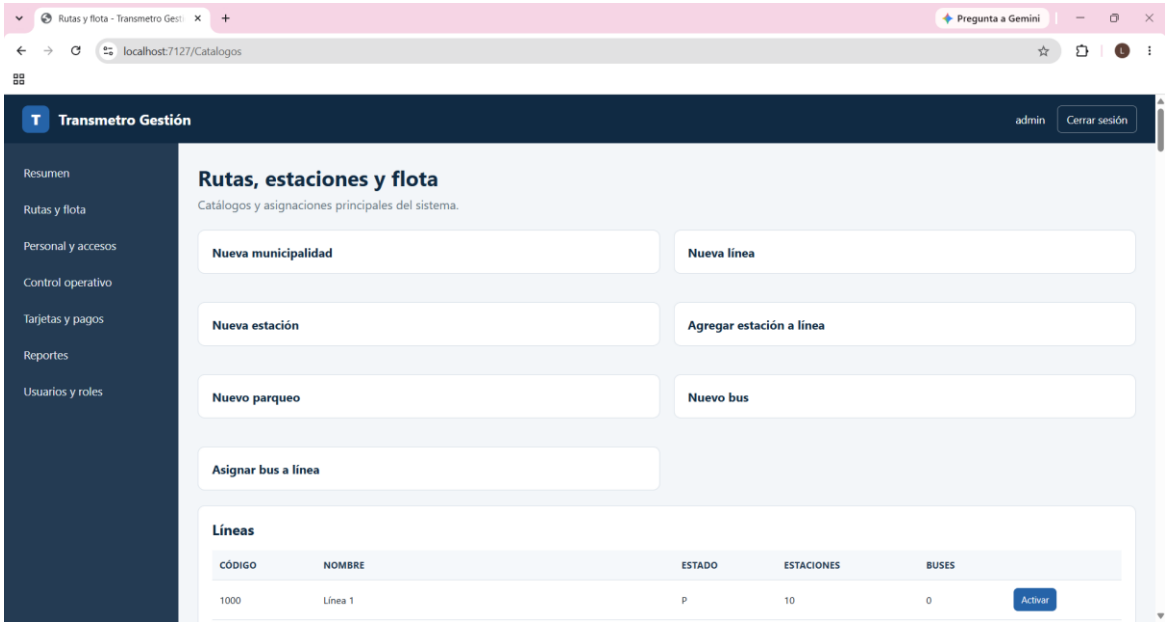
Login del sistema Transmetro Gestion



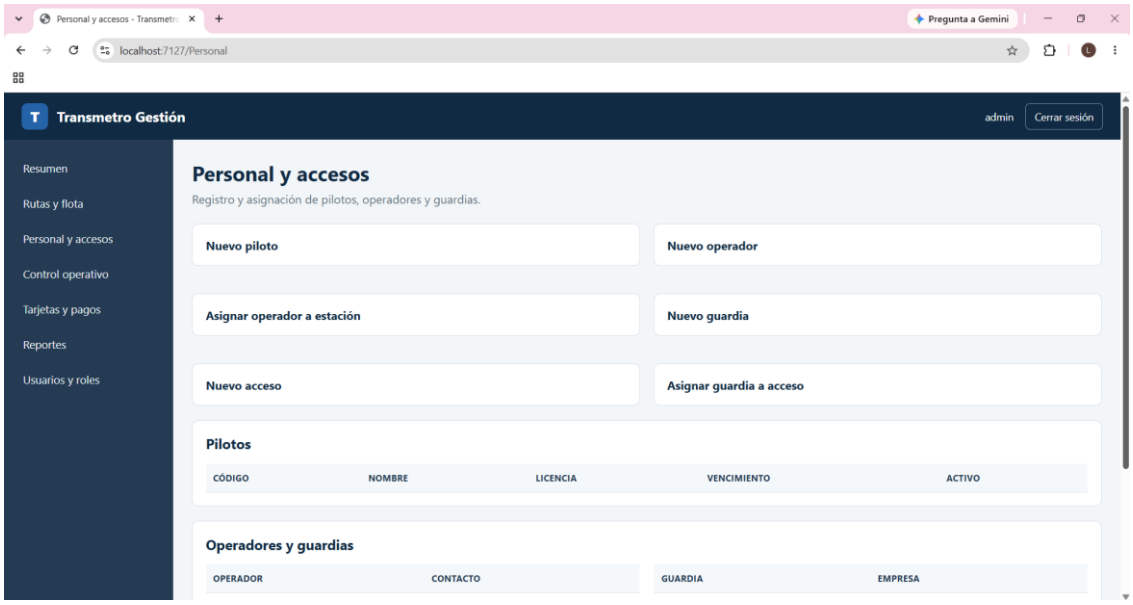
Panel de Control o Dashboard



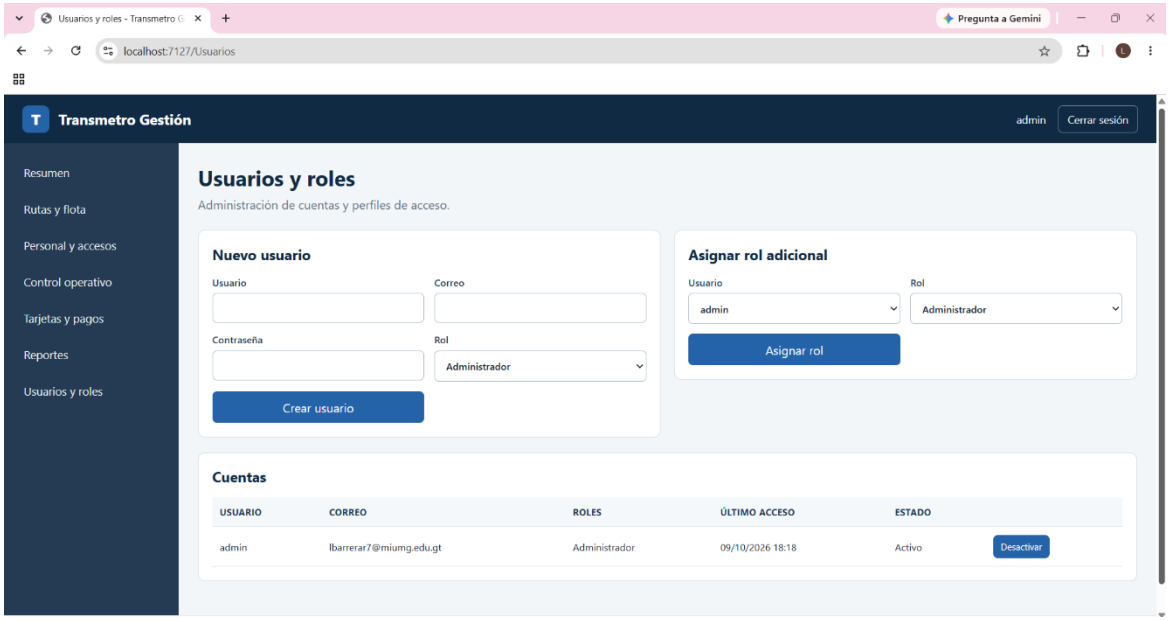
Módulo de Rutas, Estaciones y Flota



Modulo de Personal y Accesos



Modulo de Usuarios y Roles



15. PLAN DE PRUEBAS

El Plan de Pruebas del Sistema Web de Control y Gestión de Transmetro establece la forma en que se verificó el funcionamiento de la solución y organiza los escenarios que permiten repetir su evaluación. Se toma como referencia la versión final, integrada por la aplicación ASP.NET Core MVC, su base de datos SQL Server y los scripts de actualización correspondientes. Las pruebas funcionales de la versión final fueron realizadas por el responsable del proyecto, quien confirmó que el sistema funciona correctamente dentro del entorno utilizado para la entrega académica.

Este apartado documenta los objetivos, el alcance, la preparación de datos, los procedimientos de comprobación y los resultados esperados de los casos principales. De esta manera, la revisión no se limita a observar pantallas, sino que permite comprobar relaciones, restricciones y cambios de estado desde la configuración inicial hasta la finalización de un recorrido y la consulta de sus reportes.

Objetivo general

Verificar que el Sistema Web de Control y Gestión de Transmetro permita administrar la infraestructura, el personal, las tarjetas y la operación de los buses, aplicando las reglas de negocio y preservando la integridad de la información durante el flujo completo de trabajo.

Objetivos específicos

Comprobar el registro, consulta, modificación y desactivación de los elementos de infraestructura y personal, respetando sus dependencias.

Validar las asignaciones de estaciones, buses, pilotos, operadores y guardias, evitando duplicidades y relaciones incompatibles.

Verificar el inicio del recorrido, la secuencia de llegadas y salidas, el cálculo de pasajeros y los cambios de estado de la unidad.

Comprobar la generación de alertas de estación y la espera de cinco minutos cuando la ocupación del bus es inferior al porcentaje establecido.

Validar la emisión de tarjetas, la actualización del saldo, las recargas, los pagos y la separación entre el acceso del personal y el portal ciudadano.

Comprobar las validaciones del servidor, los permisos, la búsqueda, la carga paginada, los reportes independientes y la trazabilidad de operaciones.

Alcance de las pruebas

Las comprobaciones abarcan los módulos de Resumen, Rutas y flota, Personal y accesos, Operación y recorridos, Registro de tarjetas, Usuarios y roles, Reportes y el portal ciudadano. Se consideran tanto las operaciones correctas como los intentos que deben rechazarse, por ejemplo, una licencia vencida, un conteo imposible, una tarjeta sin saldo o un dato personal inválido.

El seguimiento del bus se evalúa a partir de las llegadas y salidas registradas por el personal. El mapa ciudadano se considera un recurso de consulta de líneas y estaciones. La aceptación académica no incluye rastreo mediante dispositivos GPS, integración bancaria, validación oficial del DPI, instalación de equipos municipales ni certificación de disponibilidad en producción.

Ambiente y herramientas de comprobación

La solución se verifica con la aplicación ejecutándose sobre .NET 8 y conectada a SQL Server mediante Entity Framework Core. Visual Studio permite compilar y depurar el proyecto; SQL Server Management Studio permite consultar los registros y comprobar restricciones, procedimientos y resultados almacenados. La interacción se realiza desde un navegador web y la impresión se revisa mediante la vista previa del navegador.

Antes de repetir las pruebas debe comprobarse la cadena de conexión, la existencia de la estructura y de los permisos iniciales, y la aplicación de la actualización V3 de la base de datos. V6 incorpora validaciones en la aplicación y no requiere una nueva modificación del esquema respecto de V3. Para iniciar sin datos operativos se conserva la configuración técnica necesaria para crear al administrador y acceder al sistema.

Responsables y organización de la ejecución

Luis Carlos Barrera Ramírez, como responsable del desarrollo, realicé la comprobación funcional, la corrección de incidencias y la revisión de la versión final. La docente o persona evaluadora podrá reproducir los escenarios y revisar el cumplimiento de los criterios de aceptación durante la presentación. La firma de conformidad corresponde a quien efectúe la recepción y evaluación del proyecto.

Las pruebas se organizan en preparación, ejecución, comparación del resultado, registro de observaciones, corrección y repetición. Después de una corrección se comprueba

nuevamente el caso afectado y las operaciones relacionadas, con el propósito de evitar que una mejora perjudique una función que ya había sido validada.

Tipos de pruebas aplicables

Las pruebas funcionales comprueban lo que el usuario puede registrar y consultar. Las pruebas de integración revisan que una operación complete sus cambios en los módulos y tablas relacionados. Las pruebas de valores límite verifican condiciones como el 150 % de ocupación de una estación, el 25 % de ocupación del bus, los ocho dígitos del teléfono y la longitud permitida del DPI.

Las pruebas de seguridad funcional revisan la autenticación, los permisos y el rechazo de datos inválidos enviados directamente al servidor. Las pruebas de regresión comprueban nuevamente el flujo después de las correcciones. La revisión de usabilidad y presentación considera la navegación fija, las listas bajo demanda, las búsquedas y la impresión de un solo reporte. Estas comprobaciones no equivalen a una auditoría especializada de penetración ni a una prueba de carga masiva.

Datos y condiciones de entrada

Como escenario reproducible se utiliza una municipalidad, una línea y tres estaciones denominadas Centro, Mercado y Terminal. Cada estación tiene capacidad de referencia de 20 personas. Se registra un parqueo con capacidad suficiente y se crean tres buses de 40 pasajeros. Se preparan tres operadores, un guardia por cada acceso que se active, un piloto con licencia vigente y un ciudadano con una tarjeta de Q20.00 de saldo inicial.

Las estaciones se asignan en los órdenes uno, dos y tres; se registran las distancias entre las consecutivas y se configuran cero, cinco y cinco minutos desde la estación anterior. Antes de iniciar un viaje se asignan los responsables, se activan las estaciones y la línea, y se vincula el piloto a un bus disponible. Los valores numéricos de este escenario son datos de prueba y no mediciones de la operación real del Transmetro.

15.1 Casos de prueba de acceso y configuración

CP-01 Creación inicial y acceso del personal

Referencia de requerimientos: RF-18 y RNF-01.

Procedimiento: Con la estructura inicial instalada, crear la primera cuenta administrativa e ingresar. Probar también una contraseña incorrecta y una solicitud a un módulo sin permiso.

Resultado esperado: El administrador accede a las opciones autorizadas; las credenciales incorrectas y los accesos no autorizados son rechazados. Una cuenta de empleado no sustituye la sesión ciudadana.

CP-02 Usuarios y roles

Referencia de requerimientos: RF-18.

Procedimiento: Crear una cuenta con usuario y correo válidos, contraseña de al menos ocho caracteres y un rol activo. Ingresar con ella y verificar los módulos disponibles.

Resultado esperado: La cuenta se almacena con su rol; sus permisos gobiernan el acceso. La contraseña se conserva mediante hash y sal, no como texto visible.

CP-03 Municipalidad y catálogos

Referencia de requerimientos: RF-01, RF-03 y RF-11.

Procedimiento: Registrar una municipalidad, una línea y las tres estaciones. Consultar sus fichas, corregir un campo y probar un código repetido donde exista una restricción de unicidad.

Resultado esperado: Se conservan las relaciones y las correcciones válidas; el código duplicado no produce un segundo registro. Los registros en preparación permanecen pendientes hasta cumplir sus requisitos.

CP-04 Orden y pertenencia de estaciones

Referencia de requerimientos: RF-02 y RF-12.

Procedimiento: Asignar las estaciones a la línea en orden consecutivo, configurar los minutos y registrar las distancias. Intentar duplicar una relación y modificar el orden desde la ficha.

Resultado esperado: La estación pertenece a la línea indicada sin duplicar la misma relación. Se conserva una secuencia definida; la configuración nueva se aplica a nuevos viajes y se revisan las distancias afectadas por una reorganización.

CP-05 Flota y límites de activación

Referencia de requerimientos: RF-05 y RF-06.

Procedimiento: Crear los buses y asignarlos a la línea de tres estaciones. Comprobar la activación con dos, tres y seis buses, y el rechazo de un séptimo bus asignado a esa línea.

Resultado esperado: La línea requiere entre tres y seis buses para quedar activa. Puede prepararse pendiente con menos de tres, pero no activarse. Un bus sólo tiene una asignación vigente de línea.

CP-06 Parqueos y distancias

Referencia de requerimientos: RF-07 y RF-12.

Procedimiento: Registrar el parqueo antes de crear el bus, asociarlo y consultar la ficha. Configurar 1.500 km y 2.000 km entre las estaciones consecutivas.

Resultado esperado: El bus conserva un parqueo válido y se respetan sus reglas de capacidad. El resumen de la línea presenta 3.500 km para las distancias del escenario; no se aceptan distancias no positivas.

CP-07 Personal y validaciones

Referencia de requerimientos: RF-09, RF-15 y RF-20.

Procedimiento: Registrar pilotos, operadores y guardias; agregar educación del piloto. Probar José Ángel y De León, un nombre con números, teléfonos de siete, ocho y nueve dígitos, y una petición directa con información inválida.

Resultado esperado: Se aceptan letras con tildes, ñ y espacios en nombres y apellidos personales. Se rechazan números y signos no permitidos. El teléfono informado contiene exactamente ocho dígitos; los opcionales pueden quedar vacíos. El servidor vuelve a validar antes de guardar.

CP-08 Accesos y responsables

Referencia de requerimientos: RF-04, RF-08, RF-10 y RF-15.

Procedimiento: Crear un acceso por estación, asignar guardia y turno, y vincular un operador responsable a cada estación. Intentar activar el acceso sin guardia y la estación sin operador o sin línea.

Resultado esperado: Se exige guardia para activar el acceso y se exige operador y línea para activar la estación. La ficha permite identificar la entrada física, su estación y su personal asignado.

CP-09 Activación y edición protegida

Referencia de requerimientos: RF-01, RF-03 y RF-20.

Procedimiento: Con estaciones, responsables, buses y distancias completos, activar las estaciones y la línea. Abrir un registro, modificarlo desde otra sesión e intentar guardar la revisión anterior.

Resultado esperado: La activación valida la preparación de los registros. El editor conserva identificadores y fechas automáticas y rechaza una revisión desactualizada para evitar sobrescribir cambios ajenos.

15.2 Casos de prueba de operación y recorridos

CP-10 Filtro de línea y asignación del piloto

Referencia de requerimientos: RF-05, RF-09 y RF-20.

Procedimiento: Seleccionar la línea en Operación y recorridos. Asignar el piloto vigente a un bus disponible. Intentar asignarlo simultáneamente a otra unidad o utilizar un piloto con licencia vencida.

Resultado esperado: Sólo se ofrecen buses de la línea y pilotos disponibles. Se rechazan asignaciones incompatibles; el personal ocupado no queda disponible para otra unidad.

CP-11 Inicio y seguimiento del viaje

Referencia de requerimientos: RF-02 y RF-20.

Procedimiento: Iniciar el recorrido con la línea y las estaciones activas y con piloto asignado. Revisar la siguiente estación, el destino y las estimaciones; contrastar el portal ciudadano filtrado por línea.

Resultado esperado: El viaje queda en curso y el bus en ruta. Se crean las paradas en su orden; las estimaciones se actualizan con los reportes operativos y se muestran únicamente los buses correspondientes a la línea consultada.

CP-12 Llegada conteos y salida

Referencia de requerimientos: RF-13, RF-14 y RF-20.

Procedimiento: En Centro registrar llegada y luego cero bajas, veinte subidas y dos personas esperando. Intentar salir antes de guardar el conteo y registrar una cantidad de bajas mayor a los pasajeros a bordo.

Resultado esperado: La salida requiere llegada y conteo previos. A bordo quedan veinte pasajeros. Se rechazan bajas imposibles, valores negativos, ocupación superior a cuarenta y un segundo conteo para la misma parada.

CP-13 Alerta de ocupación de estación

Referencia de requerimientos: RF-13.

Procedimiento: Con capacidad de veinte personas, probar conteos de veintinueve, treinta y treinta y cinco personas esperando. Consultar Alertas de estación para la línea seleccionada y registrar su atención.

Resultado esperado: Los porcentajes son 145 %, 150 % y 175 %. Se genera alerta desde 150 %, inclusive, y se indica la necesidad de una unidad adicional. La atención queda registrada; atenderla no despacha automáticamente un bus.

CP-14 Conteo sin llegada de bus

Referencia de requerimientos: RF-13.

Procedimiento: Desde Conteo sin bus seleccionar una estación activa de la línea y registrar las personas esperando. Si se informa un operador, utilizar el asignado a esa estación.

Resultado esperado: Se almacena el conteo de estación sin inventar una llegada o un viaje. Se evalúa el mismo umbral de alerta y se rechazan estaciones u operadores que no correspondan al contexto permitido.

CP-15 Espera por baja ocupación

Referencia de requerimientos: RF-14.

Procedimiento: Con un bus de cuarenta pasajeros, comprobar diez pasajeros a bordo y, en otro conteo válido, nueve. Para el segundo caso intentar salir antes de cinco minutos y repetir al cumplirse el tiempo desde el conteo.

Resultado esperado: Diez pasajeros equivalen al 25 % y no generan espera por esta regla. Nueve equivalen al 22.50 % y generan cinco minutos reales. La salida se bloquea antes del plazo y se permite después; la espera se marca aplicada al salir.

CP-16 Finalización del recorrido

Referencia de requerimientos: RF-14 y RF-20.

Procedimiento: Completar llegada, conteo y salida de todas las paradas. En Terminal registrar la bajada de los pasajeros restantes; si el bus queda vacío, cumplir la espera que exige la lógica vigente antes de registrar la salida final.

Resultado esperado: Al completar la última salida, el viaje queda finalizado y el bus disponible. Se conserva el historial de paradas y pasajeros. La regla de baja ocupación también se aplica en la terminal en esta versión.

CP-17 Mantenimiento y reactivación

Referencia de requerimientos: RF-02, RF-03 y RF-20.

Procedimiento: Colocar Mercado en mantenimiento y planificar otro viaje. Comprobar su omisión y luego reactivar la estación. Revisar la ruta configurada y los viajes posteriores.

Resultado esperado: La estación inactiva se omite como próxima parada sin borrar su relación ni su orden. Los nuevos viajes vuelven a incluirla al reactivarse. En un viaje vigente no se fuerza al bus a regresar a una posición ya recorrida.

CP-18 Aislamiento e historial operativo

Referencia de requerimientos: RF-17 y RF-20.

Procedimiento: Consultar pilotos asignados, alertas, esperas, conteos, tiempos y últimos recorridos con una línea seleccionada; repetir con una segunda línea configurada para la comparación.

Resultado esperado: Cada consulta operativa muestra registros de la línea elegida. Los conteos conservan sus valores y el recorrido finalizado permanece disponible en el historial sin confundirse con el trazado general de estaciones.

15.3 Casos de prueba de tarjetas y portal ciudadano

CP-19 Titular y emisión de tarjeta

Referencia de requerimientos: RF-16 y RF-20.

Procedimiento: Registrar un titular con DPI de ocho o nueve dígitos, buscarlo por nombre o DPI y emitir una tarjeta con Q20.00. Intentar un DPI duplicado, de longitud inválida o con letras, y un número de tarjeta ya utilizado.

Resultado esperado: Se vincula la tarjeta al titular seleccionado y se registra el monto inicial. Se rechazan duplicados y DPI inválidos. La tarjeta no solicita vencimiento; la validación del DPI es de formato y unicidad, no una consulta oficial de identidad.

CP-20 Ingreso y búsqueda de tarjetas

Referencia de requerimientos: RF-16 y RNF-05.

Procedimiento: En el módulo del personal buscar por nombre y DPI, cambiar de página y actualizar un estado. En el portal ciudadano ingresar el número creado e intentar ingresar con uno inexistente.

Resultado esperado: La búsqueda conserva el filtro y muestra hasta veinticinco tarjetas por página. El portal permite el ingreso de la tarjeta válida; su navegación se muestra después de ingresar. No se crea una cuenta de empleado para el ciudadano.

CP-21 Recarga y pago de tarifa

Referencia de requerimientos: RF-16.

Procedimiento: Desde el portal ciudadano recargar Q10.00 a la tarjeta con Q20.00; después seleccionar una estación de la línea y confirmar un pago. Consultar el saldo y el historial.

Resultado esperado: El saldo pasa a Q30.00 con la recarga y a Q28.00 tras cobrar Q2.00. El movimiento identifica la estación seleccionada. Recargar y pagar se presentan como acciones separadas; cada pago confirmado produce su cargo.

CP-22 Validaciones financieras

Referencia de requerimientos: RF-16 y RF-20.

Procedimiento: Intentar una recarga negativa, un pago sin saldo suficiente y un movimiento con una tarjeta bloqueada o inactiva. Reenviar una misma solicitud financiera con su identificador original.

Resultado esperado: Los movimientos no permitidos se rechazan y no alteran el saldo ni dejan registros parciales. La misma referencia de operación no debe generar un segundo movimiento. El historial conserva los movimientos válidos y sus saldos.

CP-23 Seguimiento y mapa ciudadano

Referencia de requerimientos: RF-02, RF-16 y RNF-05.

Procedimiento: Ingresar al seguimiento, seleccionar una línea y observar un viaje activo. Registrar una llegada desde el portal del personal y volver a consultar. Abrir el mapa de líneas y estaciones.

Resultado esperado: El ciudadano consulta el último avance registrado y los buses de la línea seleccionada; el mapa está en un apartado independiente. La prueba no presupone localización GPS ni actualización cartográfica automática de estaciones creadas para la demostración.

15.4 Casos de prueba transversales

CP-24 Reportes e impresión independiente

Referencia de requerimientos: RF-17 y RNF-12.

Procedimiento: Abrir Reportes y seleccionar, por separado, Resumen de líneas, Estaciones, Buses por línea, Recorridos y Bitácora de auditoría. Utilizar la impresión de cada opción y revisar la vista previa.

Resultado esperado: Se carga únicamente el reporte elegido y se imprime su contenido sin el menú ni otros reportes. La auditoría exige su permiso específico y conserva la consulta de los últimos cien registros; Recorridos presenta el trazado de estaciones por línea.

CP-25 Carga de listas y navegación

Referencia de requerimientos: RNF-03 y RNF-05.

Procedimiento: Abrir Personal y accesos y Rutas y flota, desplegar un directorio, buscar, paginar y abrir una ficha. Cambiar de módulo y regresar. Desplazar una página extensa.

Resultado esperado: Los directorios se cargan bajo demanda con hasta veinte resúmenes por página y la ficha se solicita al seleccionarla. El menú permanece visible durante el desplazamiento. Se comprueba continuidad de uso sin atribuir un tiempo exacto no medido.

CP-26 Seguridad e integridad de datos

Referencia de requerimientos: RF-19, RF-20, RNF-01, RNF-02 y RNF-07.

Procedimiento: Revisar consultas parametrizadas, permisos y relaciones. Enviar datos personales inválidos directamente al servidor y entradas con caracteres SQL en un campo de prueba; comprobar que las tablas y registros no se alteren por una instrucción introducida como dato.

Resultado esperado: La información se trata como valores de parámetros y no como código ejecutable. Las restricciones y transacciones protegen las relaciones; los datos personales inválidos se rechazan. La prueba no constituye una garantía absoluta contra toda vulnerabilidad.

CP-27 Auditoría y regresión final

Referencia de requerimientos: RF-19, RNF-11 y RNF-13.

Procedimiento: Crear y modificar un registro cubierto por auditoría y consultar la bitácora con una cuenta autorizada. Repetir el flujo principal de configuración, emisión de tarjeta, recorrido y reportes después de las correcciones.

Resultado esperado: Las operaciones auditadas conservan la entidad, operación, registro y datos de trazabilidad disponibles. El flujo vuelve a completarse y las correcciones no impiden las funciones previamente verificadas.

Condiciones de salida y tratamiento de incidencias

Una comprobación se considera satisfactoria cuando el resultado obtenido coincide con el esperado y la consulta posterior confirma los datos almacenados. Un rechazo previsto, como impedir una salida anticipada o un pago sin saldo, representa un control correcto y no un defecto del sistema. Si se produce una falla que impide el flujo principal, afecta saldos o expone información, debe corregirse y repetirse la prueba antes del cierre.

Cada incidencia debe identificarse con el módulo, los pasos que la reproducen, los datos utilizados, el resultado esperado y el observado, su severidad, la corrección aplicada y la verificación posterior. Los errores que bloquean la operación o comprometen la integridad se consideran críticos; los que afectan una función principal, altos; y los de presentación que no impiden continuar, menores.

Resultado de las pruebas realizadas

La validación funcional final fue concluida satisfactoriamente por el responsable del proyecto, quien informó que el sistema funciona correctamente. Este resultado respalda la preparación de la entrega y la demostración de los procesos descritos. Los escenarios anteriores organizan la repetición de las comprobaciones y permiten relacionarlas con los criterios de aceptación del siguiente apartado.

El resultado se expresa cualitativamente. Los porcentajes de cobertura, los tiempos de respuesta, la concurrencia y la disponibilidad requieren registros de medición específicos; no se asignan cifras sin ese respaldo. Las metas de Métricas de Software continúan como objetivos y no como resultados certificados de la prueba funcional.

16. CRITERIOS DE ACEPTACIÓN

Los criterios de aceptación definen las condiciones observables que debe cumplir el Sistema Web de Control y Gestión de Transmetro para considerarse conforme dentro de su alcance académico. Se establecen a partir de los requerimientos documentados y de las funcionalidades de la versión final V6, de manera que la evaluación pueda relacionar cada necesidad con un comportamiento verificable.

La aceptación funcional se apoya en las pruebas realizadas por el responsable del proyecto. Para la revisión formal, cada criterio debe contrastarse con la demostración de la función, la consulta de los registros o el resultado de su caso de prueba. Los identificadores CA permiten mantener esa relación sin modificar la numeración de los requerimientos originales.

16.1 Criterios de infraestructura y administración

CA-01 Administración de municipalidad líneas y estaciones

Se acepta cuando es posible registrar, consultar y corregir los datos de la municipalidad, las líneas y las estaciones; las relaciones se mantienen válidas y los estados distinguen la preparación, la operación activa y la inactividad.

Verificación: CP-03 y CP-09. Requerimientos relacionados: RF-01, RF-03 y RF-11.

CA-02 Orden y relaciones del recorrido

Se acepta cuando cada línea conserva sus estaciones en un orden definido, una estación puede pertenecer a varias líneas sin duplicar la misma relación y los minutos y distancias configurados se consultan desde la ficha correspondiente. La distancia total debe corresponder a los tramos registrados.

Verificación: CP-04 y CP-06. Requerimientos relacionados: RF-02 y RF-12.

CA-03 Flota y parqueos

Se acepta cuando un bus tiene un parqueo válido, puede identificarse su línea vigente y no se asigna simultáneamente a dos líneas. Para activar una línea con N estaciones debe cumplirse $N \leq$ buses asignados $\leq 2N$; la preparación pendiente no autoriza iniciar la operación.

Verificación: CP-05 y CP-06. Requerimientos relacionados: RF-05, RF-06 y RF-07.

CA-04 Personal accesos y responsables

Se acepta cuando se administran los datos de pilotos y su educación, operadores y guardias; cada acceso pertenece a una estación; y pueden consultarse sus asignaciones y turnos. La activación exige guardia en el acceso y operador y pertenencia a línea en la estación.

Verificación: CP-07 y CP-08. Requerimientos relacionados: RF-04, RF-08, RF-09, RF-10 y RF-15.

CA-05 Edición y mantenimiento

Se acepta cuando se pueden corregir los campos de negocio sin alterar identificadores y fechas automáticas, y se rechaza una revisión desactualizada. La estación en mantenimiento conserva su orden y relaciones, se omite como parada pendiente y vuelve a ser utilizable al reactivarse según el estado del recorrido.

Verificación: CP-09 y CP-17. Requerimientos relacionados: RF-02, RF-03 y RF-20.

16.2 Criterios de operación y control de pasajeros

CA-06 Asignación e inicio del servicio

Se acepta cuando se selecciona primero la línea y sólo se utilizan sus buses disponibles; el piloto está activo, tiene licencia vigente y no está ocupado en otra unidad. El viaje requiere línea activa y al menos dos estaciones activas; al iniciarlo se crean las paradas y el bus queda en ruta.

Verificación: CP-10 y CP-11. Requerimientos relacionados: RF-05, RF-09 y RF-20.

CA-07 Secuencia de la parada y cálculo de ocupación

Se acepta cuando cada parada exige llegada, conteo y salida en ese orden. Los pasajeros posteriores deben ser iguales a los anteriores menos los que bajan más los que suben. No se aceptan cantidades negativas, bajas superiores a los pasajeros a bordo, sobrecupo ni duplicación del conteo de una parada.

Verificación: CP-12. Requerimientos relacionados: RF-13, RF-14 y RF-20.

CA-08 Alertas y conteo de estación

Se acepta cuando el conteo utiliza la capacidad de referencia de la estación y genera alerta al alcanzar o superar 150 %. La alerta identifica la necesidad de una unidad adicional y

permite registrar su atención. También debe admitirse contar personas esperando sin registrar una llegada de bus; el despacho de una unidad sigue siendo una acción operativa del personal.

Verificación: CP-13 y CP-14. Requerimientos relacionados: RF-13.

CA-09 Espera de cinco minutos reales

Se acepta cuando una ocupación inferior al 25 % del bus genera cinco minutos reales desde el conteo y la salida se bloquea hasta que transcurra ese plazo. Una ocupación exactamente igual al 25 % no genera esta espera. La regla vigente incluye la terminal si la unidad queda por debajo del umbral.

Verificación: CP-15 y CP-16. Requerimientos relacionados: RF-14.

CA-10 Seguimiento cierre e historial por línea

Se acepta cuando el seguimiento refleja el último reporte, las próximas paradas y las estimaciones de la línea elegida. Al registrar la última salida el recorrido queda finalizado y el bus disponible. Los historiales, alertas, esperas y asignaciones operativas deben corresponder a la línea seleccionada.

Verificación: CP-11, CP-16 y CP-18. Requerimientos relacionados: RF-02, RF-17 y RF-20.

16.3 Criterios de tarjetas y atención al ciudadano

CA-11 Identificación del titular y emisión

Se acepta cuando el titular se identifica con un DPI único de ocho o nueve dígitos, puede buscarse por nombre o DPI y se vincula a una tarjeta con número único y monto inicial válido. La tarjeta no solicita vencimiento. El control del DPI corresponde a su formato y unicidad dentro del sistema.

Verificación: CP-19 y CP-20. Requerimientos relacionados: RF-16 y RF-20.

CA-12 Saldo recarga y tarifa

Se acepta cuando recarga y pago están separados, cada recarga válida incrementa el saldo y cada pago confirmado descuenta Q2.00 con la estación seleccionada. No se paga nuevamente por cada parada del mismo recorrido de una línea; otro pago confirmado, como el

correspondiente a otra línea, genera otro cargo. Se rechazan saldos insuficientes y movimientos no permitidos sin dejar cambios parciales.

Verificación: CP-21 y CP-22. Requerimientos relacionados: RF-16.

CA-13 Portal ciudadano y consulta de buses

Se acepta cuando el ingreso ciudadano se realiza con una tarjeta creada por el personal, el menú aparece después de ingresar y el saldo y movimientos se consultan en su portal. El seguimiento requiere seleccionar línea y el mapa se presenta en un apartado separado; el último avance proviene de los reportes operativos y no de un GPS.

Verificación: CP-20 y CP-23. Requerimientos relacionados: RF-16 y RNF-05.

16.4 Criterios transversales de calidad y seguridad

CA-14 Validación de datos personales

Se acepta cuando nombres y apellidos de personas admiten letras, tildes, ñ y espacios, y rechazan números o signos no permitidos. El teléfono informado tiene ocho dígitos; se conserva la posibilidad de dejar vacíos los opcionales. Las reglas se aplican en navegador y servidor a las altas y a los campos personales modificados, sin restringir indebidamente nombres como Línea 2.

Verificación: CP-07 y CP-26. Requerimientos relacionados: RF-20 y RNF-07.

CA-15 Acceso autorizado y tratamiento seguro de consultas

Se acepta cuando los permisos se comprueban en el servidor, las contraseñas del personal se almacenan mediante hash y sal, y los valores aportados por usuarios se envían como parámetros en las consultas. No se considera protección suficiente prohibir palabras SQL; el contenido introducido debe tratarse como datos y no como instrucciones ejecutables.

Verificación: CP-01, CP-02 y CP-26. Requerimientos relacionados: RF-18, RF-20, RNF-01 y RNF-02.

CA-16 Integridad y trazabilidad

Se acepta cuando los registros relacionados apuntan a elementos existentes, las transacciones financieras conservan la coherencia del saldo y los cambios cubiertos por

auditoría pueden consultarse con el permiso correspondiente. Las restricciones deben impedir duplicidades y las operaciones fallidas no deben dejar resultados parciales.

Verificación: CP-22, CP-26 y CP-27. Requerimientos relacionados: RF-19, RF-20, RNF-07 y RNF-11.

CA-17 Consulta e impresión de reportes

Se acepta cuando se elige entre Resumen de líneas, Estaciones, Buses por línea, Recorridos y Bitácora de auditoría, y se presenta e imprime sólo la opción seleccionada. La auditoría requiere AUDITORIA_VER y muestra los últimos cien registros. El reporte Recorridos representa el trazado; los viajes realizados se consultan en el historial operativo.

Verificación: CP-24. Requerimientos relacionados: RF-17 y RNF-12.

CA-18 Navegación búsquedas y carga limitada

Se acepta cuando la navegación permanece visible al desplazar el contenido, los directorios de Personal y Rutas cargan bajo demanda hasta veinte resúmenes por página y cada ficha se abre individualmente. Las tarjetas se buscan por nombre o DPI y se paginan de veinticinco en veinticinco sin perder el filtro.

Verificación: CP-20 y CP-25. Requerimientos relacionados: RNF-03 y RNF-05.

CA-19 Entrega y repetición de pruebas

Se acepta cuando se entrega el código final V6, la solución del proyecto, los scripts de base de datos y las indicaciones necesarias para configurar la conexión y ejecutar el sistema. La documentación debe permitir reproducir el flujo y relacionar los requerimientos con pruebas y criterios de aceptación.

Verificación: CP-27 y revisión de los archivos entregados. Requerimientos relacionados: RNF-09, RNF-13 y RNF-14.

Reglas para emitir la conformidad

La conformidad se establece cuando el flujo principal puede completarse, los controles críticos actúan correctamente y los registros consultados coinciden con las operaciones realizadas. Deben estar resueltos los defectos que impidan iniciar sesión, activar la configuración válida, operar un recorrido, conservar saldos o proteger la información. Las

observaciones de presentación que no bloqueen el uso deben identificarse y valorarse durante la revisión, sin ocultarlas bajo una declaración general de funcionamiento.

El responsable del proyecto confirmó el funcionamiento correcto de la versión final después de sus pruebas. Sobre esa base se presenta la solución para evaluación y recepción académica. La aceptación de la persona evaluadora se formaliza mediante la carta del apartado siguiente y su firma; la comprobación del desarrollador no sustituye esa decisión.

Relación con las métricas y el alcance de la entrega

Los criterios funcionales anteriores permiten emitir una decisión sobre las operaciones implementadas. Las metas cuantitativas de la sección de Métricas de Software, como respuesta máxima de tres segundos, disponibilidad del 99 % y respaldos exitosos, requieren registros específicos, un período definido y condiciones de medición. No se declaran alcanzadas únicamente porque el sistema haya funcionado durante la demostración.

La disponibilidad continua, la restauración de respaldos, el crecimiento con múltiples usuarios y el despliegue en Azure deben verificarse en el ambiente donde se utilicen. Estos aspectos permanecen como condiciones de operación y evaluación de infraestructura, y no se presentan como servicios externos ya contratados, implementados o certificados por esta entrega académica.

17. Carta de Aceptación de Proyecto

La siguiente carta formaliza la recepción y conformidad de la versión final del Sistema Web de Control y Gestión de Transmetro dentro del alcance académico del proyecto. Se presenta para su revisión y firma por la persona evaluadora; la fecha de aceptación y los datos de quien recibe se completan al momento de suscribirla.

Carta de aceptación

Guatemala, _____ de _____ de 2026.

Señor
Luis Carlos Barrera Ramírez
Responsable del desarrollo del proyecto
Presente.

Asunto: Aceptación académica del Sistema Web de Control y Gestión de Transmetro

Estimado señor:

Por medio de la presente, quien suscribe hace constar la recepción del Sistema Web de Control y Gestión de Transmetro, en su versión final, desarrollado como parte del proyecto académico de la Universidad Mariano Gálvez de Guatemala. La recepción comprende la aplicación web, el código fuente, los scripts de base de datos y la documentación que acompaña la solución.

Con fundamento en la revisión de la entrega, la demostración de sus funcionalidades y los criterios de aceptación establecidos en este documento, se expresa conformidad con el alcance funcional evaluado. El responsable del desarrollo ha informado la conclusión satisfactoria de las pruebas funcionales de la versión entregada.

La solución comprende la administración de líneas, estaciones, flota y parqueos; el registro de pilotos, operadores, guardias y accesos; las asignaciones del personal; y el control de recorridos, llegadas, conteos, salidas, alertas y esperas. También incluye la emisión de tarjetas, el portal ciudadano para saldo, recargas y pagos, la consulta del seguimiento por línea y del mapa, así como usuarios, roles, reportes y auditoría según los permisos establecidos.

La conformidad reconoce las reglas de la versión recibida, entre ellas la tarifa de Q2.00 por pago confirmado, la alerta de estación desde el 150 % de su capacidad de referencia y la

espera real de cinco minutos cuando la ocupación del bus es inferior al 25 %. Se reconoce también que el seguimiento depende de los reportes del personal y que el alcance de la entrega corresponde a una solución académica funcional.

La presente aceptación no acredita una implantación oficial en la Municipalidad de Guatemala, integración con GPS o servicios bancarios, ni certificación de disponibilidad continua en producción. Cualquier ampliación de funcionalidades, despliegue externo o servicio adicional deberá definirse y evaluarse de manera independiente.

Al suscribir esta carta se deja constancia de la recepción y aceptación de la versión descrita para los fines académicos correspondientes. Las observaciones específicas de la revisión, si las hubiera, se consignan en el espacio previsto a continuación.

Observaciones de recepción: _____

_____

_____

Atentamente,

Firma de quien recibe y acepta: _____
Nombre completo: _____
Cargo o calidad académica: _____
Institución: Universidad Mariano Gálvez de Guatemala
Fecha de firma: _____

Entregado por: Luis Carlos Barrera Ramírez
Responsable del desarrollo del proyecto
Firma: _____

18. Enlace A Repositorio En GitHub

https://github.com/Luisca04umg/Transmetro-Guatemala

19. Enlace Al Sistema En La Web

https://transmetro-luis-web-2026-gxemdbbcenf9hwcf.canadacentral-01.azurewebsites.net/Cuenta/IniciarSesion

20. Enlace Al Video Del Sistema

https://drive.google.com/file/d/1dSQmfzBqFkUSiOfxREd097N6gjZbB3On/view?usp=sharing